Ad-hoc network on Android

Rabie Khodr Jradi s072470
Lasse Seligmann Reedtz s072434

Kongens Lyngby 2010
IMM-B.Sc.-2010-37

Technical University of Denmark
Informatics and Mathematical Modelling
Building 321, DK-2800 Kongens Lyngby, Denmark
Phone +45 45253351, Fax +45 45882673
reception@imm.dtu.dk
www.imm.dtu.dk

Abstract

This report describes the development process of creating an ad-hoc protocol layer for the Android operating system and an text messenger application for Android using this layer.

There has been successfully developed an ad-hoc library that is able to create an ad-hoc network on Android and route data between arbitrary mobile devices in such a network, with the Ad-hoc On-demand Distance-Vector (AODV) routing protocol. The current supported and tested Android devices includes HTC Hero and Google Nexus One.
The developed Android application is simple, but applies the functionality of the ad-hoc protocol layer and is used as a "proof of concept".

The Eclipse Galileo Integrated Development Environment (IDE), has been used to develop both the protocol layer and the Android application in Java. Furthermore, the Android Development Tool (ADT), where used to compile the Android application against the Android 2.1 platform.

Resumé

Denne rapport beskriver udviklingsprocessen af en ad-hoc protokol lag til Android platformen og en "text messenger" applikation til Android, der udnytter dette denne protokol.

Der er med sucess blevet udviklet et ad-hoc bibliotek der er i stand til at oprette et ad-hoc netværk på en Android enhed og finde ruter for data mellem arbitrære mobile enheder i sådanne et netværk. Dette gøres ved at bruge en ad-hoc On-demand Distance-Vektor (AODV) rutnings protokol. De nuværende understøttede og testede Android enheder er HTC Hero og Google Nexus One. Den udviklede Android applikation er simpel, men bruge funktionaliteten der tilbydes af ad-hoc protokollen og bruges som et bevis på det virker.

Eclipse Galileo IDE er blevet brugt til at udvikle både protokollen samt Android applikationen. De er ligeledes begge blevet udviklet i Java. Herudover er Android Development Tool (ADT), blevet brugt til at kompilere Android applikationen op imod Android 2.1 platformen.

Preface

This thesis is a part of the mandatory requirements for acquiring the B.Sc. degree in engineering and corresponds to 15 ECTS points.

The authors of this thesis is currently students at Informatics and Mathematical Modelling, Technical University of Denmark.

The supervisor of this project is, associate professor at the Department of Informatics and Mathematical Modelling, Technical University of Denmark, Hans Henrik Løvengreen.

We want to thank Hans Henrik for letting us work with the Android operating system in combination with ad-hoc networks, Sun Microsystems for the open source project sunSPOTWorld - the source code helped us get us an idea of one way to design the AODV protocol and finally, the open source project android-wifi-tether [1] - the source code helped us to know how to configure the wireless adapter of our Android phones.

Lyngby, August 2010

Rabie Khodr Jradi Lasse Seligmann Reedtz

Contents

Abstract i

Resumé iii

Preface v

1 Introduction 1
- 1.1 Project Goal . . . 2
- 1.2 Report Structure . . . 2

2 Background Notions 5
- 2.1 Open Systems Interconnection Reference Model . . . 5
- 2.2 Wireless Ad-hoc Routing protocols . . . 9
- 2.3 Android Operating System . . . 18

3 Ad-hoc Library Requirements 21
- 3.1 Ad-hoc . . . 21
- 3.2 Specification Requirements . . . 23

4 Library Design 25
- 4.1 Design analysis . . . 25
- 4.2 Packages . . . 28
- 4.3 Concurrency design . . . 42

5 Library Implementation 47
- 5.1 Observer-pattern . . . 47
- 5.2 Sending Messages . . . 48
- 5.3 Receiver . . . 51
- 5.4 Routes . . . 59

5.5 Ad-hoc Network on Android . . . 62
5.6 Implementation Remarks . . . 64

6 Library Test **65**
6.1 Unit Test . . . 66
6.2 Functional Test . . . 68

7 Example Application: Text Messenger **75**
7.1 Specification Requirements . . . 75
7.2 Analysis . . . 76
7.3 Design . . . 77
7.4 Implementation . . . 85
7.5 Text Messenger test . . . 89

8 Improvements and Optimizations **91**
8.1 Improving the Library . . . 91
8.2 Text messenger improvements . . . 95

9 Conclusion **97**

A Workload Distribution **101**
A.1 Report . . . 101
A.2 Programming . . . 102

B Expanded UML Class Diagrams **103**
B.1 Exception package . . . 104
B.2 Etc package . . . 105

C User Manual **107**

D Tests Source Code and Printouts **109**
D.1 Library Functional Tests . . . 109
D.2 Library Unit Tests . . . 116
D.3 Text Messenger Unit Tests . . . 129

E Ad-Hoc Library Source Code **135**
E.1 Aodv . . . 135
E.2 Routes . . . 172
E.3 Udp . . . 186
E.4 Pdu . . . 190
E.5 Setup . . . 201
E.6 Exception . . . 205
E.7 Etc . . . 208

F Text Messenger Source Code **209**
F.1 Model 209
F.2 View 230
F.3 Control 243
F.4 Exceptions 245

Chapter 1

Introduction

Android is a new mobile operating system (OS), developed by the Open Handset Alliance [2] for portable devices. It is an open source operating system, meaning that all the source code, is freely available for everyone.

When using this operating system on a device, there is often a desire to communicate with one or several other portable devices. Such communication is needed if the devices run cooperative applications. Unfortunately, on a Android device, this can only be done by connecting to a central computer/router dedicated to manage connections and data traffic.

Communication with other Android devices is thus dependent on existing infrastructure. This can become a problem, if a group of people want to connect to each other in a place where no existing network is available, or the use of it is to expensive. In these situations, it would be convenient to create a local decentralized network. Decentralised networks is also known as "peer-to-peer" or ad-hoc networks. Because of the decentralized nature of such networks, there is no need of existing infrastructure to manage communication. Today's mobile technology make such network possible, since it is becoming increasingly common to have build-in antennas for wireless communication.

There are many applications which can exploit wireless ad-hoc networks: Various military operations, search-and-rescue operations, data collection for science

purposes, file/information sharing, text communication and entertainment purposes e.g. in the form of multi-player games.

From the different applications stated above, it is implicit that each wireless device should be able to communicate with any other device in the network. Since the devices are limited by the ability of the antennas to transmit data, the desired property is not necessarily guaranteed. The physical distance between two devices, can be larger than the technology's limit.
Therefore wireless ad-hoc networks, need to have a mechanism to search and establish connections, through an unknown number of intermediate devices. Such a mechanism is known as a routing protocol. The main task of a routing protocol, is then to route traffic trough other portable devices, in order to reach a desired destination. This is also known as multi-hop communication.

Wireless ad-hoc networks are typically dynamic and scalable, because of the mobility of the devices and the decentralized management. The limits of wireless mobile ad-hoc networks are typically the power supply (a battery), its computation power and small memory size. The design of a routing protocol, should therefore consider such characteristics when used on these networks.

1.1 Project Goal

The main goal of this project is to design and implement a suitable distributed routing protocol to manage the communication among many Android devices, running concurrently. For this to be possible, there has to be discovered a way to allow creation (and termination) of ad-hoc networks, with the Android OS. The second goal is to implement a simple Android application, to run on these devices, utilizing the main possibilities of the created ad-hoc network, as a "proof of concept".

1.2 Report Structure

This report describes the process of achieving these goals, through a number of chapters. In order to analyse a suitable routing algorithm for an ad-hoc network on Android devices, several subjects have to be studied. These subjects are discussed in Chapter 2 and includes understanding the Open Systems Interconnection Reference Model (also known as the OSI reference model), knowledge about the different designs and types of routing protocols, and understanding the structure of the Android OS.

After studying the background notions Chapter 3 analyses, how to design an ad-hoc protocol layer and furthermore specify the requirements. Chapters 4, 5 and 6 present the development process of the desired goal from design and implementation to testing of the functionality. These chapters thereby explain in detail, how the main goal is solved.

The development processes for achieving the second goal is afterwards described in Chapter 7. The chapter states the specification requirements for the functionality of an Android application and continues with describing the steps of design, implementation and tests.

Finally Chapter 8 deals with possible improvements of the developed ad-hoc library and Android application. Chapter 9 then wraps up all the findings and thoughts that has been dealt with throughout the chapters and concludes on these.

Chapter 2

Background Notions

2.1 Open Systems Interconnection Reference Model

The Open Systems Interconnection reference model (OSI) is a model that covers and standardises the way systems must interwork across a communication network, independent of the manufactures. [19] The way OSI does this is trough a layered architecture, where each layer provides a service to the layer above it and extends the service that the layer below it provides. [9] A model of the OSI can be seen in Figure 2.1

From Figure 2.1 the different layers can be seen. In this chapter there will be more focus on some of the layers, meanwhile others will just get at quick overview to get a better understanding of the whole.

2.1.1 Application

The application layer in the OSI model is a protocol that the provides a interface to the network for the application the user uses, the "user application". The user application uses the application layer to transmit messages over network.

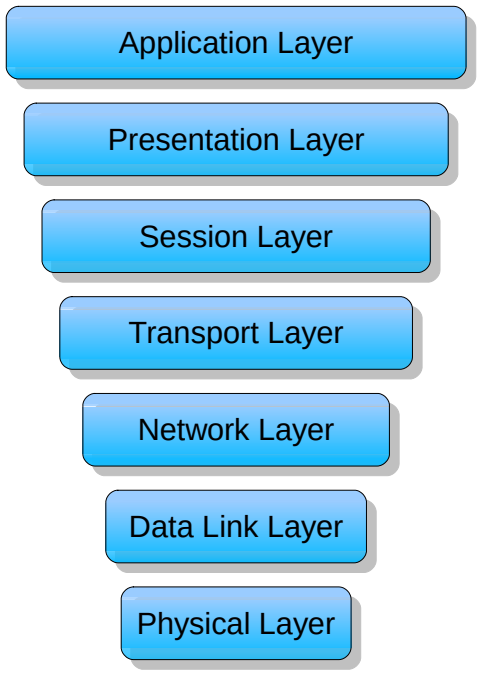


Figure 2.1: The Open Systems Interconnection Reference Model

The application layer can consist of such protocols as, HTTP, FTP, SMTP and various other protocols that can provide services for a "user application"

2.1.2 Presentation

The next layer is the Presentation layer, this is where the data from the underlying layers are transformed. The transformation is made to ensure that the application layer gets a consistent interface for receiving and sending data even if some of the underlying layers change. The transformation of data is also done to ensure that no matter what system the application layer is one a message from one application layer to another always will be readable and consistent.

2.1.3 Session

In this layer the connection between devices is handled, opening, closing and managing sessions between end user applications. The session layer is often used for remote procedure calls (RPCs).

2.1.4 Transport

This layer handles the end-to-end transfer of messages is provided. The message from the above layers are addressed, and packed with header of a packet protocol. The most commonly used protocols are UDP and TCP

UDP

UDP is a connectionless protocol, used to send and receive datagrams without acknowledgement or retries. The protocol therefore can not ensure that the packet reaches its destination, if some sort of acknowledgement is required this must be implemented in the application layer. The only reliability UDP provides is a checksum of the data, this ensures that UDP has a relatively small protocol overhead in comparison to TCP. The UDP header consists of a source port, a destination port, a length field and a checksum, when the UDP package is delivered the checksum must match and the destination port must be open on the destination computer otherwise the package will be dropped.

TCP

TCP is a more complex protocol than UDP. It is a connection-oriented protocol that uses stream communication. What this means is that the application can put an arbitrary amount of data into the stream, TCP handles the data by splitting it up before sending it and putting it back together in the same order, where as UDP only takes packages that are under 64 Kbytes [9] . TCP also ensures that lost data is resend by using an ACK protocol. Furthermore TCP handles flow control and message duplication, this ensures a stable and reliable protocol but it also means that TCP has a larger protocol overhead then UDP.

To get an idea of the difference in the package and protocol overhead of TCP and UDP there has been made benchmarks using the soap protocol, published in the article ”A benchmark on soap’s transport protocols performance for mobile

applications" [18]. The benchmarks are preformed using the soap protocol over both UDP and TCP on mobile devices showing that the package overhead on TCP and UDP is almost the same, but the protocol overhead makes the TCP more expensive eg. when sending a string the TCP protocol overhead almost makes up for half of the UPD's Total overhead.

2.1.5 Network

The network layer is where it is made possible to transfer data between arbitrary nodes[1] in the network. This can be done by using the Internet Protocol also known as IP which is used for addressing the different nodes in the network. When dealing with IP addressees there are two standards used today, IPv4 and IPv6. The most commonly used standard for local networks is the IPv4 standard, at some point the IPv6 standard proberly will take over but for now, the IPv4 is the one used. When dealing with private network addresses using the IPv4 standard, there are tree different address classes: A, B and C where A can have up to $2^{24} - 2$ hosts, B up to $2^{16} - 2$ host and C can have up to $2^8 - 2$ hosts. When talking about private networks the IP address needs to be unique within the network. Often this is handled by a DHCP(Dynamic Host configuration Protocol) server. This solution requires that one node in the networks acts as a server and that it must be reachable at all time, if a new node is to join the network. Static IP addresses can be used, but other measures must be taken to ensure that the IP addresses are unique.

Getting the packet from one node to another node where the two nodes are not neighbours requires more then just an unique IP address. If the nodes are not neighbours the packet must travel trough other nodes in the network. Finding the way for the packet requires some kind of routing in a ordinary LAN setup there are one or more routers that direct the packages in the right direction. In a ad-hoc network there is no routers therefore there has to be a build in routing protocol in the nodes, these routing protocol are discussed in section 2.2

2.1.6 Data link

The data link layer is the layer where the direct transmission between nodes that are directly connected by the physical layer, are handled. The data link layer contains a sub-layer called Medium Access Control (MAC) layer, this is

[1] The term node is used for modelling an actual network of mobile devices to a graph where nodes represents a device, and a vertex represents the ability to reach another node for exchanging data

used to provide addressing and channel access control that enables nodes to communicate in a network consisting of more then two nodes.

2.1.7 Physical

This is the hardware layer, the hardware that receives and transmits the packet in raw binary form. There are a lot of ways this is done, either by electric signals trough different wires or cables, with light trough a fibre optic or different electro magnetic waves, Wi-Fi, 3G or other radios. Some of these hardware components are described in sec. 2.3.3

2.2 Wireless Ad-hoc Routing protocols

Protocols are a formal set of communication rules which defines the behaviour of communicating nodes, to specific events [19] Section 4.3. A protocol thus consist of both defining the set of legal messages[2] that can be communicated, and how to react to these messages.

There are various ways in which ad-hoc routing protocols is designed. Ad-hoc Routing protocols are typically based on a Data Link layer protocol between nodes that are connected directly. Directly means that no intermediate node exists in the communication. With the OSI model in mind from section 2.1, a routing protocol then offers a service to a higher layer, and is based on an existing lower layer of service.

The service offered by the lower layer is a way in which nodes can communicate to each other directly. The service offered with routing protocols is the ability to communicate with nodes that is not directly reachable. In OSI terminology this layer is called Network Layer.A routing protocol thus have to manage communication routes in a network. As a consequence routing protocol connects direct one-to-one communications together into larger coherent networks with the possibility of many intermediate nodes between each communication.

The known routing protocols that exists can be divided into two main classes. These are known as Proactive Routing Protocols and On-demand Routing Protocols. In general, the difference between all types of routing protocols, are how they map the network. Some protocols store full routes to destinations, while others only know partial topology information.

[2]Strictly speaking a `data packet`

The performance of routing protocols is measured by the total needed Protocol Data Unit[3] (PDU) overhead so the protocol can function, the amount of memory it will use and the response time before messages are delivered. Battery consumption is also an important factor, that increases proportional to the protocol overhead.

2.2.1 Routing loops

A problem that routing protocols have to deal with are so called routing loops. Routing loops can occur if a node try to send a packet to a node that is not a neighbour. An intermediate node is needed to forward the packet, but if this packet has invalid route information stored, the packet may be forwarded back and forth between two nodes. The simplest network setup for which this scenario can occur, is illustrated in Figure 2.2. If node `A` wants to send a message to `C`, it

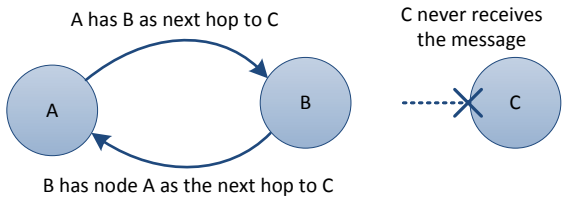


Figure 2.2: Node `A` and `B` form a routing loop.

will consult its routing information and find out that it should route it through `B`. When node `B` receives the message and then checks its information, it will find out that node `C` is reachable through `A`. A routing loop therefore exist, unless the problem is prevented or dealt with in the design of the routing protocol.

2.2.2 Routing by flooding

The most simple way of solving the problem of routing messages to the correct destination, is by a technique called *flooding*. When the need arises for any node in the network to send a message to a destination, it will broadcast the message to all neighbours. Any neighbour that is not the destination node, will also broadcast the message. The result is a flooding of the entire network. Whenever a node broadcasts a message, it will buffer that message, so that a node only will broadcast a message a single time. This is needed so that the flooding will terminate.

[3] A PDU is the data exchanged as part of a protocol

This type of protocol do not need to know any topology information. It only defines a single PDU message, which is a PDU containing the desired data that should be sent. By flooding the entire network for each message, it is easy to imagine that such strategy becomes very inefficient, especially as the network size grows. This is a consequence of not mapping the network at all. Since the nodes in a wireless mobile ad-hoc network are typically limited by the energy available, flooding is not a widespread routing protocol. This type of routing protocol may though be the only solution in *highly* dynamic network topology and high risk of lost packets. It should also be noted, that routing by flooding do not need to consider routing loops, since no routing information is kept at all.

Flooding is a technique used by many on-demand routing protocols, for discovering destinations in a network. Therefore optimizing flooding is important, in order to reduce the overhead of such routing protocol.

2.2.2.1 Expanding ring search

There exist different ways of reducing the protocol overhead in a network flooding. Some are described in [12] Section 3.2.1.

The expanding ring search is a technique that uses a TTL value (such as the hop-count) with each flooding that is initiated. The TTL is decremented at each node receiving a flood packet. If the value is non-negative the node broadcasts the packet. With a TTL value bound to each flood of a request for some destination, the ring of which that node is searched for, has the TTL as a diameter. In the process of the search, the initiating node will have to wait for a response that depends on the TTL value and the estimated time that sending a message takes.

If the initiating node does not get a response packet within that time, it will have to initiate another search request, but with a larger TTL value. Thus the name, expanding ring search. The amount that TTL is incremented for each failure may be an exponential increasing value. Should the search fail two times, the third and last search is flooded through the entire network.

2.2.3 Proactive Routing Protocols

Proactive routing protocols is characterised as the class of protocols where routes between all pairs of nodes are discovered and stored. Routes are discovered and

stored even if they may never be used.

This approach have both advantages and disadvantages. In the case of a request to communicate with an other node, the protocol will not have to initiate a route discovery. Route discovery means a search for a desired node on the network. It will be able to accommodate the request immediately.

The table which have to store all the route entries will be relatively large, and will use a lot of memory. If the network topology is highly dynamic, then this type of protocol is likely to encounter that many of its known routes becomes invalid. Thus triggering route discovery once again, if the destination is still needed.

Routing protocols that apply the proactive approach, can be divided into two types:

- Link-state protocols
- distance-vector protocols

The main difference is how these protocols share route information to other nodes in the network.

In **link-state** protocols, nodes maintain routes to every other nodes in the network, with a cost for each link. Each node in the network periodically floods the entire network with link-state updates that contain the cost of using each link. The nodes are then able to locally calculate the shortest path to each destination, such that a next-hop can be chosen for that link.

Some of the link-state routing protocols for ad-hoc networks that have been proposed are Optimized Link State Routing (OLSR) [8] and Topology Broadcast Based on Reverse-Path Forwarding (TBRPF) [14].

With **distance-vector** protocols, each node periodically broadcasts to neighbouring nodes the cost of using the best known route, for each of it known destination. The broadcast thus contains vectors for each destination, formed by a cost metric and next-hop identifier. As nodes propagate updates to neighbouring nodes, eventually all the nodes in the network will know the cost using a link for reaching every other node in the network.

Several distance-vector protocols for ad-hoc networks have been proposed. Some of the important protocols are Destination-Sequenced Distance-Vector (DSDV) [16] and Wireless Routing Protocol (WRP) [13].

The following section will describe the DSDV protocol, because of its simple way of preventing routing loops.

2.2.3.1 Destination-Sequenced Distance-Vector (DSDV)

DSDV is a distance-vector protocol, that prevents routing-loops by introducing the use of destination sequence numbers. Each node maintain it own sequence number, which can be incremented. Each message that a node transmits/broadcasts is tagged with this number. The number is used as a 'freshness' metric by any receiving node. A node will only contain a route to a destination with the higher sequence number that is bound to that destination.

The protocol periodically broadcasts an update PDU to any neighbours containing entries for every known destination. The PDU contains its own incremented sequence number, the address of the transmitting node, and all the entries. Entries are specified by a cost metric for using the link (usually the hop-count), the destination address and the last known destination sequence number. A node which may receive such an update, will then check if the freshness (sequence number) and the cost is better than the path to that destination. In that case it will use the received route instead, discarding the current information, and setting the next-hop address to the originator of that update.

The protocol distinguishes between two types of routing updates. These are *full dumps* and *incremental* routing updates. The previous PDU is called a full dump, since the entire table is broadcast. Incremental updates should fit into a single PDU, while a full dump may span over several PDUs, depending on the table size. Incremental updates are used whenever *important* routing information should be propagated, such as if a broken link is discovered.

Broken links can either be discovered by an underling service (a protocol in the data link layer), or discovered if no message has been received in a while from a neighbour. If a node discovers a broken link to a neighbour, the cost of using any entry that have that neighbour as a next hop, is set to the maximum allowed value plus one. Also the destination sequence number, that is bound to each of these broken links, is incremented with one.

Sequence number may, in general, only be incremented by the node itself, unless a broken link is discovered. In order to distinguish between sequence numbers that are incremented because of a broken link, it is specified by the protocol, that a node only increments its own sequence number to even numbers. If a node discovers a broken link it thus increments the destination sequence number of each of the broken links to uneven numbers. If the link should be re-established,

the sequence number that is tagged along the sent PDU from the broken destination, will always supersede the sequence number broken entry.

The DSDV protocol tries to reduce the protocol overhead in several ways. It estimates for example, the time for a route to stabilize. This means, that every route update is not propagated immediately upon reception, since better metrics for the same route is likely to be received later. Thus preventing unnecessary route updates from rippling through the network.

2.2.4 On-demand Routing Protocols

On-demand routing protocols are different from the proactive protocols, by not initiating any route discovery before the need arises to reach an unknown destination. As a consequence, the information known about the network is kept to a minimum, and thereby the memory used for storing route information is minimized. This is an advantage if it is known that the ad-hoc network has a high risk of topology changes. On the other hand, such a strategy may result in a long delay, because the protocol have to initiate a route discovery at the time of request.

On-demand protocols have been proposed that are designed for ad-hoc networks. These include Dynamic Source Routing (DSR) [10] and Ad-hoc On-Demand Distance-Vector (AODV) [15]. DSR is simliar to AODV in the route discovery procedure, but caches the entire route in each node. The AODV protocol knows only to its neighbour in a route. The two protocols are though very different from each other on the amount of routing information that is stored in each node. The following section, will describe in detail the AODV protocol.

2.2.4.1 Ad-hoc On-Demand Distance-Vector (AODV)

AODV is also a distance-vector protocol, but it is designed for efficient routing in highly dynamic networks. Simulation have been conducted, running AODV with a network size up to 1000 nodes ([17] Section 3.2). The protocol borrows the idea of sequence numbers from DSDV in order to prevent routing-loops from occurring.

Neighbours are detected by letting each node periodically broadcast hello messages, that only contains its network address and its sequence number. A node thereby always have routes to its neighbours. If the need arises to communicate

with an unknown destination (meaning that the node is not a neighbour), the AODV protocol initiates the route discovery procedure.

The node floods the network with a RREQ (route request). This PDU contains the source address, source sequence number (which must be incremented before each *new* RREQ), a hop-count, destination address, the last known destination sequence number and a request/broadcast ID.

If the node does not have any previous knowledge of the destination, then it sets the destination sequence number to `UNKNOWN_SEQ_NUMB`. The broadcast ID is a value that is simply incremented, for each sent request. Thus a RREQ can be uniquely identified by the pair, consisting of the source address and the broadcast ID.

Each receiving node that is not the destination, will check its routing entries for a match. If a node know a valid route, it will reply with a route reply(RREP) PDU back along the route, that the RREQ came from. Such a route is referred to as a *reverse route*. Also RREP is sent to the destination in order to insure symmetric routes. Such a message is known as a *gratuitous* RREP, though it contains the same values. A valid route is an entry that is not known to be broken or timed out, because the route is not needed any more.

A route reply is unicast to the source node, and not flooded as a RREQ. Figure 2.3 illustrate an example of how the two PDUs is disseminated in a network, if an intermediate node receives a RREQ and is able to accommodate the request.

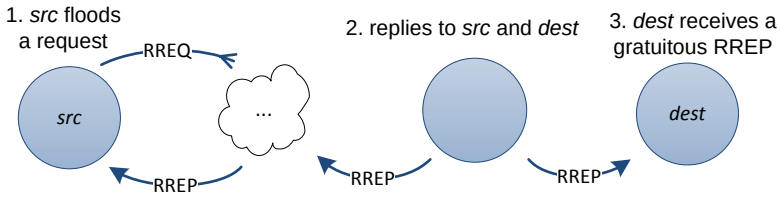


Figure 2.3: PDU dissemination in the network

A reverse route is made by letting each node create an entry to the source, with the sending neighbour as the next-hop and also incrementing the hop-count of the PDU. The information contained in the RREQ PDU is enough to create such a reverse route.

Should the destination node receive the RREQ, it will react in the same way as an intermediate node, but before replying with a RREP, it must increment its own sequence number, in order to insure that it will supersede any RREP created by other nodes. It is important, because other nodes may contain less

optimal or older routes to the desired destination. In order to prevent the originating node from receiving RREPs containing old route information about the destination, each node that receives a RREP must insure, that the PDU contains information that is either as fresh as its own entry or better. Should the RREP contain better route information for a destination, a receiving node will update that route entry.

Several nodes may succeed in sending RREPs back to the originator, meaning that they did have a fresh route. The source should handle each of the received RREPs, so that the best route is used. The best route, is the one with the higher sequence number. Should they be equal, the one with the lowest hop-count is chosen.

AODV is conservative in regard to management of the route table entries. The table only holds *one* entry per destination, which is tagged with a sequence number for that destination. Should a PDU contain a route to an already known destination, that can be reached through a different next-hop node (with a possible lower hop-count value), the information is discarded if the information is tagged with a lower sequence number for that destination.

Each entry in the table is stamped with a TTL, which is reset each time that entry is used. Entries that are not active (i.e. are not used), become stale as the TTL is not updated. Such routes are not immediately discarded, but marked as invalid. Invalid routes may not be used, but is kept, such that the destination sequence number for that destination is known for another TTL period. Routes are first removed completely, when a route is marked as invalid and the TTL is exceeded.

An invalid route may not be used, since the TTL is reached, even though the route might still be correct. As a consequence, if the TTL is set to a small value, many correct routes are invalidated (and later removed), while a too big value, results in keeping many unneeded routes.

Route entries consist of a destination address, destination sequence number and the cost of using the route. Each node thereby only knows a minimum information about the topology, but relies on each of the other nodes to know their next-hop for that destination too.

In order to know which neighbours uses the node for a destination, each node maintains a list of *precursors* for each route entry. Whenever a neighbour node wants to reach a destination through another node, it registers the neighbour as a precursor for that entry. The destination(s) that must be notified with a RRER in the case of a link breakage, is then known by the precursors list. Figure 2.4 illustrates how a RRER is sent to precursor nodes, eventually reaching the

end-nodes of each precursor.

Should a node detects a link breakage to an active neighbour (a neighbour used in a next-hop), a route error (RERR) PDU is sent along the intact part of the route. If a node detects a link breakage to a neighbour, it is likely that the neighbour node also detect this breakage eventually, because of the symmetry. Broken links are therefore cleansed in both directions, removing the route completely from the tables. Should the RERR message somehow not be received by all the precursors, a node is therefore able to also send RERR messages, should a node try to route a packet through a neighbour node that has no entry for that destination.

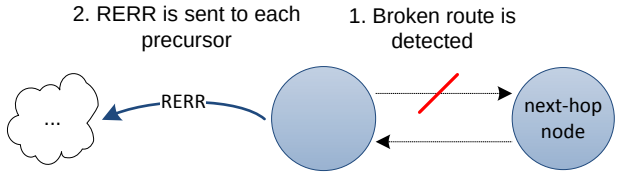


Figure 2.4: Route clean-up by use of the RERR PDU

The AODV protocol is design to be able to run correctly on unreliable connections, where packets may disappear or nodes may move away from the transition range of the medium.

It is achieved by re-flooding the RREQ PDU up to `RREQ_RETRIES`, and sending RERR messages to nodes that request routing through a node, that do not have the next-hop. A re-flood for a RREQ may be needed even if the desired destination received the request. When one or several RREPs are unicast back through the reverse route, there exist the risk of packet drops or stale reverse routes. A reverse route may exceed its TTL value so the RREP cannot be unicast back, since no assumption can be made, of the time a packet is about to travel back to the originator.

2.2.5 Location-based Routing Protocols

Most of the new mobile devices that are produced today, come with a built-in GPS antenna. It allows the device to get geographical location information through a satellite. Such technology opens for a different way of designing route protocols. Assuming that some service exist for knowing the location of destination nodes, it eliminates the need for node to exchange PDUs. With GPS information about other destinations, routing tables do not exist as known by

the previously described protocols. Scalable location-based routing protocols, may therefore be designed.

The most basic approach that a routing protocol can follow, is a so called greedy routing. Assuming that each node have the location of each desired node, by some location service, it routes its packet(s) to the destination, through the neighbouring node that is determined to be located geographically closer. Each node receiving packets to be forwarded, will follow the same behaviour.

This section will not describe protocols of this type any further, but only state that routing through the use of geographical information is a novel routing paradigm that should be considered. Several other protocol have been proposed in this area including hybrid designs e.g. the AODV protocol aided by location information [11]. Such protocols may reduce the overhead created by the route discovery procedure, by flooding within an *expected zone*.

2.3 Android Operating System

The Android operating system (referred to as Android) is a new mobile operating system, aimed at the smartphone market, but is also moving towards tablet PCs and netbooks. Android is an open source software stack consisting of a set of different layers which can be seen in Figure. 2.5 [3] below.

2.3.1 Android software stack

The bottom layer is a modified Linux 2.6 kernel, the kernel handles system services such as security, network stack, process management and memory management. Furthermore the radio drivers are in the kernel, which will be used for establishing a ad-hoc network.

On top of the Linux kernel lies two layers, the libraries and the "Android Runtime". The libraries consists of a collection of c and c++ libraries that handles data from the application framework layer, and passes it to the the kernel.
The "Android Runtime" consists of the Dalvik virtual machine (Dalvik VM) and a group of core libraries, which provides most of the functionality provided in Java. The Dalvik VM is a virtual machine that is optimised for low memory requirements, and to run multiple virtual machines at once, making it possible for each application to run on its own virtual machine, thus increasing safety

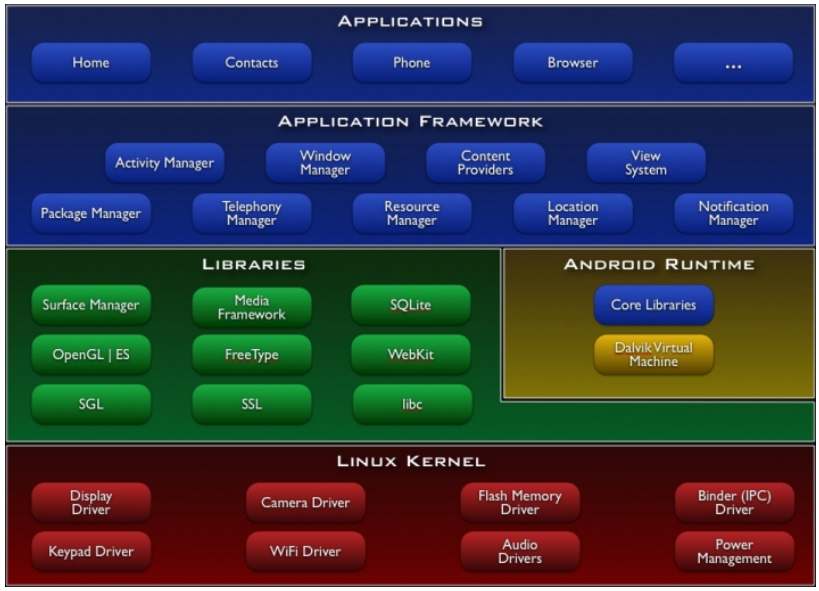


Figure 2.5: Android operation system software stack

and stability. The Dalvik VM relies on the underlying layer to handle threads, process isolation and memory management.

Above these two layers is the framework layer that is the layer between the applications and the libraries, this layer is there to making it easier for the developers by offering a higher level of abstraction then the c and c++ libraries. On the top level is the applications, which is the interface for the user .

2.3.2 Writing

Applications (app) for Android are written in Java and are compiled to ".dex" files for the Davlik VM. The principle behind the structure of an Android app is that it is build around activities, which is what the user sees. Only one activity can be visible to the user at a time, and there for the user can only interact with one activity at a time. The activity can hold many views whiche is Besides the the code that can be written i Java taking advantages of androids build in Java liberis, there can also be written som C or C++ code. The tool for this is known as the NDK "Native development kit" this lets the programmer write c/c++ code fore the platform. To use the c/c++ it must be loaded in a ordenary Java based application, and JNI can be used to parse arguments to and from the

native code.

2.3.3 Radio

Having the ability to create and connect to a ad-hoc network requires a radio transmitter and receiver and on most Android devices have tree different options to do wireless connections .

The 3G antenna is used for transmitting and receiving data over a grate distance by using the infrastructure provide bye the phone operators. But this option is not build for direct connection between two mobile phones.

Blue-tooth is a standard developed bye Ericsson, and is intended for creating ad-hoc networks between mobile devices in an easy and safe way. With blue-tooth an ad-hoc network can be created called a picho net, this is a network where there is a master and up to seven slaves where all the data must go trough the master. If a bigger network is required a node can be an slave in one piconet and a master in another but not at the same time, so it must switch back an fourth from being master to slave.

The most common WIFI standard in Android devices is the on based on IEEE 802.11g standard. This standard transmits on the 2.4 GHz band and can transmit up to 54 Mbit/s [19] Section 5.3.2. The IEEE 802.11g supports the ad-hoc architecture, that allows the nodes to communicate directly with other devices in range, either bye sending direct messages, broadcasting or multi-cast using the IP standard described in chapter 2.1. The WIFI antenna always sends the message out in all directions, with a approximated range of 25 m (inside) to 300m(outside) [19] . The Android API does not at this point support ad-hoc mode for this to work there must be written some lower level C/C++ code that manipulates with the Linux kernel layer.

Chapter 3

Ad-hoc Library Requirements

In this chapter the general requirements, and overall design choices fore a ad-hoc network are are analysed and discussed to find the best design. The over all design will also be view in relations to the OSI model as well as the Android system model.

The purpose of making a application that can create and manage a ad-hoc network is to enable a application to be able to use this network. Instead of making the application itself provide this service, a smarter way is to make a library that provides the necessary services for the application. The advantages of making a library is the it provides a higher abstraction level for the application, bye providing a specific interface, that the application can use, and therefore it does not have to deal with the underlying problems. Furthermore by crating a library the code can be reused in many applications simply by including the library.

3.1 Ad-hoc

To clarify what a ad-hoc network is, and witch services i should provide, once it is implemented on a Android device. Hence Android is a mobile platform the Android device should be able to move and change its position in the network.

For this to be true, multi-hop communication must also be possible, otherwise two devices will lose connection if one of them moves out of direct connection with the other device. With multi-hop a packet can travel trough other devices in the ad-hoc network and reach its destination. In ad-hoc mode there is no routers, only the mobile phones connected to the ad-hoc network, so in order to have a successful multi-hop there must be implemented a routing algorithm. This routing algorithm must be able to handle a dynamic changing network. A more detailed analyse of the right routing protocol is in chapter 4.

For the routing algorithm being able to work, there has to be some layers below that can handle the sending and receiving of packets between neighbouring devices. These layers are described in Chapter 2.1, the two lowest layers, data link- and physical layer. For the physical layer there are some different options witch are described in Chapter 2.3, but there has to be some sort of setup for the physical layer.

From the things discussed there can be made a guiding system design, witch can be seen on Figure 4.1. Building the library up in layers ensures that a layer can be swapped out with out effecting the other layers, taking a example the data link layer could be switch from one connection form to another without effecting the other layers, of cause the hardware support fore this connection form must be there.

When looking at the OSI model in Chapter 2.1 in reference to the overall system design in Figure 4.1, a parallel can be seen. The first layer from the top is the application layer, this is where the Text Messenger (specified in Chapter 7) application is. The application relies on the underling layers to be able to function.

The OSI model does not match the ad-hoc library but some of the core ideas can be taken, shown in Figure 3.1. The model can be split op into some layers that corresponds to the OSI model. The Figure 3.1 shows the different layers. At each layer the data sent, gets a new header that each offers an extra service. The bottom layers is the physical layer just as the OSI model states. On top of the physical layer the data link layers is placed. This is also the case in the OSI model. The data link layer will add or remove its header depending on which way the data is going. The data links header ensures that the package can get from one neighbour device to another.

Above the data link layer is the Routing protocol layer corresponding to the Network layer in the OSI model. This layer ensures that the data send from the above layer is transferred to the right node, even over multiple intermediate nodes and that the data will not get up to the layers above if the node is not the destination. Above the Routing protocol layer is the application layer. This

means that some layers in the OSI model has been skipped, leaving out some services. The application has its own header for the data sow it can handle the data received in the right way.

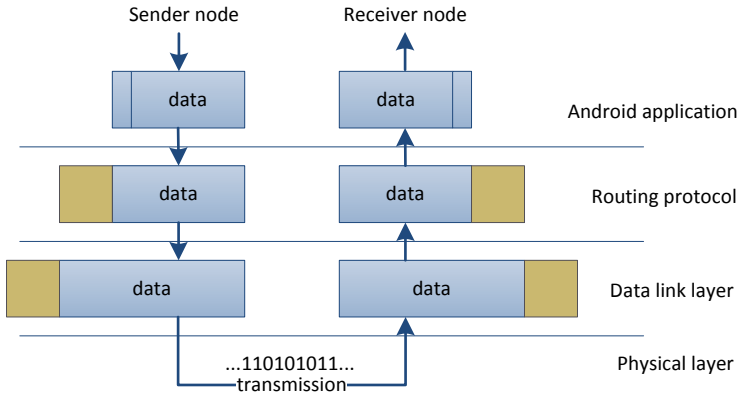


Figure 3.1: The PDU packing layers

Now looking at overall design guidelines in reference to the Android OS design, shown in Figure 2.5, it is seen that also here the ad-hoc library and the application spans over different layers. First looking at the application that uses the Library, this application is placed in the top layer, the application layer. When looking at the library it spans over more then one layer and the ad-hoc library's network- and data link layer is placed in the Android OS's Application Framework layer decried in section 2.3 this layer is just beneath the application layer, and provides the application with services. The setup layer that enables the ad-hoc mode in the device must be written in C/C++ and placed in the Libraries layer, making i able to manipulate with the drivers for the wireless radio.

3.2 Specification Requirements

The main requirement is to implement a *wireless* ad-hoc network on Android mobile devices. Thus a wireless technology and a routing protocol should be chosen that is suitable for the characteristics of ad-hoc networks consisting of mobile devices.

By the analysis of this chapter, it is a requirement that the design of the ad-hoc network is structured as a library, so that other potential developers are able to utilize it. The interface of the library should offer the following functionalities:

- Start/join an ad-hoc network
- Terminate/leave an ad-hoc network
- Send data to a specific destination, that may require multi-hop communication
- Broadcast data to all neighbours
- Pass notification messages to an application using the library, about any relevant events in the network

A notification to the application layer should be given if:

- Data is received from another node in the network
- The node is not able to discover a desired destination on the network
- The data is sent successfully from the device
- The desired destination to discover is not a valid destination in the network
- A route to a destination is discovered

The notifications should contain some way of identifying what event happened along with additional data if necessary. In the case of a received message, the data would contain the actual message received.

The applications that may utilize the library may be very different, thus it is required that the library reflects this throughout the development steps.

Chapter 4

Library Design

This chapter will present how the library is designed in order to meet the discussion of chapter 3.

4.1 Design analysis

The following sections will analyse the different possible solutions that might exist, and argue for the chosen solution.

4.1.1 Routing Protocols

As described in chapter 2 section 2.2, the existing routing protocols can be divided into three classes. Proactive, on-demand and location-based routing protocol. By the description of the different routing protocols, it seems that an on-demand routing protocol is to prefer. This is reasoned to say since an ad-hoc network consisting of portable devices, will result in a high change in the network topology. It is therefore preferable that the routes held in each device, only consist of the routes *needed*. Since only the needed routes are

stored, it is much less likely for a route to break because of topology changes. Mobile devices typically have relatively little main memory available, so having to store routing tables cached with much or all of the network connectivity is inappropriate especially as the network size grows.

With the high network topology changes with mobile ad-hoc networks, it would be suitable to implement the AODV protocol because of its conservative use of routes, and the minimum mapping of the network.

The theoretical performance bottleneck of AODV, is the route discovery procedure, that requires network flooding for each route request. This bottleneck can be optimized as described in Section 8.1.

The reason why location-based routing protocols are not suitable for Android mobile devices, is because of its inaccurate location service (up to 500 meters inaccuracy has been experienced). The geographical information is especially inaccurate when the device is used within buildings.

Location-based or aided routing protocols is though an interesting paradigm and may become useful to implement as technology improve.

4.1.2 Wireless Technologies

Two widespread wireless technologies exist with today's smartphones. These are Bluetooth and Wi-Fi (IEEE 802.11g). Both are supported by the Android OS through its API. 3G is also very common, but since this technology relies on an existing infrastructure, it will not apply with the idea of having an independent ad-hoc network.

As described in section 2.3, Bluetooth is developed specific for ad-hoc networking and low energy consumption. In order to have a true ad-hoc network with Bluetooth, each node should both act as a slave and a master, concurrently. This could be achieved by having two threads.

The disadvantages of using Bluetooth, is the short reachability and the small bandwidth. Also it is not clear from the Bluetooth protocol stacks, how broadcasting is possible. The supported transport protocol on Android is RFCOMM [4], which is a reliable connection-oriented protocol. As a consequence broadcasting to neighbouring nodes is not possible with this protocol.

It is a major drawback for routing protocols such as AODV. For example, the Route discovery procedure in AODV, relies on the ability of flooding the net-

work. Since it also is a requirement that the library offers a way of broadcasting application messages, Bluetooth as a wireless technology on Android, cannot be used.
Wi-Fi offer a longer communication range as well as a larger bandwidth, but consumes more battery as a consequence. Since the library should be designed as general as possible, to accommodate a variety of applications, it is therefore reasonable to use Wi-Fi as the wireless technology.

4.1.3 Transport Protocols

Section 2.1 in chapter 2 describes two protocols in the transport layer of the Internet protocol stack. They are known as TCP and UDP respectively, and are both supported in the Android API. The following will discuss which of the two protocols is best suitable for `direct` communication between two devices.

For an ad-hoc library on Android, using TCP to transmit application messages is possible and would mean that packages is guaranteed to reach their neighbour destination.

TCP is not suitable though, as the transport protocol for transmitting the PDUs of the routing protocol, since it is connection-oriented. Meaning that no broadcasting is available. Furthermore TCP has more overhead compared to UDP, because it has to guarantee that the data stream gets delivered and because of the set up phase of the connection.

Using UDP in an ad-hoc network means that the design of the routing protocol should consider that application messages as well as PDUs can fail to reach a destination. UDP is suitable for the requirements of the library, because it is a connectionless protocol. It offers a primitive way of broadcasting packets, and unicasting single datagrams to neighbouring nodes.

Both UDP and TCP follow the server/client model, which is similar to the master/slave with Bluetooth. To create a true ad-hoc network, each node thus has to have two threads (a server and a client), so that every node is able to send and receive data concurrently.

4.1.4 Configuring The Wireless Adapter

In order to create a wireless ad-hoc network on any operating system, the wireless network adapter must be accessed and reconfigured. This is not allowed

on Android mobile devices with factory settings. Since Android is a modified Linux kernel, developers have been able to run the OS as the administrator with super user rights. Numerous web-pages exists, explaining how to do this, since it is dependent on the device and the OS version. Some low level programming language like C or C++ is needed in order to run system commands with super user permission, when the device is run as administrator. Section 2.3 in chapter 2 described that the Android OS offers native C libraries, that can be accessed through the JNI, so that it is possible to configure the wireless adapter.

When configuring the wireless adapter, it should be set to run in *ad-hoc mode* instead of *infrastructure mode*. Also to join/create a network the SSID has to be set. Finally a netmask should be agreed upon and a static unique IP-address should be set. A static IP-address is needed since no DHCP server can be in an ad-hoc library. How to determine if an IP-address is unique across an ad-hoc network is not the main focus in this project. Chapter 8 will discuss this topic further.

4.2 Packages

The ad-hoc library can be seen as divided into several layers, containing one or more packages, so that each is responsible for some smaller part. Figure 4.1 shows how the ad-hoc library spans over several layers and how the packages is located in each layer.

The application layer requires that an underlying routing layer is offered and that the application is running on a ad-hoc network. From Figure 4.1 it is clear that the application layer abstracts away from *how* this is achieved. Similar abstraction exists between the other underlying layers. The routing layer requires for example, the ability to communicate with neighbours by also utilizing an underlying layer.

Figure 4.1 only show which packages exist in each of the layers. The packages themselves consist of several classes, that are coherent across layers. Figure 4.2 gives a compact UML class diagram of all the classes, that are in the packages of the library.

The class diagram of Figure 4.2 show how the classes are coherent across the packages and between classes of same packages. Generally it can be said that classes across packages are loosely coupled, except `Receiver` and `UdpReceiver`. There exist an bidirectional association between these two classes, which will be explained in the sections that follow.

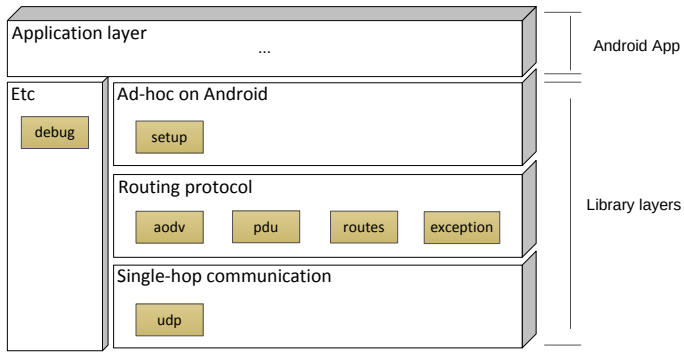


Figure 4.1: Shows the placement of the packages

4.2.1 Aodv

The `aodv` package is responsible for the main functionality of the AODV protocol, such as cleanse the tables of stale routes and handling library packets. The handling of PDUs, thereby defines the behaviour of the AODV protocol.

The following sections will describe each of the classes seen in Figure 4.3 and the coherence between.

4.2.1.1 Node

`Node` is the interface between the routing protocol and the application layer, by having methods for sending and receiving arrays of bytes. The `sendData()` method is used for sending packets. Because this class acts as the interface, it is also responsible for managing notification of events through an observer-pattern. It therefore extends `Observable`. Also it implement the `Runnable` interface, such that the observer-pattern is run on a separate thread. Section 4.3.1 will discuss this further.

In order to notify the application it defines two internal public classes called `ValueToObserver` and `PacketToObserver`, that both inherit from an interface called `MessageToObserver`. The interface defines two methods that enables the application layer to know which type of event happened and the belonging value. `Node` also manages the sequence number of the node, and lets other classes from the package retrieve it, through get-methods.

The application layer must create an instance of this class to use the library. The

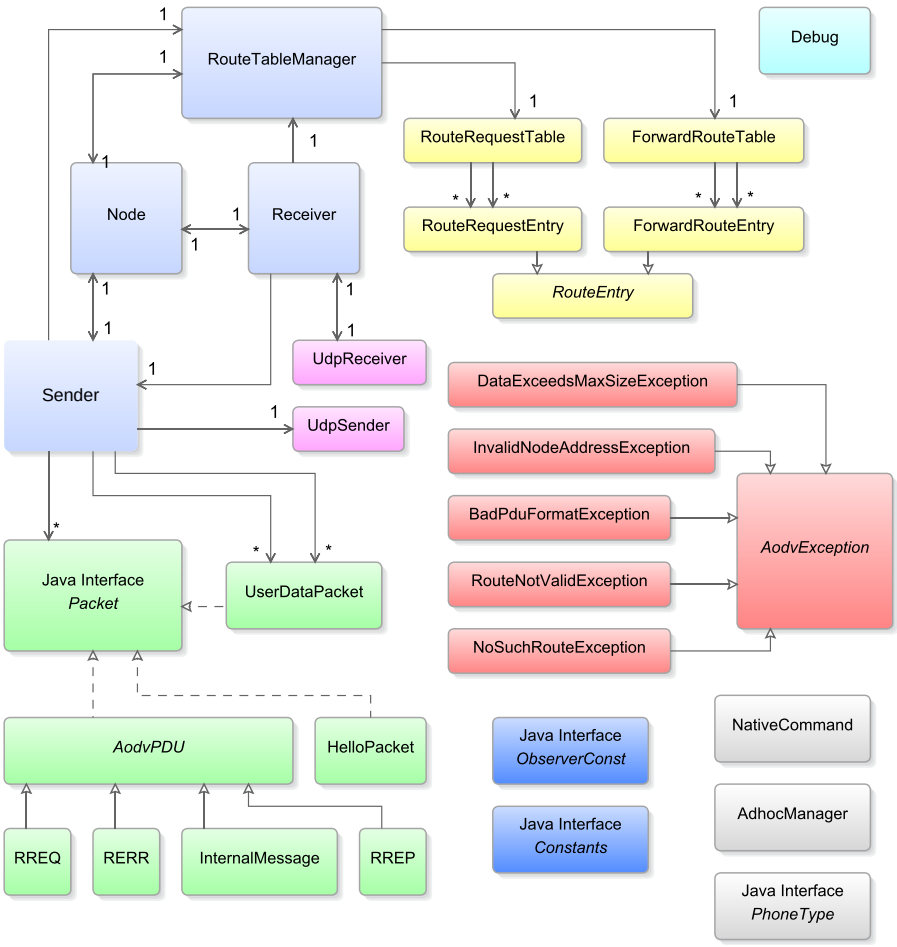


Figure 4.2: Compact UML class diagram of the library

constructor take an integer as a parameter. The integer is the nodes network address, which therefore *must* be unique in the ad-hoc network. When a `Node` object is created, it will instantiate the other classes of this package. These are `RouteTableManager`, `Sender` and `Receiver`. All of them take a `Node` object The routing protocol will first commence if `start()` is invoked.

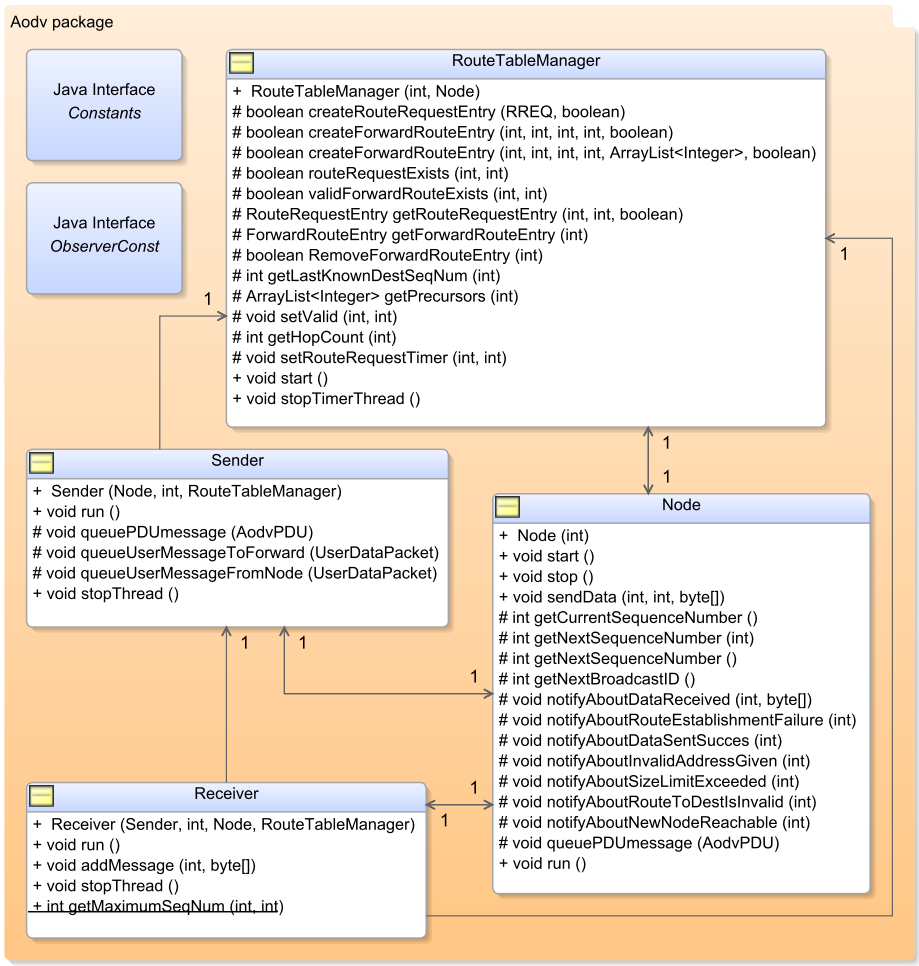


Figure 4.3: Expanded UML class diagram of the aodv package

4.2.1.2 RouteTableManager

`RouteTableManager` Manages the access to the two tables holding routing information. The tables (from the `routes` package) are only accessed through this class.

`RouteTableManager` also deals with stale route entries, whenever they get too old. Dealing with the entries means, that they are either removed or marked as invalid. Thus the class defines both a method for removing and marking

a `ForwardRouteEntry`. These are called `RemoveForwardRouteEntry()` and `setValid()` respectively.

The route table manager also enables the creation of forward route entries and route request entries. Forward entries are created by a `createForwardRouteEntry()` method, which takes the parameters as specified by the routing protocol and a boolean value called `notifyObserver`. This is set to true if the creation of a route should be known to the application layer.

Route request entries are created through a corresponding `createRouteRequestEntry()` method. The parameters taken is an RREQ object (described in the `pdu` package), and a boolean value `setTimer`. The method is used to buffer received (or sent) route requests, which explains the RREQ object. The boolean is set to true if the entry should be removed after a defined `PATH_DESCOVERY_TIME`.

In order to know *when* an entry is old, the class defines an internal private class called `TimeoutNotifier`. This class runs a thread to manage the time to live (TTL) of each entry, and to cleanse the table of stale ones. This thread is referred to, as the timer-thread. `RouteTableManager` handles all requests from other classes of the same package, when they need to know the current routing information, or when new route entries should be created. The `RouteTableManager` is created by the node class, when that class itself is instantiated. Route table manager contains only protected methods, except two methods to start and stop the timer thread, since this class is not known outside the `aodv` package.

4.2.1.3 Sender

`Sender` is the class used whenever PDUs should be sent. This includes messages that contain the data the application layer want to send. The main responsibility of this class is to convert the PDUs into raw data, passing them to the UDP layer for sending. The `Sender` class thus know to the UDP layer by creating and holding an instance of `UdpSender`.

The `Sender` is also responsible for initiating route discoveries, should the destination is not known. The class implements `Runnable`, such that sending is run on its own thread.

The sender offer other classes from the same package to send library packets (including application packets), by defining three protected methods. The methods signature can be seen in Figure 4.3.

Which method to use dependent of which library packets that should be sent. The sender distinguishes between application packets, that simply should be forwarded further, and packets that is request sent from the application of its own node. The third method is for sending AODV PDUs. Section 4.3 explains how sending PDUs is designed.

4.2.1.4 Receiver

`Receiver` is responsible for parsing messages from the UDP layer and afterwards, reacting on the PDUs according to the behaviour defined by the AODV protocol. Actions performed by the `Receiver` is e.g. notifying the application layer, about some user data or letting the `RouteTableManager` create a route to a new destination.

The `Receiver` class defines an internal private class, called `Message`, that holds the received raw data from the UDP layer. This class also have a field for the node address of the neighbour, which the data were received from. The class defines a single method called `getType()`. It returns an integer that indicates what PDU it is and is used, such that the `Receiver` may parse it to an PDU. `Receiver` implements the `Runnable` for handling `Message` objects. When the `Receiver` is created by the `Node` class, it will instantiate an `UdpReceiver` object. The receiver will not invoke the `start()` method of the `UdpReceiver`, before its own `start()` method is invoked (by the `Node`).

4.2.2 Routes

`routes` enables the `aodv` package to retrieve known route information, through two tables. Figure 4.4 shows the classes of this package and the functionality offered by each of the classes.

4.2.2.1 ForwardRouteTable

`ForwardRouteTable` is a class used to contain and manage access to the route entries for destination nodes, which are represented by `ForwardRouteEntry` objects. By Figure 4.4 it is seen that the class, therefore offer methods for adding, removing and fetching such objects. It is responsible to hold *one* route entry per destination at most. How this is achieved, is described in Chapter 5 section 5.4.1.

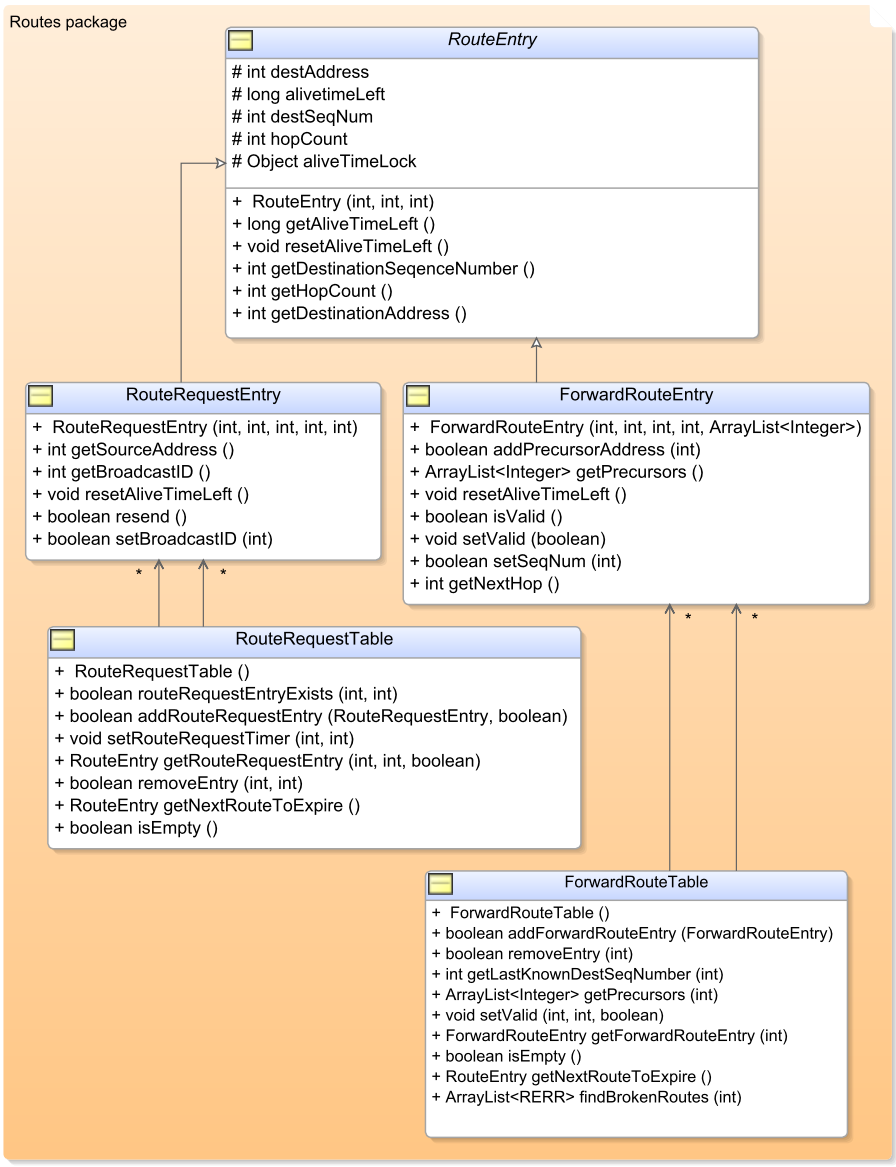


Figure 4.4: Expanded UML class diagram of the routes package

In addition, the table offer a way for searching it entries that have a next-hop that matches the node address given. This method is called `findBrokenRoutes()` and returns a list of RERR PDUs, one for each match. The method is used

whenever a broken link to a neighbour is discovered.

4.2.2.2 RouteRequestTable

`RouteRequestTable` is similar to the `ForwardRouteTable`, but is used to contain and manage access to `RouteRequestEntry` objects. The table also ensures that only *one* `RouteRequestEntry` are held per destination per request. This table is used whenever the `Receiver` receives a route request from a flooding, or when the node itself initiates such a request. It enables the `Receiver` to check if the request has been reacted upon before, as defined by the behaviour of the AODV protocol. The methods of the `RouteRequestTable`, is different in the parameters required, when looking on common methods that exist between the `ForwardRouteTable`. This is visible from the class diagram of Figure 4.4. Therefore, the two tables do not inherit from some common abstract table class or interface.

4.2.2.3 RouteEntry

`RouteEntry` is an abstract class that defines common attributes and operations for route entries. `RouteEntry` is used to store all necessary information for a single AODV route entry.

Following attributes are defined by this class: destination address, destination sequence number, hop-count and TTL. All these except the TTL are defines as integers. The TTL value is a long, since the system time is used to define when a route stale.

4.2.2.4 ForwardRouteEntry

`ForwardRouteEntry` inherit from `RouteEntry`, so it represents an AODV route. A `ForwardRouteEntry` object is created for each known destination that is used by the AODV protocol to forward messages. Each route are able to buffer the neighbouring nodes that uses the route in a *precursors list* [15]. Neighbouring nodes that forward through this route can be added to the list by a public method called `addPrecursorAddress()`. The class also `getPrecursors()` exist, in order to retrieve the precursors. The `ForwardRouteEntry` can be marked as invalid through `setValid()`, and checked upon through `isValid()`.

4.2.2.5 RouteRequestEntry

`RouteRequestEntry` also inherit from `RouteEntry`, but do not represent an AODV route directly. An object of this class is created whenever a route request PDU is received. These object are then buffered such that the node is able to determine if it should react if the same request request should be received twice. Whenever a route request PDU is received, a route to the originator of the request is created by adding an `ForwardRouteEnty` instance to the `ForwardRouteTable`. When a `RouteRequestEntry` object exceeds its TTL value, the `resend()` method is used to determine if a new route discovery may be initiated.

4.2.3 Udp

The `udp` package is responsible for single-hop communication, and function as the data link layer for the library. The `udp` package consist of two classes, functioning as the server and client side respectively.

4.2.3.1 UdpSender

`UdpSender` is equivalent to a server socket. This class only has two methods which is used for sending UDP packets (including broadcasts) and closing the UDP socket. These are called `sendPacket()` and `closeSoket()`.

This class is created by `Sender` in order to actually sent data. Any other classes do not know to the `UdpSender`, such that the coherence between the UDP layer and the protocol is minimum.

4.2.3.2 UdpReceiver

`UdpReceiver` is again similar to the client side of the server/client model. It listens for UDP packets that should be received. `UdpReceiver` differentiate between receiving datagrams that are sent by broadcast and datagrams with the node itself as destination. `UdpReceiver` therefore have an internal private class that only listens for broadcast packages, on a port that is only used for such messages. The `UdpReceiver` is only known by the `Receiver` in the AODV package, by instantiating it. The `Receiver` object is given as a parameter to the `UdpReceiver`, such that the UDP layer is able to pass on any received data.

A better design to reduce coherence between the layers could have been achieved by using an observer-pattern. The observer-pattern would then be used to notify the `Receiver` whenever new UDP data is received.

4.2.4 Pdu

`pdu` defines the legal messages that the protocol use to communicate, in order to function.

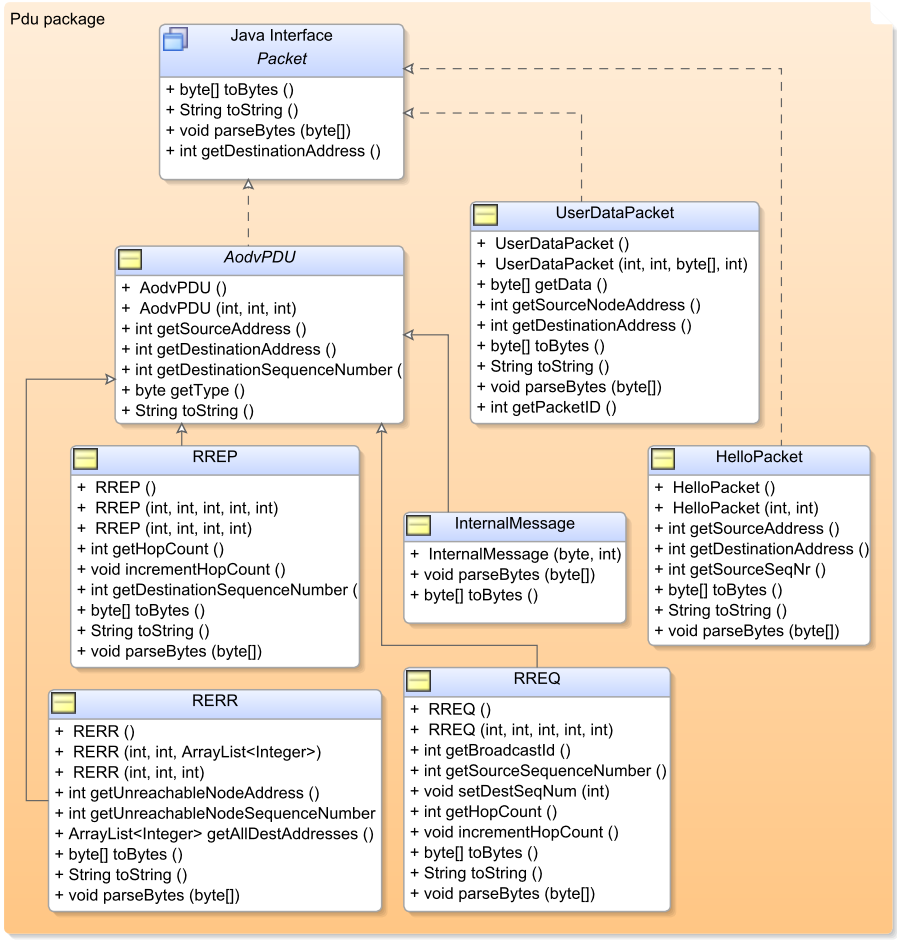


Figure 4.5: Expanded UML class diagram of the pdu package

4.2.4.1 Packet

`Packet` is an interface that all classes of this package inherit from. The interface defines common methods that are required in order to be a legal library message. Any class that implements the interface, will have to implement methods so the class can be:

- Converted into an array of bytes by `toBytes()`
- Converted into a string by `toString()`
- And a method to parse an array of bytes a PDU again by `parseBytes()`

The first two methods are used by the `Receiver` and `Sender`, in order to let the `Udp` package handle them. The `Receiver` uses `parseBytes()`, when packets are received. Finally it is required by the interface that classes implement `getDestinationAddress()`, so the destination for which this message is intended, can be known.

4.2.4.2 HelloPacket

`HelloPacket` is the message that is broadcasted periodically, to any neighbouring nodes. It contains the node address from which this message originated, and the sequence number of that node. These two parameters are needed, for the neighbouring nodes to establish a route.

4.2.4.3 UserDataPacket

`UserDataPacket` represents the type of message, that the application layer communicate. When the application in a node want to send data, it will have to attach a packet identifier along with the data, to enable the protocol are able to reply if the message was sent successfully. The packet identifier is a value that only is unique in the node locally. It is therefore not sent to other nodes.

4.2.4.4 AodvPDU

`AodvPDU` is an abstract class. It implements common properties and methods for all PDUs that belong to the AODV protocol. It implements the method

for retrieving the destination address and defines that all AODV PDUs shall also store the source that originated the PDU. In addition, each PDU must have a way for the protocol to know which *type* of PDU this message is, and a destination sequence number (also with a way of retrieving it). The destination sequence number indicates when this node received any information (such as a AODV PDU or packet) about the destination.
Classes that inherit from the `AodvPDU` are:

- `RREQ`
- `RREP`
- `RERR`
- `InternalMessage`

Each of them implements the conversion to a string, an array of bytes and parsing an array of bytes into the message. The `RREQ` class is used to represent an request to find a route to a destination node. The `RREP` is a route reply message, that is sent from nodes that can accommodate a request for a route. The `RERR` message is used to notify nodes about a broken route. Since several nodes may be using a route through a node, a `RERR` can contain a list of destinations. The `InternalMessage` is a class used for notifying about route changes, that the `Sender` class needs to know. These changes are when a new route is discovered or when a request to find a node in the network fails. These messages are never exchanged with other nodes.

4.2.5 Setup

The `setup` is the only Android-depended package and deals with configuring the wireless adaptor, so that it runs in ad-hoc mode and has a static IP-address. The ad-hoc library thus consist of two separate parts:

- A single package that is Android-specific (`setup`)
- A second part that implements the AODV protocol. This protocol does not care what type of OS the ad-hoc network runs on.

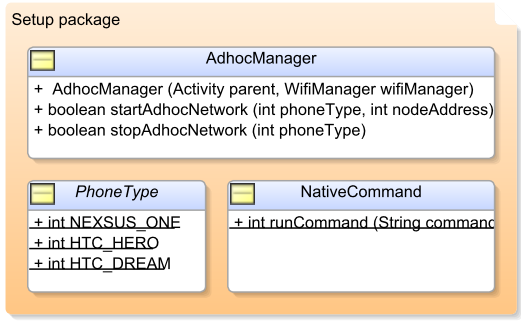


Figure 4.6: Expanded UML class diagram of the setup package

4.2.5.1 AdhocManager

`AdhocManager` is the main class of this package. It is the Android-dependent interface to the application layer, while the `Node` class is the protocol interface. Together they act as the *library interface* for the application layer. `AdhocManager` offers the functionality of starting and stopping an ad-hoc network on the Android device. The methods of this classes can be seen in Figure 4.6. The node address parameter must be the same as the parameter given to the `Node` class. Section 5.5 in Chapter 5 explains in greater details how an ad-hoc network is created.

4.2.5.2 adhocsetup.so

`adhocsetup.so` is a shared library that contains a C-file called `native_task`. `native_task` makes JNI calls possible. It enables the packet to execute shell commands by its methods.

4.2.5.3 startstopadhoc

`startstopadhoc` is a binary written in C and lets the packet create and stop an ad-hoc network on Android, by configuring the wireless adapter of the device.

4.2.5.4 NativeCommand

`NativeCommand` is a simple class, that enables the `AdhocManager` to run shell commands in Java, by loading the shared library `adhocsetup.so`.

4.2.5.5 PhoneType

`PhoneType` is also a simple class that is used to define static constants. Such a constant is given as a parameter when the start or stop methods of the `AdhocManager` class are used. The constants defined in `PhoneType` tell the application layer, which phone types the ad-hoc library currently support.

4.2.6 Exception

`exception` defines *protocol* error/notification messages that may occur in different situations. These error messages are not exchanged with other nodes, but are only for internal use. It should also be noted that these message is not exchanged with the application layer, though an application may be notified because of these exceptions.

All classes of this package inherit from the Java `Exception` class. They are offer no additional functionality and are only used for better code style and allow easier modifications in the protocol behaviour for future development. The abstract class named `AodvException`, enables the library to distinguish other Java exceptions, from ones that are related to the protocol. All other classes in this package, inherit from the abstract class. These are:

- `BadPduFormatException`
- `DataExceedsMaxSizeException`
- `InvalidNodeAddressException`
- `NoSuchRouteException`
- `RouteNotValidException`

All these exceptions are cast in different situations. `BadPduFormatException` is e.g. thrown whenever a PDU failed to parse an array of bytes into that message. `DataExceedsMaxSizeException` is thrown by the `udp` package if a PDU, exceeds

the size of a single UDP packet. An `InvalidNodeAddressException` is thrown if the destination address of an application packet is not in the valid address space (as defined in the `Constants` class). `NoSuchRouteException` is cast when route information about an unknown destination is attempted fetched from the tables. `RouteNotValidException` may be thrown if the routing information that is attempt fetched exists but is not suitable for forwarding.

For a detailed class diagram it is referred to Appendix B Section B.1, if Figure 4.2 do not suffice.

4.2.7 Etc

This package covers the classes that is not part of the library, but offer some functionality that is useful anyway. From Figure 4.1 it is seen that this package is located in a layer that may be used by all other layers, including the application layer. The package contains only a single class.

`Debug` is a simple class that is used for debugging while developing the ad-hoc library. It contains two static methods. One that enables the developer to set where the output should be printed and one to actually print a message, and another to print debug message if the print stream is set. The methods are named `setDebugStream(PrintStream ps)` and `print(String s)` respectively. The usage of this class also covers the need for any developer of an application, for the ad-hoc library, to gain insight in what is occurring in the library. Therefore the visibility of these methods are public. For a detailed class diagram of this package it is referred to Appendix B Section B.2

4.3 Concurrency design

In section 4.1.3 it was suggested that there is a need of at least two threads to manage sending and receiving data over UDP. When a message is received, it has to be parsed to know what type of PDU the data contain. Other sent messages may get lost if the receiver is in the process of parsing a PDU. To ensure that all messages that *must* be received *are* received, two receiver-threads are needed for ensuring correct reception:

- A thread to process the received UDP packets into PDUs
- A thread for receiving UDP packets

In addition, AODV needs a separate timer-thread to clean any stale route entries, and a thread to broadcast periodic hello messages. The mentioned threads deal with running AODV and UDP. In other words, these threads are all created by the ad-hoc library. The application layer that uses this ad-hoc library, runs in its own application-thread.

The threads need to have a simple way of interacting. It is achieved, by letting threads interact through shared First-In-First-Out (FIFO) queues. Figure 4.7 is an illustration of how all the library threads interacts through the queues.

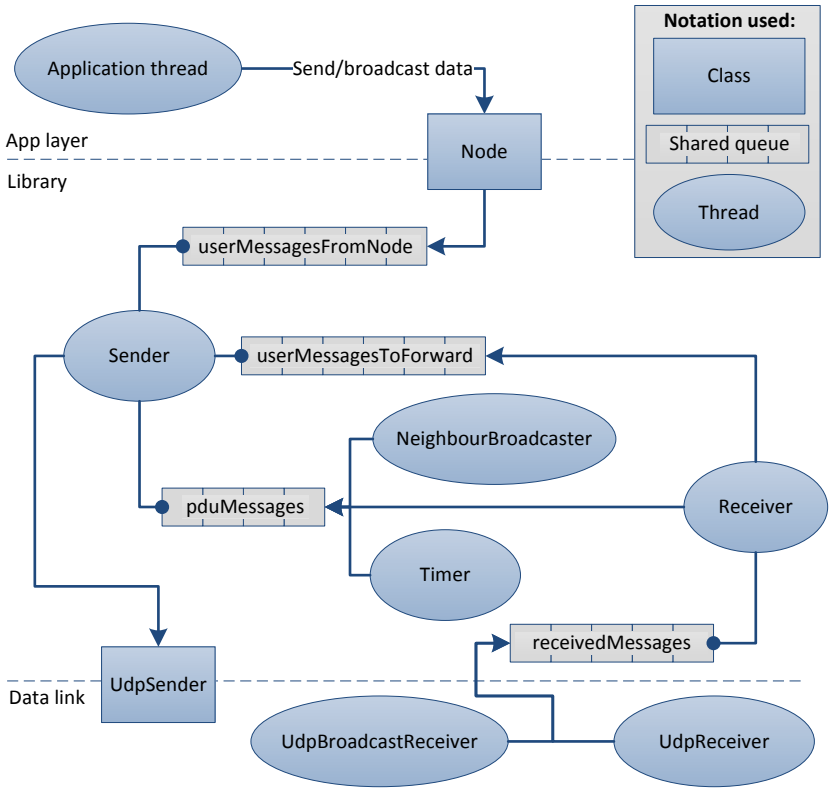


Figure 4.7: Thread interaction through shared FIFO queues

The sender thread share access to three queues called:

- `userMessagesFromNode`
- `userMessagesToForward`
- `pduMessages`

`userMessagesFromNode` contains only `UserDataPacket` elements. These elements are packets from the application layer of the node itself.
`userMessagesToForward` is similar, but contains only application messages that are received by other nodes and need further routing.
`pduMessages` is used for queueing messages of the type `Packet`. The queue contains both `AodvPDU` messages and `HelloPacket` messages, that the neighbour-broadcaster enqueues periodically.
The reason why there are tree separate queues, instead of a simpler implementation with a single queue, is that it enables the sender to prioritize which type of messages to process first. The order in which they are mentioned, is also the order in which they are processed by the sender-thread.

Note that the UDP layer actually comprises of two threads. One for receiving data, sent specifically to a device and one for receiving broadcast data. `receivedMessages` is the queue that the UDP layer uses to queue messages for processing by the receiver-thread.

Shared queues are used when one or more threads want to use the same functionality, so that synchronization for accessing the (shared) resource is needed. Such design simplifies the synchronization issues, by uses a message passing paradigm. Using FIFO queues, it is ensured, that any queued message *eventually* will be polled and processed. This property would not necessarily be true, if e.g. the neighbour-broadcaster always queued its messages at the head of the queue. It could happen, that an internal request for a hello-broadcast always managed to be queued in the head, right before some other internal request where polled. This would result in starvation of the other elements.
The trade-off with FIFO is, that there is no way of knowing *when* a message will be processed. This is not optimal for internal messages, that require to be processed within some constant interval such as periodic hello broadcasts, but that is a consequence of this design choice.

4.3.1 Notification messages

Whenever the routing protocol needs to notify the application layer about some event, it queues an internal message on a shared queue that is accessed through the `Node` class. A separate thread created by this class, then deals with notifying the application layer about the events in the queue.
Several types of notifications can be made, as specified by the requirements. These include notifications of received application messages or notification about some node which could not be discovered in the network. The queue of notification messages can thus be accessed by several threads. This is illustrated by Figure 4.8.

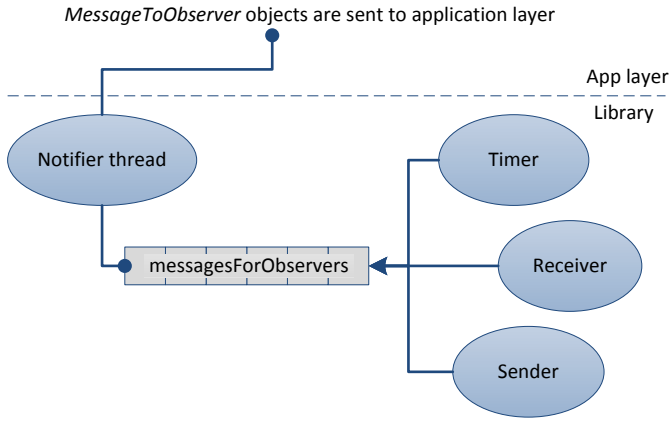


Figure 4.8: Additional thread interaction when application is notified

An observer-pattern is used to notify the application layer. The purpose of the queue of notification messages is only for performance reasons. If a thread, e.g. the sender thread wants to notify about an event, this thread will not be able to process its main task, if it where to use the observer-pattern directly. So by having a separate thread, only to use the observer-pattern, all other library threads will be able to continue their main tasks. They only have to queue a `MessageToObserver` element, if they want to notify the application layer. The separate thread, then deals with actually notifying any observers.

Chapter 5

Library Implementation

This chapter will describe in detail, how the library is implemented after overall design-issues have been addressed in the previous chapter. Focus in this chapter includes, how the threads are implemented and how critical sections are secured. Codesnippets will be used along explaining important parts.

5.1 Observer-pattern

The `Node` class is the subject of the observer pattern by extending the `Observable` class. As described in section 4.3.1, a separate thread is used and a method is available for every type of event. Whenever such a method is used, the condition-queue of the notification queue is notified. The only thread that can be waiting in the condition-queue is the notification thread. The thread will only be waiting in the condition-queue of the monitor, if it does not contain any events that has not been processed.

Codebox 5.1: Preventing the notifier-thread from busywaiting

```
1 synchronized (messagesForObservers) {
2    while(messagesForObservers.isEmpty()){
3       messagesForObservers.wait();
4    }
```

```
5 }
6 setChanged();
7 notifyObservers(messagesForObservers.poll());
```

Codebox 5.1 shows the condition of the monitor. `messagesForObservers` is the shared notification queue. The while-loop provides against spurious wakeup, that can occur with Java. The `synchronized` keyword ensures that the body externally appears as an atomic action externally. This is necessary or a message can be put in the queue, just after the *isEmpty()* check and just before the thread waits in the condition-queue of the monitor of `messagesForObservers`. As a consequence, the thread would not register the change in the condition and therefore will not notify the application layer, unless some other event is queued later so the thread is woken.

When a library thread queues an event, it first puts the message in the end of the queue, and then wakes the notifier-thread if it is sleeping. Since several threads may try to put messages in the queue concurrently, there is a need for synchronization so inconsistent states are avoided. Also the notifier-thread may concurrently try to pull messages. Access to the queue is thus a critical section. Correct synchronization is simply achieved by instantiating the queue as a `ConcurrentLinkedQueue` offered by Java. It ensures that the actions of *putting* and *pulling* from the queue externally appear as atomic actions.

5.2 Sending Messages

As mentioned earlier, sending packets of various types are achieved using one of the three shared queues in the `Sender` class. All three queues are instantiated as `ConcurrentLinkedQueue`. Since *putting* and *pulling* elements in the queues are the only way the queues can be modified by threads, no race conditions can occur. Places where the sender-thread uses multiple methods, that the some queue offers, are surrounded by the `synchronized` keyword, so the statements seems like a single atomic statement to other threads.

The sender thread is the only thread that consumes messages from the queues. The thread is synchronized on an object called `queuelock`. When producers put messages in the queues, they notify the sender-thread, that may be waiting in the condition-queue of the `queuelock` monitor. This is done by notifying the condition-queue. The condition for which the thread lies to wait, can be seen in codebox 5.2.

Codebox 5.2: Preventing the sender-thread from busywaiting

```
1 synchronized(queueLock){
2     while(pduMessages.isEmpty() && userMessagesToForward.isEmpty()
          && (isRREQsent || userMessagesFromNode.isEmpty())){
3         queueLock.wait();
4     }
5 }
```

When the sender-thread passes the condition, it will begin process any messages that may exist in the queues. The sender-thread will always first process the head element of any queue. `userMessagesFromNode` is prioritised first, since route discoveries is likely to be requested by the application layer. It will first check if the destination is known. If that is is the case, it will simply let the UDP module send the message. Otherwise it will have to queue a `RREQ` object in the `pduMessages` queue. If a `RREQ` is queued, a corresponding `RouteRequestEntry()` is created and buffered in the `RouteRequestTable` by using the `createRouteRequestEntry()` method in the `RouteTableManager`. This method takes a boolean parameter called `setTimer`. This parameter is set to false such that the `RouteTableManager` do not count down the time it is buffered. The timer is not started until the `RREQ` is actually sent.

The thread will then skip any remaining messages in the `userMessagesFromNode` until all other queues have been processed and an `InternalMessage` object is received. The internal message will tell if a route to the request destination where successfully created or if the destination was not found at all.

In cases where the request was not satisfied, the sender thread queues a notification to the application layer about the failure, and cleanses the `userMessagesFromNode` queue from any packets that have the same destination. A boolean called `isRREQsent` is used by the sender-thread in order to check if the head of the `userMessagesFromNode` is ready to be sent. This boolean is set to false when an internal message is detected, so the queue is not skipped.

In order to avoid busywaiting, it has to be ensured that the condition of codebox 5.2 considers all the states that would result in unnecessary looping. These states are:

- When the three queues do not contain any messages to consume
- When both `pduMessages` and `userMessagesToForward` are empty and a route request is sent because no route is known for the head messages in `userMessagesFromNode`.

These situations are exactly what the condition considers, by using the `isEmpty()` method and the `isRREQsent` boolean. The only issue to consider is to ensure that the boolean is set correctly in the different situations.

The only cases where boolean is set, is when an internal message is received for a created route to the requested destination or when a route discovery fails. The internal message contains the destination address of the created or failed route. The boolean is only set if the destination address in the internal message corresponds to the destination that the route discovery were initiated for. Thus the boolean can not accidentally be set because of a created route for different destination.

The second queue that is processed, is the queue of user messages for forwarding. It is attempted to send each message through a known valid route. Should a route not exist or be invalid, the message will be removed and an `RRER` message will be sent back to each neighbouring node that is known to have used that route. These nodes are also known as the precursors.

The third queue containing AODV PDUs is the final queue to be processed. If any elements exist to be processed in `pduMessages`, the sender-thread will pull the head object from the queue. It then make an `instanceof` check, in order to know if the object is an `AodvPDU` or `HelloPacket`. A hello packet will be simply be sent over UDP as a broadcast.

In case of an `AodvPDU` a private method `handleAodvPDU()` is called, with the object as a parameter. The method then switches over all AODV PDU types (from the `Constants` class) until the object `getType()` method matches the constant. If the PDU is an instance of:

- a `RREQ`, the PDU is sent over UDP as a broadcast. Since a corresponding `RouteRequestEntry` is buffered in the `RouteRequestTable` when the request was created, the sender-thread only have to let the `RouteTableManager` count down the time in which the entry shall be buffered. This is done by calling the `setRouteRequestTimer()` method of the `RouteTableManager`.
- a `RREP`, the PDU is sent to the originating device that initiated a route request. The address of that node is kept in the `RREP` message.
- a `RERR`, the PDU is sent to each of the nodes that at any time have used the now broken route. These addresses are the precursors from a `ForwardRouteEntry` which are copied to the PDU.
- an `InternalMessage` containing a message about a route request failure. All elements in `userMessagesFromNode` that is to be sent to the unknown

destination, are deleted. The `isRREQsent` boolean is set to `false`, so the queue will be considered. As the last step, the condition-queue of the `queuelock` monitor is notified.

- an `InternalMessage` containing a message about a route creation. The head of the `userMessagesFromNode` can be consumed. Therefore the `isRREQsent` boolean is set to `false` and the sender-thread is woken.

5.3 Receiver

The receiver-thread consumes messages from a single queue called `receivedMessages`, that is shared by the `Receiver` class. Two producers exist for that queue: `UdpReceiver` and `UdpBroadcastReceiver`. Again the synchronization issues that exist here, are dealt with in the same manner as previously, by using the `ConcurrentLinkedQueue` class. The receiver-thread, will consume messages as long as it is not empty. Otherwise the thread will wait in the condition-queue of the `receivedMessages` monitor. Codebox 5.3 show the condition for which the receiver-thread stops processing any messages.

Codebox 5.3: Preventing the receiver-thread from busywaiting

```
1 synchronized (receivedMessages) {
2    while (receivedMessages.isEmpty()) {
3       receivedMessages.wait();
4    }
5 }
```

When the receiver-thread passes the condition, it will begin processing the elements of `receivedMessages`. The thread will pull the head `Message` object, and switch over the PDU types that exist. It is done by matching `getType()` method of the object, with the PDU constants. The method tries to parse the first byte from the array of bytes that is held in `Message`. In the attempt to parse the first byte, a `NumberFormatException` can be thrown. The message will be discarded, as it will be regarded as corrupted data. In case of a correct parsing, but no matching PDU type, the data will also be discarded since the data do not belong to the domain of AODV PDUs. In the final case where a match is found, the array of bytes that are held in `Message` will be parsed into a corresponding PDU object, and a private method to handle the object is called.

The following sections will describe how, the private methods that exists for each case, will handle the object.

5.3.1 Handling a Hello PDU

When a `HelloPacket` is received, the forward route to the originator of the PDU is consulted in the `RouteTableManager`. The route is attempted to be marked as valid by using the `setValid()` method in the table manager. If no forward entry is known with the originator as destination, the method will throw a `NoSuchRouteException`. In this situation the exception is caught and a route is created by the `createForwardRouteEntry()` method.

5.3.2 Handling a Route Request PDU

`routeRequestReceived()` is the method in the receiver that handles route request PDUs. The method takes a `RREQ` object, and an integer `senderNodeAddress` as parameters. The integer is the address of the neighbouring device, where the message was received from.

The first check made when a `RREQ` PDU is received through a flooding, is to check if the same PDU has been handled before. If the `routeRequestExists()` method of `RouteTableManager` returns true, meaning that is has been handled before, the PDU is discarded by simply returning from the receiver method.

The rest of this section will describe the case where the request is received for the first time. Figure 5.1 gives an overview of how the programflow for handling an received route request PDU is.

A PDU is received from a maybe undiscovered neighbour, therefore creation of a forward route to the `senderNodeAddress` is attempted.
The `createForwardRouteEntry()` method of the table manager is used, with an unknown sequence number and a hop-count of one as parameters. The hop-count of the route request PDU is then incremented with one, as a new node (this node) is handling the PDU. A corresponding route request entry is then created by the `createRouteRequestEntry()` method.

After incrementing the PDU hop-count, a forward route to the originator of the request is created. Because the destination of the route is the originator, this route is also referred to as the reverse route. Since a route to the source may already be known, a local reference to the possible existing `ForwardRouteEntry` object is attempted acquired. The `getForwardRouteEntry()` method is used and returns a reference object of exactly that type if a valid route exists.

The method call is surrounded by a try-catch clause in the event of a thrown

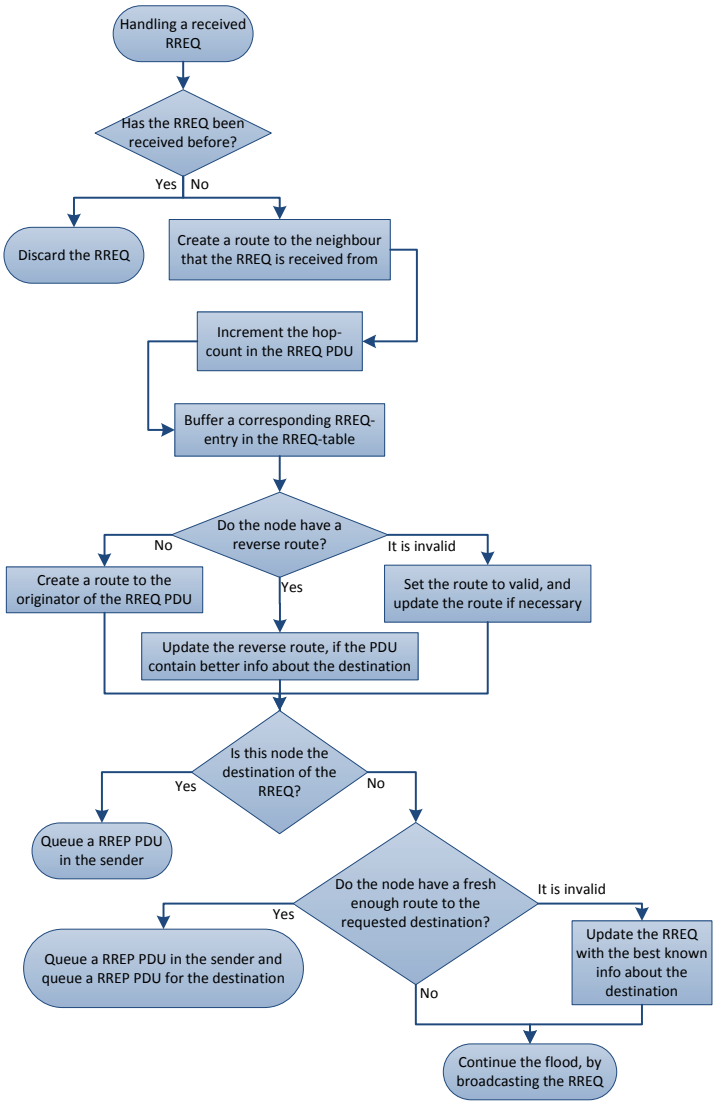


Figure 5.1: Flowchart showing how a received route request is handled

`NoSuchRouteException` or `RouteNotValidException`. The case where no route exists, the reverse route is simply created with the sequence number and hop-count known from the PDU. If an invalid route exists, it is marked as valid and updated with the best known sequence number through the `setValid()` method, offered by the table manager. Since the timer-thread may have deleted

the route between the `getForwardRouteEntry()` and `setValid()` calls, the `setValid()` may cast a `NoSuchRouteException`. If this is caught a route is created as before with the same sequence number and hop-count from the PDU.

Finally, if the route exists and is valid, the freshness of the entry is compared to the information contained in the PDU. If the contained PDU has the best route information, the old route is removed (using `RemoveForwardRouteEntry()`) and a route is created to the same destination, with the updated destination sequence number and hop-count. The best route information is compared through the static `isIncomingRouteInfoBetter()` method of the `Receiver` object. At this point a valid reverse route certainty exist.

The next thing to do is to check if this node matches the final destination of the PDU message. If this is the case the sequence number of this node is incremented with one, if it is equal to the PDUs destination sequence number. A `RREP` object is then created, with the originator of the request as the destination, the address of this node as the source, and the nodes sequence number (that may just have been incremented). The created `RREP` object is held in a local variable called rrep.

If the route request has not reached its destination, meaning that it is an intermediate device, a table lookup is made through the table manager, in order to check if this node has a route to the desired destination. This table lookup can cause either a `NoSuchRouteException` or a `RouteNotValidException` if it fails, thereby 3 cases can occur:

1. The simple case where `NoSuchRouteException` is caught. The route request is simply broadcast, so the flood can continue.

2. If a `RouteNotValidException` is caught. A `RREP` object is created with the same parameters except for one. The destination sequence number that is put in the `RREP` is the best of the sequence numbers from the route entry and the sequence number from the route request PDU. Fetching the sequence number of a route is done by the `getLastKnownDestSeqNum()` method of the route table manager. This method can cast a `NoSuchRouteException` as the timer-thread in the meantime, may have removed the route entry. If this exception is thrown, nothing can be done, the route request PDU is simply broadcast.

3. The last case where no exceptions are thrown means a valid route was fetched. A `RREP` object is only created if the destination sequence number of the route entry is greater or equal to the sequence number of the PDU. This ensures that the originator of the request do not receive replies containing routes that are older than the requested ones.

As the last thing, a `RREP` object is created, and sent to the destination, that the request wanted to find. This is done, in order to ensure that a symmetric route is created when a request is accommodated, as communication between two nodes often needs a two-way message exchange.

5.3.3 Handling a Route Reply PDU

The method that handles received route replies is called `routeReplyReceived()` and take two parameters. A `RREP` object and a `senderNodeAddress` similar to the `routeRequestReceived()`. As in the previous handler creation of a route to a possible new neighbour (with `senderNodeAddress` as address) is attempted. The parameters to the `createForwardRouteEntry()` method are the same as in the route request handler. The PDU's hop-count is also incremented with one.

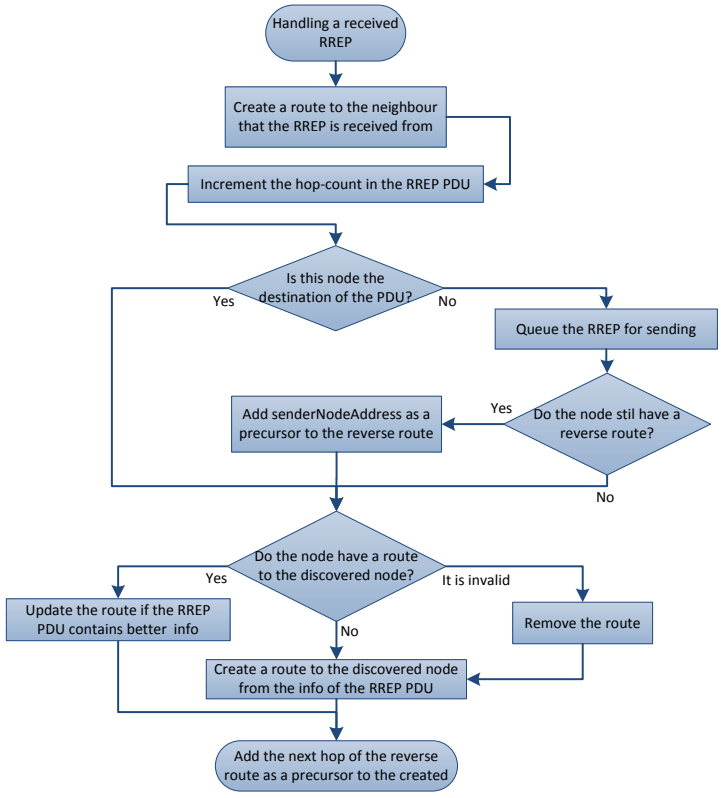


Figure 5.2: Flowchart showing how a received route reply is handled

Figure 5.2 show the steps taken when a route reply is received. The next step is an if-clause checking if the `RREP` object has reached the originator of the initial RREQ by matching its own `nodeAddress` with the `getSourceAddress()` method of the PDU.

If the node is not the originator of the initial `RREQ` object then the PDU is queued for sending by calling `queuePDUmessage` method of the `Sender`.

A try-catch clause surround the attempt of fetch the reverse route created previously when the RREQ was initially received, so the precursors list can be accessed. The `senderNodeAddress` is added to the precursors list of the route, since it is the next-hop of the reverse route.

If the attempt to fetch the reverse route results in a caught `AodvException`, nothing can be done. The RREP may never reach the source of the route request PDU, as a route back is not currently known.

The method continues though the steps of updating the second part of the forward route. From this node to the destination node that replied with the RREP. It is thus attempted to fetch the forward route in a similar manner, by surrounding the `getForwardRouteEntry()` method with a try-catch-clause. Three cases then exist:

1. If the attempt succeeds, the route is updated with the best known sequence number and hop-count information about the destination, by calling `isIncomingRouteInfoBetter()`. The parameters passed to this method is two sets of hop-count and destination sequence numbers, from the `RREP` PDU and the forward route respectively. If this method returns true, it means that the PDU contains better route information for that destination.

   The forward route entry is thus removed from the table by calling `RemoveForwardRouteEntry()` with the destination as parameter. A new route with the better information is created and added to the table by calling `createForwardRouteEntry()`. The parameters passed to this method apart from the destination address and next-hop, is the better sequence number, hop-count and also the precursors list from the removed forward route.

2. If the attempt to fetch the forward route to the destination node results in a caught `NoSuchRouteException`, the route is simply created from the route information for that destination from the `RREP`.

3. If instead a `RouteNotValidException` is caught, the route to the destination node is removed and a new valid route is created with route

information contained in the PDU.

As the last step in the three cases seen in Figure 5.2, the precursor for that route is set to the next-hop address of the reverse route if such a route exist.

This is implemented by using an integer which is initialised to -1. If the fetching of the reverse route succeeds, the local value is set to the next-hop address. When reaching the final step, a simple check is made to ensure the precursor is only added if it is not equal to -1.

5.3.4 Handling a Route Error PDU

When receiving an error PDU, the claimed faulty forward route is retrieved by the `getForwardRouteEntry()` method of the table manager. This method casts an `AodvException`, if no route with that destination exists. In that case the receiver discards the PDU, since no actions can be taken. The rest following will then describe what happens if a forward route exists.

If the destination sequence number contained in the PDU is greater or equal to the destination sequence number of the forward route, the error message is reacted upon. A new `RERR` message is created with the destination sequence number and the address of the destination node from the received error PDU. The new error message must be sent to the nodes that have used the forward entry from this node at any time, so the precursors of the forward entry is used as the destination(s) of the new PDU. The PDU is then queued in `pduMessages`, thereby these nodes will know about the topology change. As the final action, the forward entry is marked as invalid and the entry therefore can not be used.

An if-clause, with the static method `isIncomingSeqNrBetter` as the condition is used to compare sequence numbers. Section 5.3.6 will describe how comparison of sequence numbers is implemented.

5.3.5 Handling a User Data PDU

A received user packet can either be a message that has reached its destination (this node) or a message that should be forwarded, therefore the destination address is checked. If it matches the node address or the broadcast address, the messages is queued in `messagesForObservers`, thereby the application layer

receives the data. If the message has to be forwarded, the message is queued in `userMessagesToForward` so further routing can be made.

5.3.6 Comparing Route Information

Comparing route information consist of comparing two parameters. These parameters are the destination sequence number and the hop-count. The `Receiver` class defines static methods, so correct comparison can be achieved:

- `getMaximumSeqNum()`
- `isIncomingSeqNrBetter()`
- `isIncomingRouteInfoBetter()`

As described in section 4.2.1.1, the `Node` class manages the incrementation of its own sequence number. The sequence number is simply an integer that is incremented with one each time, and is reset to `FIRST_SEQUENCE_NUMBER` value (from the `Constants` class) should the maximum value of an integer be reached. Therefore the implementation of comparing two sequence numbers from different nodes has to consider that one of them may have a smaller value because of a rollover.

When comparing two sequence numbers an interval is used defined as:

$$\text{SEQ_NUMB_INTERVAL} = \frac{\text{Integer.MAX_VALUE}}{2}$$

Should a sequence number have encountered rollover recently, it means the numerical difference between the two sequence numbers is greater than the interval. The only other situation where the numerical difference is greater than the interval, is if a node has had a forward route, which has not been inactive, broken or updated for relatively long time. This situation is assumed impossible, given the conservative route management of the protocol. The assumption is especially reasonable when considering the dynamic topology of an wireless mobile ad-hoc network.

If the sequence numbers are from the same generation (i.e. they have both have encountered an equal amount of rollovers), comparison is then a simple $>$ operation. For further details, the source code is in Appendix E section E.1.0.9.

5.4 Routes

As it is noticed in the previous implementation sections, route information is accessed by multiple threads. These threads may read or modify the route entries of the route tables, and also modify the datastructures used by the tables to hold their entries. Route tables are synchronized on a `tableLock` object each, when methods are used that alter the tables structurally. The entries are also synchronized on objects used as locks, when several threads may alter e.g. the TTL value. For exact synchronization details it is referred to the source code of the `routes` package in Appendix E.2.

The timer-thread, should not busywait if the two tables do not contain any entries. The thread thus have to wait in a condition-queue if that is the state, and notified if the condition do not hold. This is achieved by having a `tableLock` object, that the thread and method that may alter the state, should synchronize on. The condition of which the timer-thread tries to pass appears in codebox 5.4.

Codebox 5.4: Condition that prevents the timer-thread from busywaiting

```
1 synchronized (tableLocks) {
2    while (routeRequestTable.isEmpty() && forwardRouteTable.isEmpty
         ()) {
3       tableLocks.wait();
4    }
5 }
```

The methods that may alter the condition, are:

- `createRouteRequestEntry`
- `createForwardRouteEntry`
- `setRouteRequestTimer`

Whenever one of these methods are used, they notify the thread that may be lying in the condition-queue of the `tableLocks` monitor. This is done by acquiring the monitor, and then using the `notify()` method of that object.

5.4.1 Creating Forward Route Entries

The route table managers `createForwardRouteEntry()` method handles creation of routes. It creates an instance of the `ForwardRouteEntry` class and tries

to add the entry to the forward route table by calling `addForwardRouteEntry()` in the `forwardRouteTable` class.

Should the entry to the forward table be added successfully, the timer-thread is notified. If the `notifyObserver` parameter of the create method is set to `true`, then the `notifyAboutNewNodeReachable()` method of `Node` is called, with the address of the created destination as the parameter.

Since an entry is added to the route table, an `InternalMessage` is queued in shared queue `pduMessages`. This is achieved by using the `queuePDUmessage()` method of `Node`, so the route table manager do not know of the `Sender` class directly. The node then calls the queue method of the `Sender` class.

When a `ForwardRouteEntry` is added to the forward route table through `addForwardRouteEntry()`, the table has to ensure that a route is not added unless the destination is not known already known. The forward table therefore hold the entries in a `HashMap` datastructure, with the address of the destination as the key. Since it is required that the addresses of the node must be unique, it is ensured only *one* route for each destination is known. Whenever a route needs to be fetched, the `HashMap` is also used as such structure offers fast lookup.

For optimal performance, the table also holds a corresponding `LinkedList`, which is sorted so the entry with the smallest TTL value is kept in the head. The table is then able to efficiently fetch the first entry that is to time out.

Whenever an entry is added, it will have a TTL value set to `Constants.ROUTE_ALIVETIME`. This value is the maximum TTL value that a forward route can have, therefore new entries are added in the tail. Only if the entry was successfully added to the `HashMap`, the entry is added to the sorted list.

5.4.2 Creating Route Request Entries

Creation of route request entries are very similar in the implementation for creation of forward route entries. The `createRouteRequestEntry()` is used through the table manager whenever creation of route request entries is needed. The method creates an instance of a `RouteRequestEntry`, and tries to add the entry by using the `addRouteRequestEntry()` method of the route request table. The route request table has a `HashMap` and a sorted `LinkedList` as in the forward table to hold its `RouteRequestEntry` objects.

A route request is not uniquely defined by the destination address alone, as with

forward route entries. It is the pair of a source address (the originator of the RREQ PDU) and the broadcast ID. The table therefore defines an internal private class `EntryKey` that override both the `equals()` and `hashCode()` methods so a unique key is used to add entries to the `HashMap`. This is done by calling the `hashCode()` function on a string consisting of the two integer values which are separated by a semicolon. The `equals()` method simply defines that two `EntryKey` objects are equal, when each of the two integers are equal (using the == operator).

In the case where the attempt to add the entry succeeds, the following step is to add the entry to the tail of the `LinkedList` if only the `setTimer` boolean is set. In that case the timer-tread is also notified by acquiring the monitor of the `tableLock` object and waking the thread.

5.4.3 Handling Old Route Entries

When the timer-thread passes the condition of Codebox 5.4, it means that there is a *possibility* that routes may have become stale. In order to prevent unnecessary looping in the time that no stale routes exist, the timer-thread is set to sleep. The amount of time it should sleep is calculated by calling a private method named `getMinimumTime()`. This method returns the system time in milliseconds of the first entry to expire from the two tables
The time that the thread is set to sleep, is then the difference between the result and the current system time (also in milliseconds). Before the `sleep()` method is used on the thread, it is checked that the time to sleep is non-negative.

When the timer-thread is woken, it resumes handling each of the two tables for expired routes. The `run()` method of the timer is thus separated in two sequential parts. In each of the two parts, the timer-thread tries to fetch the route entry that is first to expire. This may result in a thrown `NoSuchRouteException` if the table is empty. It is simply caught, so the thread can proceed.

Should the fetch in one of the two parts succeed, the timer will compare TTL of the entry with the current system time to see if the TTL value is less than the current system time. This comparison is made in a while-loop, since more than one entry may have expired in the time the thread has slept.

1. Handling stale `RouteRequestEntry` objects.

   When handling expired route request entries, any entry that has exceeded its TTL, is removed immediately from the request table by `removeEntry()`.

If the entry is buffered because the corresponding RREQ PDU was originated by the node itself, it is checked if the request has been accommodated with a created valid route to the destination. This is achieved by calling the `validForwardRouteExists()` method of the table manager. If it returns true, then nothing more is done for this entry and the next entry to expire from the table is handled.

If the request is not accommodated with a valid route to the destination, then a new RREQ PDU is created, the entry reinserted in the request table with a fresh TTL and incremented broadcast ID. The PDU is afterwards queued for sending. In order to ensure that a request is not sent more than the allowed times, each request entry count the times it has been resent. A new RREQ is only sent if the entry's `resend()` method returns true. Otherwise the timer will queue an internal message to the sender, such that the packets for that destination is removed. Also the application layer is notified through its notification method for route establishment failure.

2. Handling stale `ForwardRouteEntry` objects.

   First the timer checks if the fetched entry is a *valid* route to a neighbour by checking the `getHopCount()` value. Should the entry be a local valid link, the entry is set to invalid, and `findBrokenRoutes()` is invoked on the forward table.
   This method iterates over all entries of the table, looking for routes that uses the local link as a next-hop. Any routes that match this condition, are marked as invalid. A RERR is created for each match, each of them containing the precursors for that route. The method then returns an `ArrayList` of `RERR` objects. The timer-thread then iterate over the list and queues any RERR PDUs for sending.

   If the expired link is not a local link, the route is marked as invalid and the application layer is notified by calling the `notifyAboutRouteToDestIsInvalid()` method of `Node`. The expired route to some destination, is not removed immediately should that the last known information about the destination is not lost.

   If the fetched route was not handled in one of the cases, it means that the route has expired the time it is buffered as an invalid route. The forward route is therefore removed from the table.

5.5 Ad-hoc Network on Android

Chapter 4 Section 4.2.5 described that the `AdhocManager` offered two methods: For starting and stopping the ad-hoc network.

In order to create a network, the wireless adapter is disabled (if it is running). This is necessary to do, as it has to be reconfigured. The following shell command is then executed through the JNI interface.

Codebox 5.5: Invoking the c-library through a JNI call

```
1 int result = NativeCommand.runCommand("su -c \""+" startstopadhoc
      start "+phoneType+ip+"\"");
```

Codebox 5.5 shows, how the static `runCommand` method is used through the `NativeCommand` class. The shell command that is executed, run a binary file called `startstopadhoc` with superuser rights. The arguments given are *start*, *phoneType* and `ip`.
When executed, the binary file runs its main method. The main method checks that the arguments are the legal types and calls the corresponding method, passing the *phoneType* and `ip` as parameters. It will return -1 in the case of illegal arguments. The main method runs its `startwifi()` method that switches over the *phoneType* parameter, loads phone-dependent modules and configure the wireless adapter. The method also re-enables the adapter.

Codebox 5.6: Configuring for the Nexsus One Android phone

```
1  switch (*phoneType) {
2     case 0: //NEXSUS
3        system("insmod /system/lib/modules/bcm4329.ko");
4        snprintf(cmd, sizeof cmd, "ifconfig eth0 %s netmask
             255.255.255.0", ip);
5        printf("\ncmd: %s\n", cmd);
6        system(cmd);
7        system("ifconfig eth0 up");
8        system("iwconfig eth0 mode ad-hoc");
9        system("iwconfig eth0 essid nexusbac");
10       system("iwconfig eth0 channel 6");
11       system("iwconfig eth0 commit");
12       break;
```

Codebox 5.6 shows how the Nexsus One is configured by first loading a module and then setting the IP-address, netmask etc.ZZ Stopping the network also switches over the *phoneType* in the `stopwifi()` method. Stopping the network is much simpler. The wireless adapter is disabled and the modules are unloaded. For further implementation details it is referred to the source code in Appendix E.5.

5.6 Implementation Remarks

It should be noted that no formal specification of the AODV protocol exists. The AODV protocol is experimental, so modifications to the protocol may occur in the future. The implementation of the routing protocol is based on the specification that is presented in [15] and [17]. The following points clears any differences between the implemented protocol and the suggested implementation.

- A RREQ do not contain a TTL field, so a RREQ PDU is disseminated to the entire network. In other words, the *expanding ring search* technique is not implemented as specified in [15] Section 6.4.
- A RREQ PDU do not contain a gratuitous RREP flag that can be set, so a RREP is unicast to the destination. The current implementation will always send such a RREP if an intermediate node handles the RREQ, so that symmetric routes are always created.
- The RREP PDU do not have a TTL field (see [15] section 5.2) that can be set by the creator of the PDU. Each receiving node simply sets the TTL value for the route to `ROUTE_ALIVETIME`.
- The TTL field of a forward route entry is always set to,

  System.currentTimeMillis() + Constants.PATH_DESCOVERY_TIME

  when such a route is fetched. Thus the implementation do not differentiate between invalid routes or valid routes as it should by setting the TTL value to DELETE_PERIOD instead [15] Section 6.4.

Chapter 6

Library Test

This chapter will describe how testing has been conducted for the developed ad-hoc library.

In general, testing a wireless routing protocol is a difficult but important part of the development process. The main challenge is to test all possible scenarios of communication in an ad-hoc network with a wireless medium. Because of the wireless medium, in particular with Wi-Fi which have a long transmission range, it is hard to setup the scenario of multi-hop communication.

By setting up different connectivity scenarios and then execute a series of planned actions, it can be ensured that the routing protocol execute correctly. If the internal state of the protocol datastructure is consistent with the expected, and the result packet the internodal packet exchange corresponds to the excepted packets, then it may be concluded that the protocol function correctly. Such types of test are known as functional tests but the ability to verify correctness of such tests is limited to the combination of network setup and executed actions.

Another type of testing focus on verifying the correctness of the internal logic of the functionality. The internal logic is located in the methods that handles accesses to the datastructure. Such testing must cover every combination of parameters that is likely to invoke failure. Such testing is known as unit test.

The test conducted for the ad-hoc library is therefore divided into two types of testing:

1. Functional test
2. Unit test

The following two sections describes the types of tests. Each section state what the testing includes, how it has been conducted and the result of the tests.

6.1 Unit Test

The unit tests for the library are conducted by using the `JUnit` 4.8.2 test suite [5]. It allows the usage of *assertions* to test if logic holds, when functionality is used. The functionality tested are all public methods (including static methods) that return some kind of response such as a boolean, integer, exception etc.

The classes for which the unit testing is possible are `Receiver`, `RouteRequestEntry`, `ForwardRouteEntry`, `RouteRequestTable` and `ForwardRouteTable`. The mentioned classes except the receiver, are all classes from the `routes` package because this package is the datastructure of the AODV protocol containing testable functionality.
Only a single method is tested in the `Receiver` class, which is the static `getMaximumSeqNum()` method.

When testing the `routes` package, all possible inputs are given to each of the methods that can be tested. It is then checked if the expected result were returned (by assertion). The test conducted consist e.g. of verifying that only *one* entry per destination may be held in the forward table. It is also tested that a route entry (either a forward or a request route), cannot be created if e.g. the given destination address is invalid.

The results of the unit test is positive as seen in Figure 6.1. For the source code of the unit test, it is referred to Appendix D Section D.2.

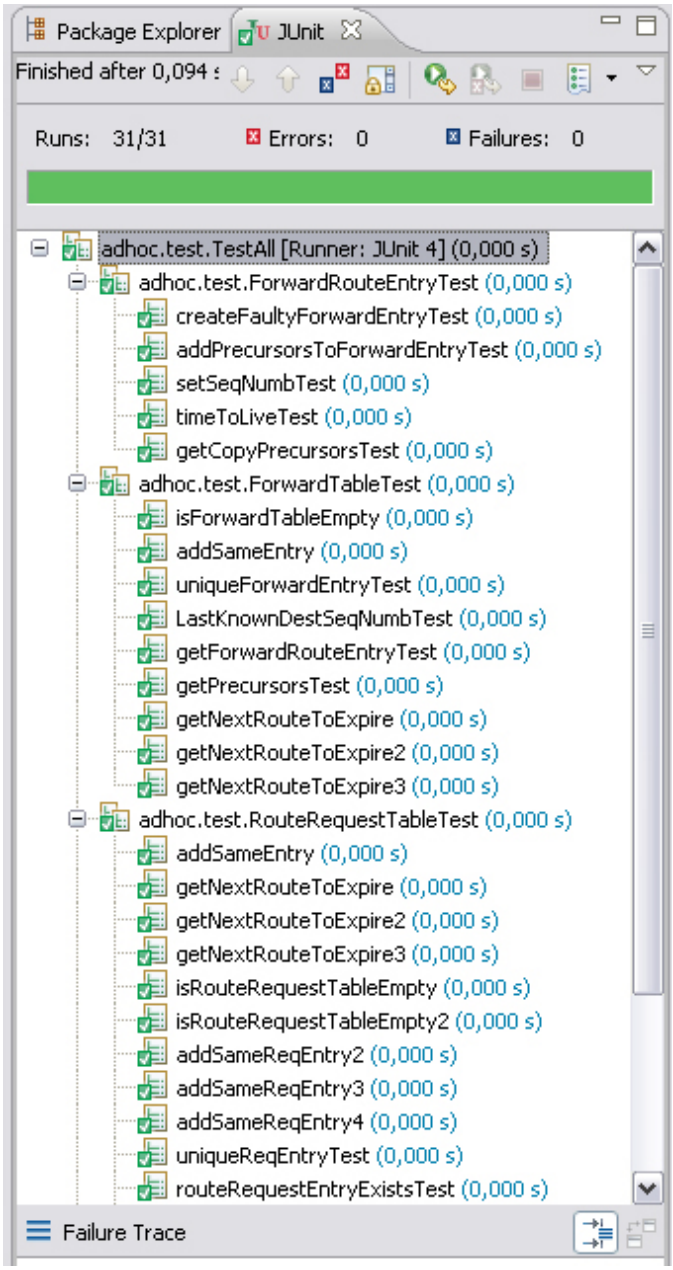


Figure 6.1: JUnit printout in Eclipse

6.2 Functional Test

The functional test that has been conducted in the ad-hoc library is divided into three separate parts:

1. Testing that the Data Link protocol (the UDP package) functions as expected.
2. Testing that the behaviour of the routing protocol is as expected.
3. Testing that the `setup` package is able to create and terminate an ad-hoc network on the available Android mobile devices.

6.2.1 Testing the UDP Package

The test of the UDP package was conducted by having a small application layer program trying to send data packets of different sizes and to different addresses (including the broadcast address). This is done by using the functionality of the UDP classes directly, so the routing protocol is ignored in this part. It should be noted that the `UdpReceiver` class had to be modified slightly such that it takes the test class as a parameter instead of the `Receiver` class. In this situation it is obvious that an observer-pattern between the data link layer and the network layer would have been a better design, since no modifications would have been required.

The application is designed so that all types of parameters were given so that packets of very large packets were attempted sent (larger than the maximum size of an UDP packet), and destination addresses, that both are invalid and valid. This small application is run on two laptops such that it also can be verified that receiving packets works as expectedly.

6.2.1.1 Wi-Fi Sniffing Application

Along with the functional tests of UDP, a Wi-Fi sniffing tool called Wireshark [6] has been used. This tool has been used to detect all messages that were transmitted by any reachable node, in the small testing environment that has been set up. Since this tool has been used by all the computers that participating in the functional tests, it is assumed that all transmitted protocol packages that would have be sent, would also have been captured by the tool.

6.2.2 Testing Protocol Behaviour

The functional test of the library is conducted by having a small test application (in the application layer), trying to send or broadcast application packets in two different setups. The source code of the test application can be seen in Appendix D Section D.1.0.1. The application creates instance of `Node` and executes the protocol by calling `startThread()`. The application implements the `Observer` class such that it is able to receive library notifications.

Since the routing protocol itself is not dependent of the operating system, these tests have been conducted on laptop computers running Windows XP and/or Ubuntu 10.04 LTS. The `setup` package for creating an ad-hoc network on Android is thus bypassed.

The wireless network adapters of these computers have been configured manually, so they where connected to the same ad-hoc network (with the same SSID), subnet and netmask. Each participating computer also had to be configured with a unique static IP-address.

In all the functional tests, the computers/nodes had a static position, unless otherwise is specifically stated. The functionality test of the library is divided into two scenarios:

1. Testing the functionality between two directly connected computers
2. And testing the functionality, when an intermediate node routes the communication between two computers

The setup of the nodes is illustrated in Figure 6.2

At any point in the tests, the nodes are referred to by the names specified in Figure 6.2. The two end-nodes are known as `src` and `dest`, while `inter` refers to the intermediate node.

In order to setup the scenarios of two laptops communicating through an intermediate node, the `udp` package has been modified slightly, so that any packets received from either `src` or `dest` are dropped in this layer. It enables the computers to be positioned at very close range without letting the routing protocol discover a direct routes to end points, thereby having the setup of Figure 6.2 with three nodes even though Wi-Fi have a longer transmission range.

Table following two table shows the steps that are executed sequentially with

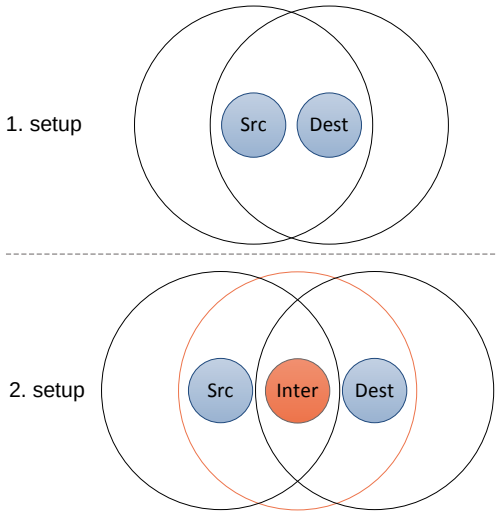


Figure 6.2: Setup for the computers participating in the functional tests

two and three nodes respectively. For each action it is stated what the node(s) should do and the excepted result.

When a new setup is needed, the routing protocol in each of the laptops are restarted, so no routing information is stored from the previous test setup.

Source node actions	Expected result
Start `dest` first and then `src` with 2 and 1 as their node address.	`src` and `dest` will broadcast and receive HELLO PDUs to and from each other, creating a symmetric route
`src` broadcast a packet containing the string "Broadcast test"	`dest` receives a notification from the library with the address of `src` and an array of bytes containing the data
`src` unicast a packet to `dest` containing the string "unicast test"	`dest` receives a notification from the library with the string and address of `src`
`src` unicast a packet to `src` containing the string "unicast test to src"	`src` node should filter such packets, so no notification will be received
`src` unicast a packet containing "RREQ fail test" to a node not in the network	`dest` participate in the RREQ flooding. `src` receives a notification about the route establishment failure after all retries fails
`src` unicast a packet containing "invalid address test" to a node with address -1	`src` receive a notification about the packet with an invalid destination address
`src` broadcast a packet with 54001 bytes of random data	`src` receives a notification about the size limit of a packet is exceeded
`src` unicast a packet with 54001 kilobytes of random data to `dest`	`src` receives a notification about the size limit of a packet is exceeded
`src` unicast a packet containing "dest stop now". When this packet is received in `dest` the node will stop the library for 10 seconds before restarting with the same node address.	`src` receives a notification that the route to `dest` is invalid, eventually deleted and that a new route known to `dest` when `dest` is restarted.
Stop both `src` and `dest`. terminate test.	

Source node actions	Expected result
Start `src`, `inter` and `dest` with 1, 2 and 3 as their node address.	`src` and `dest` will receive a HELLO from `inter` and create a route to `inter`. `inter` will receive HELLO PDUs from `src` and `dest`, creating routes to them
`src` unicast a packet to `dest` containing the string "unicast test"	`src` will initiate a route discovery. textttinter will reply with a RREP. `src` will receive a RREP, create a route and use it to send the packet. `inter` will forward the packet. `dest` receives a notification from the library with the string and address of `src`
`src` unicast a packet to a node not in the network	`inter` and eventually `dest` participate in the RREQ flooding. `src` receives a notification about the route establishment failure after all retries fails
Stop textttdest node and let `src` unicast a packet, containing "testing RERR", immediately after.	`inter` will fail to forward the packet and unicast a RERR to each precursor (`src`). `src` will receive a notification informing that the route to `dest` is invalid
Restart `dest`	`inter` wil receive a HELLO and create a route to `dest`. `dest` will also receive such a PDU and create a route to `inter`

In order to verify that the expected result and internal protocol behaviour is as it should be, there has been relied heavily on the information from printouts in crucial part in the library. These printouts are only enabled if the print stream of the `Debug` class is set. The complete results of these printouts for each of the two setups are located in Appendix D Section D.1.0.2 and D.1.0.3. From the printouts it can be concluded that the tested sequence of action in the two setups result in the desired behaviour of the routing protocol.

6.2.3 Testing Ad-hoc Network Setup on Android

The `setup` package has been tested by simply copying the shared library `adhocsetup.so` into each of the two phones and executing the main method of the `startstopadhoc` binary with parameters for starting and stopping an ad-hoc network.

It should be noted that the tests have only been conducted on two Android phones, though others are supported as well:

- **HTC Hero** rooted Android 2.1
- **Googles Nexus One** rooted Android 2.2

The two phones were connected to a laptop each through a USB cable. The shared library was then copied and the binary executed by using a remote access shell called Android Debug Bridge (ADB). ADB is simply an Android developer tool that can be used in situations as these.

When an ad-hoc network were created or terminated, the laptops were used to verify that, simply by attempting to connect to the network (with a static IP).

Chapter 7

Example Application: Text Messenger

This chapter describes the development of a text messenger application that uses the `ad-hoc library`, which is described in chapter 4. How to use the `ad-hoc library` in a application requires some modification of the Android device it must run on there is a short guide in appendix section C. In the following text there shall be differentiated between text message(a message send by the user) and then a message (a PDU message created by the application)

7.1 Specification Requirements

As mentioned in chapter 1 the application must take advantage of the services that are provided by the ad-hoc library. Using the services requires that some data must be exchanged from one device to another on the ad-hoc network. A text messenger application has the potential for using the ad-hoc libraries services to do this the text messenger application must have following functionality:

- Add contact that are on the network

- See if a contact is online or offline
- Send and receive text messages in a chat
- Guarantee the delivery of messages if delivery is possible
- Guarantee delivery of text messages in the correct order
- Have more than one chat running at a time
- Have more than one other person in a chat

7.2 Analysis

The are different ways in which the specification requirements can be met, in this chapter these different approaches will be analysed to find the best solution, to fulfil the specification requirements.

7.2.1 Contacts

The first requirement is the ability to find contacts that are on the network. As described in chapter 3 the ad-hoc library offers the ability to broadcast which can be used to flood the network looking for a contact and then having the contact send back an acknowledgement (ack) message. Using this method there are certain problems that have to be address e.g. the loss of the message and double flooding from the found contact. The flooding message is not likely to be lost if every node rebroadcast but the acknowledgement message can easily be lost because the library does not guarantee the delivery of a message.

The AODV protocol specification in section 2.2.4.1 states that when a user message is to be send to a node where there exists no valid route a route discovery is made. This means that the above method for finding a contact will flood the network twice because the contact that has to send back the ack must request a route for sending the ack. Instead finding a contact can be done by sending a user message to the contact this will trigger a route discovery. If a route is found the user message will be send and a route between the two fries will be established, but if a route could not be found the library will notify the application with a route establishment failure, that can be handle by the application.

When a contact has been added there has to be some way to see if this contact is on- or offline. For this purpose a view is used which are described in chapter

2.3. The ability to view a list of contacts is not enough, there has to be some way to detect if the contacts are on- or offline. Again this is done easiest do by taking advantage of the information the ad-hoc library delivers, more details of how this is done will be discussed in the Design section 7.3.

7.2.2 Text messages

One of the essential things in a text messages application is the ability to send text messages, and to be certain that they are delivered and are delivered in the same order as they are send.

Sending messages can be done through the ad-hoc library, using the `sendData()` method which is described i chapter 4. Using this method does not guarantee that the messages that are send will reach its destination nor that it will reach it in the same order as it was send. There has to be an implementation in the application to ensure these requirements. This can be don by using a protocol using ack messages and a timer thread, combining this with the information the library returns in case of route failure. This will ensure the message gets resend until an ack is received or a route to the destination no longer exists.

Guaranteeing the delivery of the text messages in the right order, requires a Ordering algorithm, when there also is a requirement for more then one peer per chat, it must be ordered multicast. There are three types of multicast ordering. [9] In this case the most appropriate ordering would be FIFO since it is the ordering that requires the least communication between the nodes

7.2.3 Chats

The chat is where the text messages are displayed. Having more than one chat running at a time, requires that each chat is its own instance. The user has to have a way to navigate between the different chats, for this purpose a list view would be a good solution.

7.3 Design

In this section the design of the application will be described. The design is created from the findings in the Analysis section.

7.3.1 Model-View-Control

One of the core design principles for applications is the Model-View-Control design which Text messenger application follows shown below in the class diagram on Figure 7.1. The model part (represented by the blue classes) of the application contains the data structure and handles the incoming and outgoing messages to and from the ad-hoc library.The messages that are send between the different Android devices are contained in the `model.pdu` represented by the red classes. This part does not depend on any Android specific elements, and uses an observer pattern to communicate with the view which is described in section 7.3.2. The view part (represented by the green classes) are the part the user sees it is created using activities and views which are android specific. The control part (represented by the yellow classes) is the part that registers the user interaction with the view part.

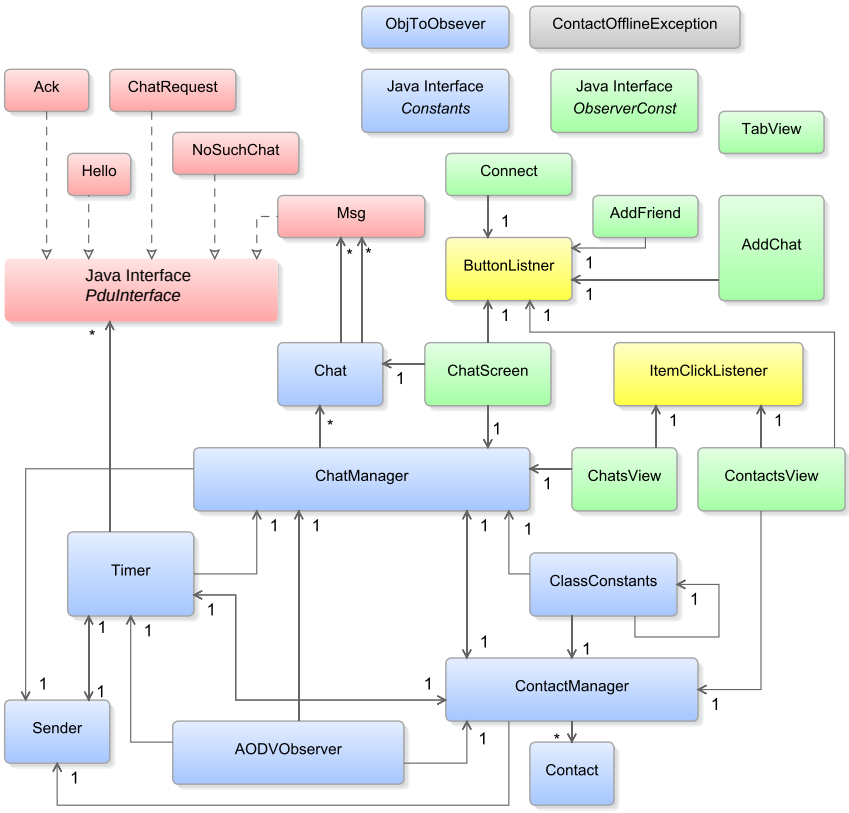


Figure 7.1: Class diagram over text messenger

7.3.1.1 Model

The model part contains the back-end of the textmessenger application, spread over different classes. The different classes and how they are connected can be seen in the class diagram on Figure 7.2

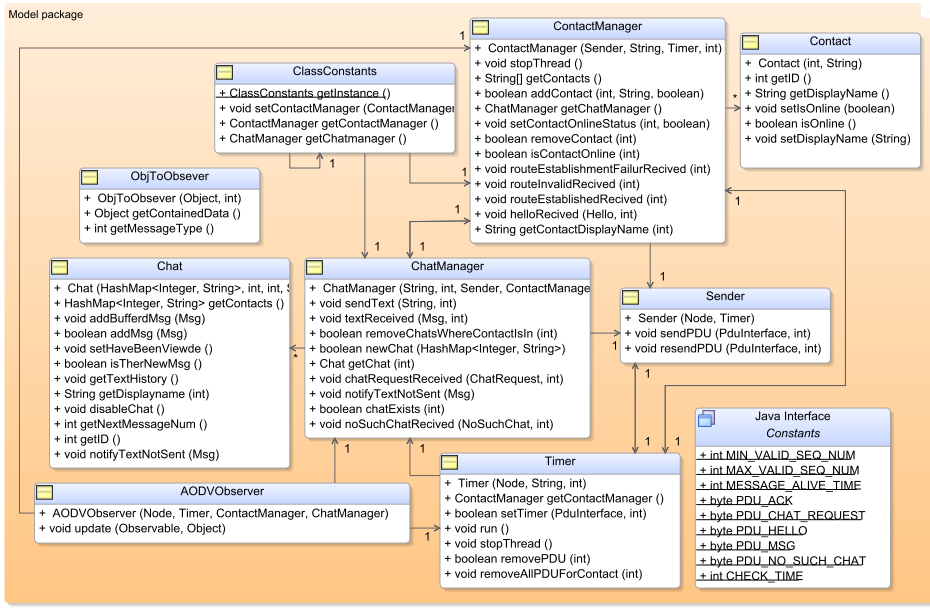


Figure 7.2: Class diagram over text messenger's model part

AODVObserver handles all the messages or notifications received from the ad-hoc library including data packages (all referred to as messages). The way the **AODVObserver** gets the messages is by observing the `Node` class in the adhoc library. When `update()` is called the `AODVObserver` gets passed an instance of the class `MessageToObserver`, which contains the information of which type of message is send, and the message. Then there is a corresponding action to each message, that is executed using a method in either `ContactManager`, `ChatManager` and the `Timer`. More information on the observer pattern is under section 7.3.2. The `AODVObserver` does not extend the thread class but relies on the thread from the calling node class, enabling it to preform actions concurrently with the other threads in the text messenger application.

Chat and ChatManager are the classes responsible for the chats, since the requirements are that there has to be more than one chat active at a time, there has to be an instance for each chat as concluded in the analyse. This is done by

having a `Chat` class that contains the information relevant to a single chat. The `chat` contains the information about who participate in the `Chat` as well as what messages have been send and received. In order to fulfilled the requirement of ordered messages using the `FIFO` method, the `Chat` has a `getNextMessageNum()` method, which is used to set the message number of the send text messages. When a text message is received the `addMsg()` method can use the message number to add the text message in the right order. When a change happens in the chat, the user sees this in the `ChatScreen` class, for this to happen the `Chat` class extends Observable, making it possible for the `ChatScreen` class to observe when new messages arrive as described in section 7.3.2.

The `ChatManager` class is responsible for handling all the different chat instances. The `ChatManager` class uses the method `newChat()` to create new chats, when this is done a request is send out to the contacts that have been included in the chat. Each chat gets its own ID, using the `createChatID()` this creates a unique id for each chat combination of contacts that are in the chat. The consequence is that there can exist only one chat which contains the exactly the same contacts. When receiving a chat request the `ChatManager` handles this by creating a new chat, with the same chat ID, and sending a acknowledgement message back to the creator of the chat.
When a contact goes offline (when there cannot be made a connection to the device) the chat should be closed, this is done using the `removeChatsWhereContactIsIn()` method, this will find all the chats the contact is in, and then remove the chats. A `Chat` should also be removed if the other contacts no longer has the `Chat`, if this happens the `ChatManager` will receive a `NoSuchChat` message and the `Chat` will be removed.
Sending and receiving text messages goes through the the `ChatManager`, when sending a text messages the `ChatScreen` call the `sendText()` method, this method uses the `getNextMessageNum()` from `Chat` to get the right message number before sending the text message. The the `textReceived` method is call when receiving a text message which calls the `addMsg()` in the chat corresponding to the chat id.

Contact and ContactManager are responsible for managing the contacts. A instance of the class `Contact` is used to store all information needed on a contact. The `Contactmanager` is the class that handles the contacts, the main purpose is to add new contacts, remove contacts and make sure that the contacts online status is updated. The adding and removing off a `Contact` is done through the methods `addContact()` and `removeContact()`. Detecting when a `Contact` has gone offline is done by using the status messages from the ad-hoc library, and acting according to them the behaviour is described in section 7.3.2. When the contacts are offline there are two ways they can be set back to online. Ons is if a message is received from that contact. The other way is if they are discovered, by the `helloToOffline()` method which is called with a certain interval, by

the inner class `CheckOfflineStatus` which extends thread.

Sender is used for sending messages, it assigns each message with a unique sequence number, which is an essential part of the `ack` protocol and then sends the message using the `sendData()` method in `Node` to send it. It offers a `sendPDU` function, that is used to send messages, as well as a `resendPDU` function that does not add a new sequence number and is used if delivery was not acknowledged.

Timer is the class where the outgoing messages are stored, and with certain intervals are resend using `Sender`. The messages are stored here and will be resend until an acknowledgement of their delivery is received or the `Contact` they are send to goes offline. Ensuring the requirement that a messages that is send will reach its destination if possible.

7.3.1.2 View

The user interface consists of several different classes, with different layouts depending on which state the program is in.

A diagram of the different classes is shown above in Figure 7.3. The `Connect` is the main activity and will be the activity where the user can input a "display name" an then press the connect button which will start op the android device in ad-hoc mode and start all the threads in ad-hoc library. When connected the user will be presented with a "tab" view containing the `ContactsView` in one tab and `ChatsView` in the other tab. From the `ContactsView` there is a button that which will start the `AddFriend` view that is used to add a contact. `ContactsView` activity will also contain a list of all contacts and display their online status. `ChatsView` will display a list of chats that can be entered and displayed trough the `ChatScreen` activity. The `ChatsView` will also have a button activating the `AddChat` activity.

7.3.1.3 Control

For the control part the are two listeners the `ButtonListner` and `ItemClickListener`.

The `ButtonListner` is used to listen on buttons in the activities, the button listener takes an activity as parameter enabling it to call the parent class. The buttonlistner is added to a button by adding a `OnClickListner`, when the

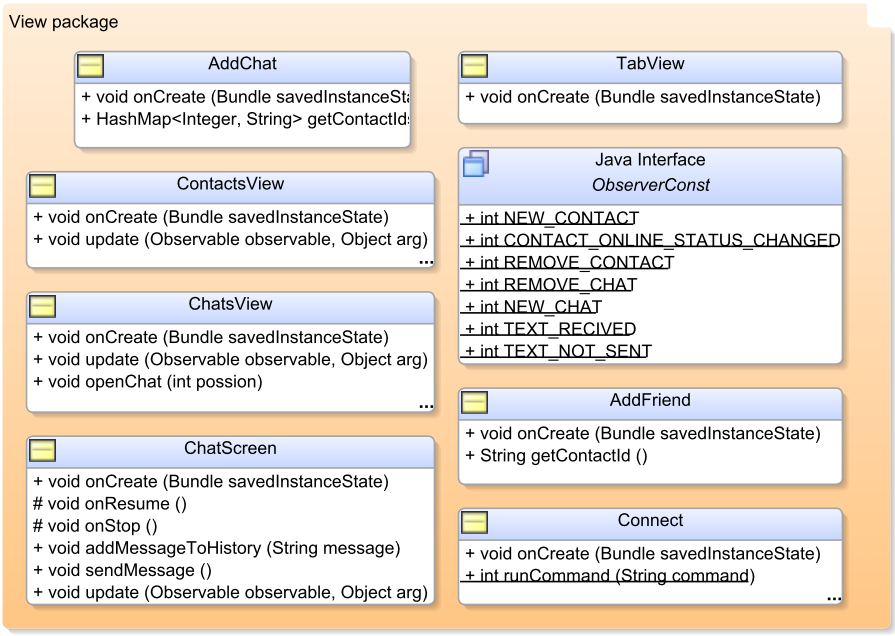


Figure 7.3: Class diagram over text messenger's view part

button is clicked the `ButtonListner` chooses which action to preform based on the button view id.

The `ItemClickListener` is used in the same fashion as the `ButtonListner` but can me used on Items and is added by using `setOnItemClickListener` on a item.

7.3.2 Observer patterns

The observer pattern is a way of implementing messages passing between the layers in the application ensuring a loose coupling between the ad-hoc library and the model layer as well as between the model layer and the view layer. A model of the observer pattern can be seen in Figure 7.4.

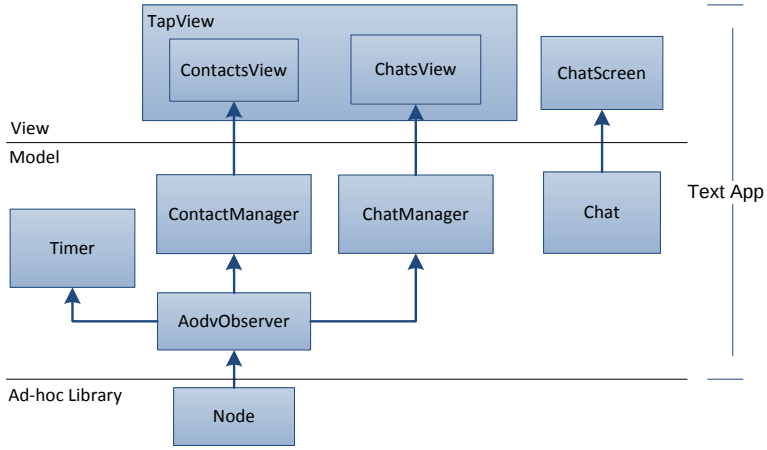


Figure 7.4: Overview of the Observer Pattern

7.3.2.1 ad-hoc library to AODVObserver

The `AODVObserver` class acts as an observer of the `Node` class in the ad-hoc library. When the `Node` class updates the `AODVObserver` will receive a instance of `MessageToObserver`. This message contains a integer indicating what type of data is in the message and the data. There are six different types of messages that can be received.

Route establishment faliure message indicates that a contact can not be reached and therefore must be offline. The method `routeEstablishmentFailurRecived()` method is then called in the `ContactManager` that handles everything involving a contact going offline.

Data received is the text messengers own PDU messages that are send between the applications, these pdu messages are handled by a method called in either the `Timer`-, the `ContactManager`- or the `ChatManager` class where the PDU message is handled, as seen in Figure 7.4;

Invalid destination address is received if the application tries to send a messages to a destination address that can not exist, in this case if the address is outside the range of 0 to 255 because of the ip protocol.

Data size exceeder max is received when the application is trying to send a message that exceeds the maxsimum value set for a package. In the design of the ad-hoc library section 4.1.3 it is decided to use UDP which gives a limitation

of package size. If this is received the package must be removed from the `Timer`.

Route invalid is received each time a route is set invalid, this could mean that the contact has gone offline or that the connection has not been used for a period of time. The message is handled by calling the `routeInvalidRecived()` method in the `ContactManager` class (that handles it).

Route created is received each time a new route to another device is established, this messages is handled by calling the `routeEstablishedRecived()` method in the `ContactManager` (class that handles it).

7.3.2.2 Model to View

In the view layer there are three views that extends `Observer` as show in Figure 7.4 the three views are:

ContactsView observes on the `ContactManager` class in the model layer. The `ContactsView` can get three different messages from the `ContactManager`, Contacts online status has changed this message is handled by updating the view to display the change for the user. The second message is "new contact" which adds the contact to the view and updates it and the third is "remove contact" which removes the contact from the view and updates the view to display the change for the user .

ChatsView observes on the `Chatmanager` class in the model layer.
The `ChatsView` can get three different messages from the `Chatmanager`, "new chat" when a new chat is created the chat is added to the view and the view is updated and "remove chat" removes a chat from the view and updates the chat. The last message is "text received" which indicates that a chat has received a new text message, an indication in the view is added and the view is updated for the user to see.

ChatScreen gets added to a observable chat every time it is opened and removed when the **ChatScreen** is closed. This is because there can exist many chats but there is only one `ChatScreen`. When pressing on a chat in the **ChatsView** the **ChatScreen** opens with this specific chat, and is added as an observer to the chat until the `ChatScreen` is closed. The `ChatScreen` can receive two types of messages "text received" which adds the text to the view and updates it. The second type of message is the "remove chat" messages and will close the `ChatScreen` activity and return back to `ChatsView`.

7.4 Implementation

The overall design of the application was discussed in the `Design` chapter, in this chapter the focus will be on the implementation of the classes where the `ad-hoc library's` services are used.

7.4.1 Sender

The `Sender` class consists of two public methods and a private, the first public method is `sendPDU()` which is an instance of the `PduInterface` and a contact id. The pdu package gets a sequence number using the private method `getNextSequenceNumber()`, that returns an integer, it is not guaranteed that this number is unique because of the role-over effect. But it is assumed in the program that it is unique considering the amount of integers available and the expected lifetime of a pdu package in the system. If the sequence number does not already exist in the timer, it is put into the queue and then send using the `Node` class method `sendData` which is described in chapter. 4.2.1.1 giving the sequence number, destination id and a byte string, as arguments. The byte string is created using the `toBytes()` method implemented in the PDU class. The second public method is `resendPDU()` which only uses the `sendData` method to send the pdu, not giving it a new sequence number or putting it in the `Timer`.

7.4.2 Timer

To manage the PDU messages a `LinkedList` "aliveQueue" is used. This contains the PDU messages and a `HashMap` "pduIdentifiers" with the PDU sequence number as the key and the contactID of the destination as the value. The timer runs in its own thread and when the aliveQueue is not empty it sleeps until the lifetime of the top PDU message in the queue is expired. When the lifetime expires it goes trough the queue until it reaches a PDU message where the lifetime is not expired. If the lifetime of a PDU has expired it is resend using the resend method in the `Sender` class. To do this it must make a lookup in the pduIdentifiers to find which contact the message must be send to. When the PDU messages has been resend the message is moved from the top to the bottom of the aliveQueue, and its lifetime is reset.

7.4.3 Send and receive text message

When the user presses the send button in the `ChatScreen` view, The `sendText()` method in `ChatManager` is called, taking a String and a chatID as arguments. The flow of the message can be seen in Figure 7.5.

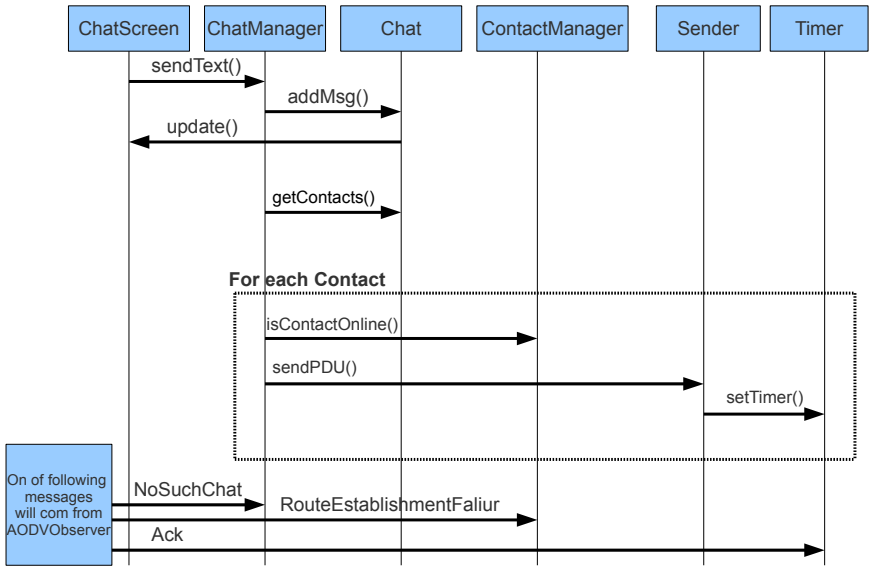


Figure 7.5: Sending a text message

The `sendText()` creates an instance of the class `msg` that takes four arguments. The first is a String, containing the text message, the second to ensure that the messages are displayed in the right order when they are received by the contact. For this the `getNextMessageNumber()` method is call in the `Chat` corresponding to the chatID, the method increments an integer and returns it. The next argument is `myContactID` which is an unique id that is used to identify where the message was send from.

The last argument the `msg` constructor takes, is the `chatID`, this is a unique hash for this chat, and is the same for every contact that is participating in the chat. Making the `chatID` is done with the function `createChatID()` which takes the list of contact ID's and sorts them, using insertion sort, then puts them in a String separated by semicolon. String is the hashed and the hash sum is then used as the `chatID` ensuring that that another chat containing the same

contacts will have the same ID.

When the `msg` has been created it it is send to each contact using the `sendPDU()` function in the `Sender` class, which is described in section 7.4.1.

When the message has been send and put in the `Timer`, there are three types of messages that can be returned. If all goes well a acknowledgement(ack) message will be return from the destination contact. If this happens the PDU will be removed from from the `Timer` and nothing else will be done. Another message that can be received from the destination contact is the `NoSuchChat` messages, if the contact does not have the chat running the chat is no longer valid and the `NoSuchChat` message is send. The `noSuchChatRecived()` method in `ChatManager` will be called and a ack message will be send and the chat will be removed.

The last message that can be returned is the messages that comes from the library layer and not from the contact, it is the `ROUTE ESTABLISHMENT FAILURE`, this message will be handled by the `ContactManager`, and will end in the chat being removed and the text message will be removed from the timer.

7.4.4 Adding contact

There are three ways a new contact can be added, either the user adds a contact and a hello messages is send to that contact or a hello messages is received from a unknown contact, which will add that contact to the list.

When the user adds a contact the `addContact()` method is called in the `ContactManager` class, which results in the contact being added with its status as offline. A `Hello` message, that contains the users display name and a boolean requesting a return hello message, is send to the new contact. The request for a return hello message is to ensure that the contact is online, and to get the contacts display name.

When the contact receives the `Hello` message, it is handled by the `helloRecived()` method if the contact(that send the hello) does not exist, the `addContact()` method will be called but wit the `sendHello` boolean set to false. Else the contacts status will be set to online. An illustration of the exchange of hello messages can me seen in Figure. 7.6

When the return hello is received the `helloRecived()`, updates the online status of the contact as well as the display name and notifies the observer of the change.

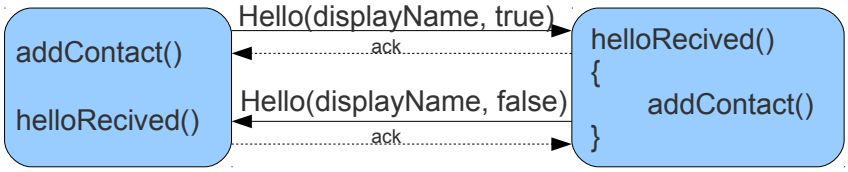


Figure 7.6: Adding a contact

The third way a contact can be added is if a route is established to another contact in the underlying library layer, when this happens the `AODVObserver` will receive the messages and the call the `routeEstablishedRecived()` Codebox 7.1 which sends a hello message to the contact, requesting a return hello message. The contact will then be added when the return hello is received.

Codebox 7.1: routeEstablishedRecived

```
1 public void routeEstablishedRecived(int contactID){
2       if(!contacts.containsKey(contactID) || !contacts.get(
            contactID).isOnline()){
3          Hello hello = new Hello(myDisplayName, true);
4          sender.sendPDU(hello, contactID);
5       }
6    }
```

7.4.5 Checking contacts online status

When a contact has been added, the contact can either be online if in reach or offline if out of reach. Discovering a contact is offline is achieved by using the messages from the ad-hoc library, when a route is set invalid, it means that the route has not been active for a period of time. This could mean that the contact i offline, therefore a Hello message is send to the contact, if a ack message is return the contact is online and nothing is done. If on the contrary a Route Establishment Failure PDU message is returned to the `AODVObserver` the `routeEstablishmentFailurRecived()` method is called and the status of the

contact is set to offline. All chats where the contact participated are removed, and all messages sent to that contact that are in the timer are removed as well.

When a contact is set to offline, the boolean `offlineExists` is set to true and the thread listed in the Codebox 7.2, is notified and will call the `helloToOffline()` method. This will set the `offlineExists` to false and send hello messages to all offline contacts requesting a reverse hello. Either a hello message is received from the contact setting it online again or a route establishment failure is received making sure that the `offlineExists` is again set to true. When the tread then awakes, after a certain period of time, the `helloToOffline()` method will be called again. If all contacts is online the tread will wait until a contact is set offline, ensuring no occurs of busy wait.

Codebox 7.2: Thread ensuring a regular offline status check

```
1  public void run() {
2            while (keepChecking) {
3               try {
4                  sleep(Constants.CHECK_TIME);
5                  synchronized (contacts) {
6                     while (!offlineExists)
7                        contacts.wait();
8                  }
9                  helloToOffline();
10              } catch (InterruptedException e) {
11              }
12           }
13        }
```

7.5 Text Messenger test

To ensure that no problems or failures within the program will occur tests must be made. There have been performed two kinds of black box test on the text messenger application, a JUnit test and a functional test. The JUnit test is used to test some of the main methods in the model part of the program whereas the functional test is used to test the user interface and that the program behaves the way it should.

7.5.1 Android JUnit test

The Android framework has an extension of the JUnit framework that makes it possible to create a test project within the Android SDK. The main use of

the JUnit test has been on the model part manly because the view part of the program does not have the same complex methods that can be isolated and tested.

The test was created as in a normal JUnit test by creating instances of the classes that are to be tested and then using assertions on the specific methods that are to be tested. Assertions is used just as in a normal JUnit test, to test different scenarios and expecting a certain return value.

The test was performed on `Chat`, `ChatManager`, `ContactManager` and `Timer` these classes where chosen because they contained methods whit a specific purpose where there where a return value to check on. The other classes in the model part where not test because they manly contained private methods and simple getters and setter methods which is not interesting to test.

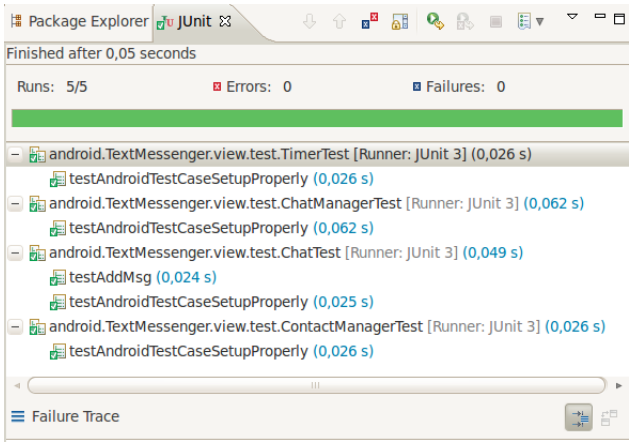


Figure 7.7: Android JUnit test result

The results of the test can be seen in Figure 7.7 and as seen all the tests ran with success, meaning that the methods that where testes behaved as expected. The actual test can be seen in Appendix D Section D.3.

Chapter 8

Improvements and Optimizations

The following chapter discusses, how the library and the Android application may be improved, and the performance optimized. The suggested enhancements may require modifications to the current implementation. What such modifications include, are also described.

8.1 Improving the Library

The improvements that may be made with the implemented library can be divided into two categories:

- Performance optimizations to the AODV routing protocol.
- Better customization of the ad-hoc network, making it usefull to a wider selection of Android applications.

The following two sections will describe the different optimizations and improvements that can be achieved, in each category.

8.1.1 Protocol Performance Optimizations

8.1.1.1 Controlled Flooding

The first area of optimizing AODV would be in the route discovery procedure, where the current implementation floods the entire network for each request.

An optimized flooding such as *expanding ring search*, described in section 2.2.2.1 could be used. This optimization will only require small changes in the implementation, and may reduce both the total protocol overhead and the delay time in a discovery, especially as the network size increases.

Two places has to be altered: The method that handles received route requests described in section 5.3.2 and the route request PDU. The PDU will have an additional integer field that tracks its TTL. The receiver method will only need to decrement the TTL with some amount (such as 1 if the hop-count is used as the TTL metric), and check that TTL is non-negative, before proceeding as usual.

8.1.1.2 Sending Multiple Data Packets from Application

Another area of possible optimizations is in the way the sender-thread handles data packets that are requested to be send from the application layer. The current implementation is only able to handle *one* packet from `userMessagesFromNode` at a time. Since the library does not guarantee the order in which the sent application messages are received at the destination, it is not necessary for the thread to skip the remaining user messages.

The suggested specification in [15] Section 6.3, state that a node should not originate more than `RREQ_RATELIMIT` requests per second. Massive protocol congestion is otherwise created, since discoveries relies on flooding. The rate limit should therefore be considered.

In order to know which messages are waiting for a success or failure response from an internal message, these messages must be buffered in a separate container. The sender should then skip processing this queue if an internal timer registers that the rate limit is reached.

8.1.1.3 Local Repair

The procedure of sending RERR messages to each precursor (and eventually reaching an end node) whenever a local link breakage occurs, can be optimised. Local repair [15] Section 6.12 is a possibility that may reduce the delay of letting the source handle a new RREQ, each time a break occurs. This may especially reduce the delay when the breakage occurs "far" from the source. Long routes with many intermediate nodes are likely to be created in large networks. Local repair can be applied in such situations.

The design of the library can be extended with local repair relatively easy, since the sender distinguishes between data packets that are to be forwarded and data packets that are sent from its own application layer. To implement a local repair procedure would thus only require a modification of the method `sendUserDataPacket()` used for sending data packets.

The method should initiate a route discovery procedure (possibly with a lower number of retry attempts), for such packages. The timer-thread will then prompt the sender-thread (through an `InternalMessage`), either of a re-established route or of a discovery failure.

It should be noted that it does not make sense to implement the local repair without using the *expanding ring search* technique, as every node would flood the entire network.

8.1.2 Ad-hoc network improvements

8.1.2.1 Custom SSID and Netmask

When an ad-hoc network is created on Android through the `setup` package, the netmask and SSID is hardcoded in the implementation. This means that a fixed number of nodes (255) are able to join the network, if each node acquires a unique address. It would have been preferable if the application layer had the ability to set an estimation of the network size, by giving the netmask as a parameter.

The way the netmask is specified depends on the phone, so different implementation is required for each supported phone. If such a change is planned in the future, several places in the library has to be altered. In that aspect the library is badly designed.

8.1.2.2 Phone Support

Support for more phone types is also an area which can be improved, since it would obviously enable a more usable library. This is easily achieved by simply adding another case to the `startwifi` and `stopwifi` methods of the `startstopadhoc.c` file. The supported phone should have its own constant value defined in the `PhoneType` class in order for the switch to work. The file would then have to be recompiled with the Android NDK so the shared library `adhocsetup.so` would be updated.

8.1.2.3 Completely Automated Network Creation

As the library is implemented in this project, the library require several things that are done be, before the creating an ad-hoc network can succeed. A brief users manual can be read in Appendix C. The usability of the library may be greatly improved if creation of ad-hoc network on Android where totally automated by using the library. Future work would definitely include this area of enhancements.

8.1.2.4 Creation of Unique Network Address

When the application layer creates a node object enabling it to use the routing protocol, it has to give a node address which *must* be unique across the ad-hoc network. How this is achieved is not an issue that the library care about. There is an opportunity for less responsibility for the application layer, by handling addresses in the library. The library could be expanded with a package which is responsible for auto-generating unique node addresses. The address could be generated by some mapping from the unique MAC address of the wireless adapter to a node address.

The issue is then to find an unambiguous mapping, which is computed locally in each node. Another proposal that requires more work, is to implement a distributed agreement algorithm [9] Section 12.3. The node would then propose an address to the network, that they all have to agree on before the node can join the network.

Regardless of how the unique address is found, it would be easy to implement with the modular design of the library.

8.1.2.5 Stream Abstraction

Possible applications that want to include the library, are restricted in the size of data they may send in a single packet. This is a consequence of using UDP. The current implementation offers no abstraction from a UDP packet. Such an abstraction would mean implementing a protocol that is located between the routing protocol and the UDP layer. This is necessary so the protocol can split messages that exceeds the maximum amount, into smaller packets for the UDP to send, and merge data into messages when receiving UDP packets. The protocol should be able to handle situations where a missing part of a larger message is lost in the UDP layer, and that parts is not necessarily received in the correct order. Buffering the parts, until the complete message is received is one solution.

The protocol is not a part of the library, since this type of abstraction can also be achieved in the application layer, should it be necessary. Implementing the protocol between the routing and UDP layers would only be useful if AODV PDUs also had unpredictable sizes.

8.1.2.6 Security

Security is a topic is left completely untouched, since it is not the focus of this project. This do not mean that the issue of security should not be considered at all. One way of securing the AODV protocol is described in [7]. Extending the library with a service for security should not be a problem, but is left for the developer of an application to add.

8.2 Text messenger improvements

The text messenger application was created as a small application that took advantage of the services the ad-hoc library offers. Because of the simple goal for the application the focus has not been on making the application a user friendly application thus leaving room for improvement.

8.2.1 Under the hood

Under the hood improvements is things that the user can not see this includes making sure that every user has a unique id on the network. This could be don with some sort of voting algorithm there are examples of such algorithm in [9]. Further more there the users "Display name" should also be unique making it possible to search after contacts not only on their contact Id but on their "Display name" as it is now the "Display name" is only used in the chat to show who send a message.

8.2.2 Contacts

Some improvements for the contacts are the ability to change ones "Display name" in real time so a user dos not have to close and restart the program to change display name.

Adding functionality to the contacts list when clicking on a contact there should be displayed a menu offering different options e.g. Send message.

8.2.3 Chat

As it is now when a contact goes offline the every chat that the contact was a participating off will close regardless of how many others there are in the chat. A big improvement would be that the chat stays open for the remaining contact and just buffers the messages so when the offline contact re-enters the contact will not have missed the conversation.

Another feature that would enhance the user experience would be the ability to add contact to existing chats.

There are a lot of features that can be added to enhance the user experience of the text messenger application. In this thesis the focus have not been on the user experience of the text messenger application but further development of the application could implement these features to improve the application.

Chapter 9

Conclusion

The main goal of this project is to develop an ad-hoc library. This has been achieved by having a layered design that meet the specified requirements.

The library is able to create an ad-hoc network on Android mobile devices. The library comprises of an routing layer implementing the Ad-hoc On-demand Distance-Vector Protocol in Java, a setup layer that is able to configure the wireless adapters of Android devices written in C and a UDP module for direct communication as the data link layer also in Java. Since the routing layer is written entirely in Java, this layer is independent of the operating system. The implemented routing mechanism is based on the AODV protocol specified in [15].

In the current implementation, the library is is not able to created and terminate ad-hoc networks without some manual configuration of the Android phones. The library require that the phones are rooted, and that some phone-specific files are located in a folder in the classpath (Appendix C state the details). Also the implemented AODV protocol is not fully optimized e.g. by the use of expanding ring search technique for flooding. Thus improvements to the library is possible in for both the setup module as well as the routing layer.

Three Android mobile phones are currently supported by the library namely HTC Hero, Nexus One and HTC Dream. Only first two phones are tested.

The Testing of the ad-hoc library consist of functional and unit tests. The functional tests cover the scenarios:

- of ensuring that the data link layer functions as required
- using the routing layer with two devices in communicating directly and through an intermediate device. These results of both the unit- and functional test shows that the ad-hoc library functions as intended.
- and finally starting and stopping an ad-hoc network with the two Android devices.

The second goal of the project is to develop an Android application that uses the ad-hoc library as "proof of concept". There has been developed an simple text messenger application that is able to locate and add contacts that are available in the network. Each chat is able to handle text messaging up to four contacts at a time, running multiple chats concurrently. For the application to run, it relies on the notifications that are received from the ad-hoc library.

The developed text messenger application is very simple, with many improving features that can be added for better usability experience.

The Android application meet the general requirements as a simple program for exploiting the functionality of the ad-hoc library.

Appendix A, B, ...

Appendix A

Workload Distribution

A.1 Report

Following table state, which chapters and sections each member of this group, has written and is responsible for.

Name	Chapters and Sections
Rabie	Abstract Chapter 1, Section 2.2, Section 3.2, Chapter 4, Chapter 5, Chapter 6, Section 8.1, Appendix, Bibliography, Preface
Lasse	Chapter 1, /resumé, Section 2.1, Section 2.3, Section 3.1, Chapter 7, Section 8.2, Conclusion

A.2 Programming

Both members of this group have contributed equally to the processes of programming the ad-hoc library and the Android application.

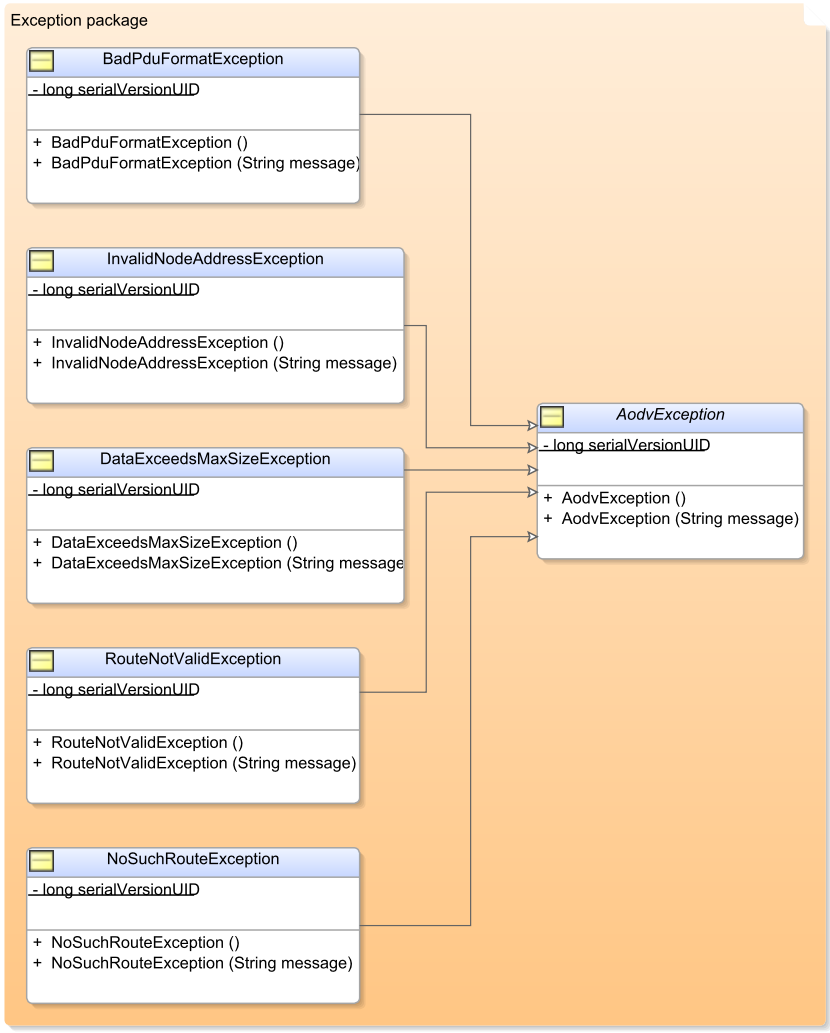
Adhoc Library test cases written by Rabie

Text Messeger test cases written by Lasse

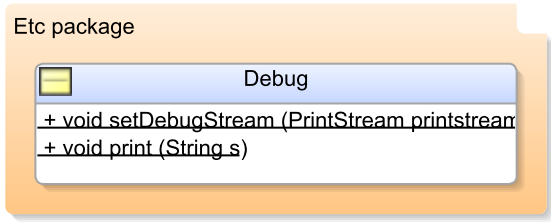
Appendix B

Expanded UML Class Diagrams

B.1 Exception package



B.2 Etc package



Appendix C

User Manual

In order for the ad-hoc library to work, the device which it is running on must be connected to an existing ad-hoc network. The ad-hoc library does not itself provide an ad-hoc network it only extends the services provided by the network by with multi-hop routing.

For Android devices there is at this point no API for creating an ad-hoc network therefore the ad-hoc library provides a library to set this up. These can be found in the package called `adhoc.setup`. Being able to use these library it is required that:

- The Android device is rooted
- The Android device supports Wi-Fi
- That the Android device has the command line interface ifconfig
- That the Android device has the command line interface iwconfg or a modified tiwlan.ini in the location "/data/local/bin/tiwlan.ini" (This depends on the phone model)

If the above requirements are met, the the two files nativ_task.c and startstopadhoc.c can be used in the Android application to create an ad-hoc network. The

nativ_task.c must be modified to point to the class used in the Android application. The two files must then be compiled using the Android NDK. The two compiled files will be startstopadhoc and libadhocsetup.so. The startstopadhoc file must be copied to the bin folder on the Android device.

To load the native library in the class this code has to be inserted

```
1 public void run() {
2    public static native int runCommand(String command);
3
4    static {
5       System.loadLibrary("adhocsetup");
6    }
```

when starting the ad-hoc network this command is used:

```
1 adHoc = new AdhocManager(this, WifiManager);
2 Connect.runCommand("su -c \""+" startstopadhoc start "+phoneType+"
    "+ip+"\"");
```

The ip must be the full ip as a String. The phone types currently supported can be found in the `PhoneType` class in the `adhoc.setup` package.

When stopping the ad-hoc network this command is used:

```
1 Connect.runCommand("su -c \""+" startstopadhoc stop "+phoneType+" "
    +ip+"\"");
```

Appendix D

Tests Source Code and Printouts

D.1 Library Functional Tests

D.1.0.1 protocol Behaviour With Two Nodes - Source Code

```
1  package adhocStressTest;
2
3  import java.net.BindException;
4  import java.net.SocketException;
5  import java.net.UnknownHostException;
6  import java.util.Observable;
7  import java.util.Observer;
8  import java.util.Random;
9
10 import adhoc.aodv.Node;
11 import adhoc.aodv.ObserverConst;
12 import adhoc.aodv.Node.MessageToObserver;
13 import adhoc.aodv.Node.PacketToObserver;
14 import adhoc.aodv.exception.InvalidNodeAddressException;
15 import adhoc.etc.Debug;
16
17 public class FunctionalTest implements Observer {
18    private Node node;
19    private Random generator = new Random();
20    private volatile boolean readyToResume = false;
```

```
21      private final static int srcAddress = 1;
22      private final static int destAddress = 2;
23
24      public static void main(String[] args) {
25          //TODO check that this address corresponds to the device.
26          new FunctionalTest(srcAddress);
27      }
28
29      public FunctionalTest(int myAddress){
30          try {
31              Debug.setDebugStream(System.out);
32              node = new Node(myAddress);
33          } catch (BindException e) {
34              e.printStackTrace();
35          } catch (InvalidNodeAddressException e) {
36              e.printStackTrace();
37          } catch (SocketException e) {
38              e.printStackTrace();
39          } catch (UnknownHostException e) {
40              e.printStackTrace();
41          }
42          node.addObserver(this);
43          node.startThread();
44
45          if(myAddress == srcAddress){
46              SourceActionsInFuncTestWithTwoNodes();
47          }
48      }
49
50      private void destActionsInFuncTestWithTwoNodes(){
51          node.stopThread();
52          try {
53              synchronized (this) {
54                  this.wait(10000);
55              }
56          } catch (InterruptedException e) {
57
58          }
59          node.startThread();
60
61          node.sendData(1, srcAddress, new String("terminate test now")
                .getBytes());
62
63          node.stopThread();
64      }
65
66      private void  SourceActionsInFuncTestWithTwoNodes(){
67          try {
68              synchronized (this) {
69                  while(!readyToResume){
70                      this.wait();
71                  }
72              }
73
```

```
74            node.sendData(1, 255, new String("broadcast test").
                  getBytes());
75
76            node.sendData(2, 2, new String("unicast test").getBytes())
                  ;
77
78            node.sendData(3, srcAddress, new String("unicast test to
                  src").getBytes());
79
80            node.sendData(4, 5, new String("RREQ fail test").getBytes
                  ());
81
82            readyToResume = false;
83            synchronized (this) {
84               while(!readyToResume){
85                  this.wait();
86               }
87            }
88            node.sendData(5, -1, new String("invalid address test").
                  getBytes());
89
90            byte[] data = new byte[54001];
91            generator.nextBytes(data);
92            node.sendData(6, 255, data);
93
94            readyToResume = false;
95            synchronized (this) {
96               while(!readyToResume){
97                  this.wait();
98               }
99            }
100
101           generator.nextBytes(data);
102           node.sendData(7, 2, data);
103
104           node.sendData(8, 2, new String("dest stop now").getBytes()
                  );
105
106           readyToResume = false;
107           synchronized (this) {
108              while(!readyToResume){
109                 this.wait();
110              }
111           }
112
113           node.stopThread();
114
115        } catch (InterruptedException e) {
116           e.printStackTrace();
117        }
118     }
119
120     @Override
121     public void update(Observable arg0, Object arg1) {
122        MessageToObserver msg = (MessageToObserver)arg1;
```

```
123         int userPacketID, destination, type = msg.getMessageType();
124
125         switch (type) {
126         case ObserverConst.ROUTE_ESTABLISHMENT_FAILURE:
127            //Note: any messages that had same destination has been
                removed from sending
128            int unreachableDestinationAddrerss  = (Integer)msg.
                getContainedData();
129            if(readyToResume == false &&
                unreachableDestinationAddrerss == 5){
130               readyToResume = true;
131               synchronized (this) {
132                  this.notify();
133               }
134            }
135            Debug.print("FuncTest: ROUTE_ESTABLISHMENT_FAILURE
                notification received - Unreachable node: "+
                unreachableDestinationAddrerss);
136            break;
137         case ObserverConst.DATA_RECEIVED:
138            byte[] data = (byte[])msg.getContainedData();
139            int senderAddress = (Integer)((PacketToObserver)msg).
                getSenderNodeAddress();
140            Debug.print("FuncTest: DATA_RECEIVED notification received
                 - from destAdr: "+senderAddress+" containing: "+new
                String(data));
141            if(senderAddress == srcAddress && (new String(data)).
                equals("dest stop now")){
142               destActionsInFuncTestWithTwoNodes();
143            }
144            if(senderAddress == destAddress && (new String(data).
                equals("terminate test now"))){
145               readyToResume = true;
146               synchronized (this) {
147                  this.notify();
148               }
149            }
150            break;
151         case ObserverConst.DATA_SENT_SUCCESS:
152            userPacketID = (Integer)msg.getContainedData();
153            Debug.print("FuncTest: DATA_SENT_SUCCESS notification
                received - packetID: "+userPacketID);
154            break;
155         case ObserverConst.INVALID_DESTINATION_ADDRESS:
156            userPacketID = (Integer)msg.getContainedData();
157            Debug.print("FuncTest: INVALID_DESTINATION_ADDRESS
                notification received - packetID: "+userPacketID);
158            break;
159         case ObserverConst.DATA_SIZE_EXCEEDES_MAX:
160            userPacketID = (Integer)msg.getContainedData();
161            Debug.print("FuncTest: DATA_SIZE_EXCEEDES_MAX notification
                 received - packetID: "+userPacketID);
162            if(readyToResume == false && ( userPacketID == 5 ||
                userPacketID == 6)){
163               readyToResume = true;
```

```
164                synchronized (this) {
165                   this.notify();
166                }
167             }
168             break;
169          case ObserverConst.ROUTE_INVALID:
170             destination  = (Integer)msg.getContainedData();
171             Debug.print("FuncTest: ROUTE_INVALID notification received
                     - for destAdr: "+destination);
172             break;
173          case ObserverConst.ROUTE_CREATED:
174             destination = (Integer)msg.getContainedData();
175             if(readyToResume == false && destination == destAddress){
176                readyToResume = true;
177                synchronized (this) {
178                   this.notify();
179                }
180             }
181             Debug.print("FuncTest: ROUTE_CREATED notification received
                     - to node: "+destination);
182             break;
183          default:
184             break;
185          }
186       }
187
188    }
```

D.1.0.2 protocol Behaviour With Two Nodes - Source Node Printout

```
1  Node: all library threads are running
2  Sender: broadcasting hello message
3  Sender: broadcasting hello message
4  Sender: broadcasting hello message
5  Sender: broadcasting hello message
6  Sender: broadcasting hello message
7  ForwardRouteTable: Adding new forward route entry for dest: 2
8  ------------------------
9  |Forward Route Table:
10 ------------------------
11 |Dest: 2 destSeqN: 1 nextHop: 2 hopCount: 1 isValid: true TTL: 3000
       precursors:
12 ------------------------
13
14 FuncTest: ROUTE_CREATED notification received  - to node: 2
15 Receiver: received hello pdu from: 2
16 FuncTest: DATA_SENT_SUCCESS notification received - packetID: 1
17 FuncTest: DATA_SENT_SUCCESS notification received - packetID: 2
18 FuncTest: INVALID_DESTINATION_ADDRESS notification received -
       packetID: 3
```

```
19 ------------------------
20 |Route Request Table:
21 ------------------------
22 |Dest: 5 destSeqN: 0 src: 1 broadID: 1 retries left: 2 hopCount: 0
       TTL: 3000
23 ------------------------
24
25 Sender: broadcasting hello message
26 Receiver: received hello pdu from: 2
27 Sender: broadcasting hello message
28 Receiver: received hello pdu from: 2
29 Sender: broadcasting hello message
30 Receiver: received hello pdu from: 2
31 RouteRequestTable is empty
32
33 ------------------------
34 |Route Request Table:
35 ------------------------
36 |Dest: 5 destSeqN: 0 src: 1 broadID: 2 retries left: 1 hopCount: 0
       TTL: 0
37 ------------------------
38
39 Sender: broadcasting hello message
40 Receiver: received hello pdu from: 2
41 Sender: broadcasting hello message
42 Receiver: received hello pdu from: 2
43 Sender: broadcasting hello message
44 Receiver: received hello pdu from: 2
45 RouteRequestTable is empty
46
47 FuncTest: ROUTE_ESTABLISHMENT_FAILURE notification received -
       Unreachable node: 5
48 FuncTest: INVALID_DESTINATION_ADDRESS notification received -
       packetID: 5
49 FuncTest: DATA_SIZE_EXCEEDES_MAX notification received - packetID:
       6
50 FuncTest: DATA_SIZE_EXCEEDES_MAX notification received - packetID:
       7
51 FuncTest: DATA_SENT_SUCCESS notification received - packetID: 8
52 FuncTest: DATA_RECEIVED notification received - from destAdr: 2
       containing: terminate test now
53 Node: all library threads are stopped
```

D.1.0.3 protocol Behaviour With Two Nodes - Destination Node Printout

```
1 Node: all library threads are running
2 ForwardRouteTable: Adding new forward route entry for dest: 1
3 ------------------------
4 |Forward Route Table:
5 ------------------------
```

```
6  |Dest: 1 destSeqN: 1 nextHop: 1 hopCount: 1 isValid: true TTL: 2998
       precursors:
7  ----------------------
8
9  FuncTest: ROUTE_CREATED notification received  - to node: 1
10 Receiver: received hello pdu from: 1
11 Sender: broadcasting hello message
12 FuncTest: DATA_RECEIVED notification received - from destAdr: 1
       containing: broadcast test
13 ----------------------
14 |Route Request Table:
15 ----------------------
16 |Dest: 5 destSeqN: 0 src: 1 broadID: 1 retries left: 2 hopCount: 1
       TTL: 3000
17 ----------------------
18
19 ForwardRouteTable: removing forward route entry for dest: 1
20 Forward Table is empty
21
22 ForwardRouteTable: Adding new forward route entry for dest: 1
23 ----------------------
24 |Forward Route Table:
25 ----------------------
26 |Dest: 1 destSeqN: 2 nextHop: 1 hopCount: 1 isValid: true TTL: 3000
       precursors:
27 ----------------------
28
29 updateForwardRouteEntry: Updating route for dest: 1
30 FuncTest: DATA_RECEIVED notification received - from destAdr: 1
       containing: unicast test
31 Receiver: received hello pdu from: 1
32 Sender: broadcasting hello message
33 Receiver: received hello pdu from: 1
34 Sender: broadcasting hello message
35 Receiver: received hello pdu from: 1
36 Sender: broadcasting hello message
37 ----------------------
38 |Route Request Table:
39 ----------------------
40 |Dest: 5 destSeqN: 0 src: 1 broadID: 1 retries left: 2 hopCount: 1
       TTL: 6
41 |Dest: 5 destSeqN: 0 src: 1 broadID: 2 retries left: 2 hopCount: 1
       TTL: 2999
42 ----------------------
43
44 ForwardRouteTable: removing forward route entry for dest: 1
45 Forward Table is empty
46
47 ForwardRouteTable: Adding new forward route entry for dest: 1
48 ----------------------
49 |Forward Route Table:
50 ----------------------
51 |Dest: 1 destSeqN: 2 nextHop: 1 hopCount: 1 isValid: true TTL: 3000
       precursors:
52 ----------------------
```

```
53
54 updateForwardRouteEntry: Updating route for dest: 1
55 ------------------------
56 |Route Request Table:
57 ------------------------
58 |Dest: 5 destSeqN: 0 src: 1 broadID: 2 retries left: 2 hopCount: 1
       TTL: 2993
59 ------------------------
60
61 Receiver: received hello pdu from: 1
62 Sender: broadcasting hello message
63 Receiver: received hello pdu from: 1
64 Sender: broadcasting hello message
65 Receiver: received hello pdu from: 1
66 Sender: broadcasting hello message
67 RouteRequestTable is empty
68
69 FuncTest: DATA_RECEIVED notification received - from destAdr: 1
       containing: dest stop now
70 Node: all library threads are stopped
71 Node: all library threads are running
72 FuncTest: DATA_SENT_SUCCESS notification received - packetID: 1
73 Sender: broadcasting hello message
74 Sender: broadcasting hello message
75 Sender: broadcasting hello message
76 Node: all library threads are stopped
77 Forward Entry: isValid has changed to: false
78 FuncTest: ROUTE_INVALID notification received - for destAdr: 1
```

D.2 Library Unit Tests

D.2.1 TestAll.java

```
1  package adhoc.test;
2
3  import org.junit.runner.RunWith;
4  import org.junit.runners.Suite;
5
6
7  @RunWith(Suite.class)
8  @Suite.SuiteClasses({
9     ForwardRouteEntryTest.class,
10    ForwardTableTest.class,
11    RouteRequestTableTest.class,
12    SequenceNumberTests.class
13 })
14
15 public class TestAll {
16
```

```
17 }
```

D.2.2 Routes package test

D.2.2.1 ForwardRouteEntryTest

```
1  package adhoc.test;
2
3  import static org.junit.Assert.assertEquals;
4  import static org.junit.Assert.assertFalse;
5  import static org.junit.Assert.assertTrue;
6
7  import java.util.ArrayList;
8
9  import org.junit.After;
10 import org.junit.Before;
11 import org.junit.Test;
12
13 import adhoc.aodv.Constants;
14 import adhoc.aodv.exception.RouteNotValidException;
15 import adhoc.aodv.routes.ForwardRouteEntry;
16
17
18 public class ForwardRouteEntryTest {
19    ForwardRouteEntry f1;
20
21    @Before public void setUp() throws Exception {
22       //create a valid forward route
23       f1 = new ForwardRouteEntry(0, 0, 1, 1, new  ArrayList<Integer
             >());
24    }
25
26    @After public void tearDown() throws Exception {
27       //set all fields to null
28       f1 = null;
29    }
30
31    @Test public void createFaultyForwardEntryTest(){
32       //dest address test
33       try {
34          new ForwardRouteEntry(Constants.MIN_VALID_NODE_ADDRESS -1,
                 1, 1, 1, new ArrayList<Integer>());
35          assertTrue(false);
36       } catch (RouteNotValidException e) {
37          assertEquals(e.getMessage(), "RouteEntry: invalid
                parameters given");
38       }
39       try {
40          new ForwardRouteEntry(Constants.MAX_VALID_NODE_ADDRESS +1,
                 1, 1, 1, new ArrayList<Integer>());
```

```
41            assertTrue(false);
42        } catch (RouteNotValidException e) {
43            assertEquals(e.getMessage(), "RouteEntry: invalid
                  parameters given");
44        }
45        //next-hop test
46        try {
47            new ForwardRouteEntry(1, Constants.MIN_VALID_NODE_ADDRESS
                  -1, 1, 1, new ArrayList<Integer>());
48            assertTrue(false);
49        } catch (RouteNotValidException e) {
50            assertEquals(e.getMessage(), "RouteEntry: invalid
                  parameters given");
51        }
52        try {
53            new ForwardRouteEntry(1, Constants.MAX_VALID_NODE_ADDRESS
                  +1, 1, 1, new ArrayList<Integer>());
54            assertTrue(false);
55        } catch (RouteNotValidException e) {
56            assertEquals(e.getMessage(), "RouteEntry: invalid
                  parameters given");
57        }
58        //seq numb test
59        try {
60            //Trying to create a forward route with an an Constants.
                  UNKNOWN_SEQ_NUMB
61            new ForwardRouteEntry(1, 1, 1, Constants.
                  FIRST_SEQUENCE_NUMBER -1, new ArrayList<Integer>());
62            assertTrue(true);
63        } catch (RouteNotValidException e) {
64            assertEquals(e.getMessage(), "RouteEntry: invalid
                  parameters given");
65        }
66        try {
67            new ForwardRouteEntry(1, 1, 1, Constants.
                  MAX_SEQUENCE_NUMBER +1, new ArrayList<Integer>());
68            assertTrue(false);
69        } catch (RouteNotValidException e) {
70            assertEquals(e.getMessage(), "RouteEntry: invalid
                  parameters given");
71        }
72        //hop-count test
73        try {
74            new ForwardRouteEntry(1, 1, -1, 1, new ArrayList<Integer
                  >());
75            new ForwardRouteEntry(1, 1, 0, 1, new ArrayList<Integer>()
                  );
76            new ForwardRouteEntry(1, 1, 1, 1, new ArrayList<Integer>()
                  );
77            new ForwardRouteEntry(1, 1, Integer.MAX_VALUE, 1, new
                  ArrayList<Integer>());
78            new ForwardRouteEntry(1, 1, Integer.MIN_VALUE, 1, new
                  ArrayList<Integer>());
79            //the hop-count can thus be set to any integer
80            assertTrue(true);
```

```
81          } catch (RouteNotValidException e) {
82              assertTrue(false);
83          }
84          //create route with with a 'null' instead of an ArrayList
                object
85          try {
86              new ForwardRouteEntry(1, 1, 1, 1, null);
87              assertTrue(false);
88          } catch (RouteNotValidException e) {
89              assertEquals(e.getMessage(), "RouteEntry: invalid
                    parameters given");
90          }
91      }
92
93      @Test public void addPrecursorsToForwardEntryTest(){
94          assertTrue(f1.getPrecursors().isEmpty());
95          assertFalse(f1.addPrecursorAddress(Constants.
                MIN_VALID_NODE_ADDRESS-1));
96          assertFalse(f1.addPrecursorAddress(Constants.
                MAX_VALID_NODE_ADDRESS+1));
97          assertTrue(f1.getPrecursors().isEmpty());
98          assertTrue(f1.addPrecursorAddress(0));
99          assertFalse(f1.addPrecursorAddress(0));
100     }
101
102     @Test public void setSeqNumbTest(){
103         assertFalse(f1.setSeqNum(Constants.FIRST_SEQUENCE_NUMBER-1));
104         assertFalse(f1.setSeqNum(Constants.MAX_SEQUENCE_NUMBER+1));
105     }
106
107     @Test public void timeToLiveTest(){
108         //forward entry: ensuring that TTL is a reasonable value
109         assertTrue(f1.getAliveTimeLeft()>0 && f1.getAliveTimeLeft()
                <= System.currentTimeMillis()+Constants.ROUTE_ALIVETIME);
110     }
111
112     /**
113      * Testing if the entry returns a COPY of the list of precursors
114      * so that synchronization issues are evaded
115      */
116     @Test public void getCopyPrecursorsTest(){
117         f1.addPrecursorAddress(1);
118         f1.addPrecursorAddress(2);
119         ArrayList<Integer> copy = f1.getPrecursors();
120         copy.remove(0);
121         copy.remove(0);
122         assertTrue(copy.isEmpty());
123         assertFalse(f1.getPrecursors().isEmpty());
124     }
125 }
```

D.2.2.2 RouteRequestEntryTest

```
1  package adhoc.test;
2
3  import static org.junit.Assert.assertEquals;
4  import static org.junit.Assert.assertFalse;
5  import static org.junit.Assert.assertTrue;
6
7  import org.junit.After;
8  import org.junit.Before;
9  import org.junit.Test;
10
11 import adhoc.aodv.Constants;
12 import adhoc.aodv.exception.RouteNotValidException;
13 import adhoc.aodv.routes.RouteRequestEntry;
14
15
16 public class RouteRequestEntryTest {
17    RouteRequestEntry r1;
18
19    @Before public void setUp() throws Exception {
20       //creating a valid rreq entry
21       r1 = new RouteRequestEntry(0, 0, 1, 1, 0);
22    }
23
24    @After public void tearDown() throws Exception {
25       //set all fields to null
26       r1 = null;
27    }
28
29    @Test public void createFaultyRouteRequest(){
30       //dest address test
31       try {
32          new RouteRequestEntry(Constants.FIRST_BROADCAST_ID -1, 1,
                1, 1, 1);
33          assertTrue(false);
34       } catch (RouteNotValidException e) {
35          assertEquals(e.getMessage(), "RouteEntry: invalid
                parameters given");
36       }
37       try {
38          new RouteRequestEntry(Constants.MAX_BROADCAST_ID +1, 1, 1,
                 1, 1);
39          assertTrue(false);
40       } catch (RouteNotValidException e) {
41          assertEquals(e.getMessage(), "RouteEntry: invalid
                parameters given");
42       }
43       //source address test
44       try {
45          new RouteRequestEntry(1, Constants.MIN_VALID_NODE_ADDRESS
                -1, 1, 1, 1);
46          assertTrue(false);
47       } catch (RouteNotValidException e) {
```

```
48            assertEquals(e.getMessage(), "RouteEntry: invalid
                  parameters given");
49        }
50        try {
51            new RouteRequestEntry(1, Constants.MAX_VALID_NODE_ADDRESS
                  +1, 1, 1, 1);
52            assertTrue(false);
53        } catch (RouteNotValidException e) {
54            assertEquals(e.getMessage(), "RouteEntry: invalid
                  parameters given");
55        }
56        //dest seq numb test
57        try {
58            new RouteRequestEntry(1, 1, Constants.
                  FIRST_SEQUENCE_NUMBER -1, 1, 1);
59            assertTrue(false);
60        } catch (RouteNotValidException e) {
61            assertEquals(e.getMessage(), "RouteEntry: invalid
                  parameters given");
62        }
63        try {
64            new RouteRequestEntry(1, 1, Constants.MAX_SEQUENCE_NUMBER
                  +1, 1, 1);
65            assertTrue(false);
66        } catch (RouteNotValidException e) {
67            assertEquals(e.getMessage(), "RouteEntry: invalid
                  parameters given");
68        }
69        //hop-count test
70        try {
71            new RouteRequestEntry(0,0,1,-1,0);
72            new RouteRequestEntry(0,0,1,0,0);
73            new RouteRequestEntry(0,0,1,1,0);
74            new RouteRequestEntry(0,0,1,Integer.MAX_VALUE,0);
75            new RouteRequestEntry(0,0,1,Integer.MIN_VALUE,0);
76            //the hop-count can be set to any integer
77            assertTrue(true);
78        } catch (RouteNotValidException e) {
79            assertTrue(false);
80        }
81        //dest address test
82        try {
83            new RouteRequestEntry(1, 1, 1, 1, Constants.
                  MIN_VALID_NODE_ADDRESS -1);
84            assertTrue(false);
85        } catch (RouteNotValidException e) {
86            assertEquals(e.getMessage(), "RouteEntry: invalid
                  parameters given");
87        }
88        try {
89            new RouteRequestEntry(1, 1, 1, 1, Constants.
                  MAX_VALID_NODE_ADDRESS +1);
90            assertTrue(false);
91        } catch (RouteNotValidException e) {
```

```
92          assertEquals(e.getMessage(), "RouteEntry: invalid
                parameters given");
93      }
94  }
95
96  @Test public void timeToLiveTest(){
97      //rreq entry: ensuring that TTL is a reasonable value
98      assertTrue(r1.getAliveTimeLeft()>0 && r1.getAliveTimeLeft()
            <= System.currentTimeMillis()+Constants.
            PATH_DESCOVERY_TIME);
99  }
100
101 @Test public void setBroadcastIdTest(){
102     assertFalse(r1.setBroadcastID(Constants.FIRST_BROADCAST_ID-1)
            );
103     assertFalse(r1.setBroadcastID(Constants.MAX_BROADCAST_ID+1));
104 }
105 }
```

D.2.2.3 ForwardTableTest

```
1  package adhoc.test;
2
3  import static org.junit.Assert.assertEquals;
4  import static org.junit.Assert.assertFalse;
5  import static org.junit.Assert.assertTrue;
6
7  import java.util.ArrayList;
8
9  import org.junit.After;
10 import org.junit.Before;
11 import org.junit.Test;
12
13 import adhoc.aodv.exception.AodvException;
14 import adhoc.aodv.exception.NoSuchRouteException;
15 import adhoc.aodv.exception.RouteNotValidException;
16 import adhoc.aodv.routes.ForwardRouteEntry;
17 import adhoc.aodv.routes.ForwardRouteTable;
18
19 public class ForwardTableTest {
20     ForwardRouteTable ft;
21     ForwardRouteEntry fe1, fe2, fe3, fe4, fe5;
22     ArrayList<Integer> precursors;
23
24     @Before
25     public void setUp() throws Exception {
26         ft = new ForwardRouteTable();
27
28         precursors = new ArrayList<Integer>();
29         precursors.add(4);
30
```

```
31          fe1 = new ForwardRouteEntry(0, 0, 1, 1, precursors);
32          fe2 = new ForwardRouteEntry(1, 0, 1, 1, precursors);
33
34          fe3 = new ForwardRouteEntry(0, 1, 1, 1, precursors);
35          fe4 = new ForwardRouteEntry(0, 1, 2, 1, precursors);
36          fe5 = new ForwardRouteEntry(0, 1, 2, 4, precursors);
37      }
38
39      @After
40      public void tearDown() throws Exception {
41          ft = null;
42          fe1 = null; fe2= null; fe3 = null; fe4= null; fe5 = null;
43      }
44
45      /**
46       * Testing an empty forward table
47       */
48      @Test public void isForwardTableEmpty(){
49          assertTrue(ft.isEmpty());
50          ft.addForwardRouteEntry(fe1);
51          assertFalse(ft.isEmpty());
52      }
53
54      /**
55       * Testing that the same entry is not accepted twice
56       */
57      @Test public void addSameEntry(){
58          assertTrue(ft.addForwardRouteEntry(fe1));
59          assertFalse(ft.addForwardRouteEntry(fe1));
60      }
61
62      /**
63       * Testing that a forward route is uniquely defined by the
             destination address parameter
64       */
65      @Test public void uniqueForwardEntryTest(){
66          assertTrue(ft.addForwardRouteEntry(fe1));
67          assertTrue(ft.addForwardRouteEntry(fe2));
68
69          assertFalse(ft.addForwardRouteEntry(fe3));
70          assertFalse(ft.addForwardRouteEntry(fe4));
71          assertFalse(ft.addForwardRouteEntry(fe5));
72      }
73
74      /**
75       * Testing getLastKnownDestSeqNumb method of the forward table
76       */
77      @Test public void LastKnownDestSeqNumbTest(){
78          try {
79              ft.getLastKnownDestSeqNumber(0);
80              assertTrue(false);
81          } catch (NoSuchRouteException e) {
82              assertTrue(true);
83          }
84          ft.addForwardRouteEntry(fe1);
```

```
85          try {
86              assertEquals(1, ft.getLastKnownDestSeqNumber(0));
87          } catch (NoSuchRouteException e) {
88              assertTrue(false);
89          }
90      }
91
92      /**
93       * Testing the getForwardRouteEntry() method of the forward
             table
94       */
95      @Test public void getForwardRouteEntryTest(){
96          try {
97              ft.getForwardRouteEntry(0);
98              assertTrue(false);
99          } catch (NoSuchRouteException e) {
100             assertTrue(true);
101         } catch (RouteNotValidException e) {
102             assertTrue(false);
103         }
104         ft.addForwardRouteEntry(fe1);
105         try {
106             assertEquals(ft.getForwardRouteEntry(0),fe1);
107         } catch (AodvException e) {
108             assertTrue(false);
109         }
110     }
111
112     /**
113      * Testing getPrecursors method of forward table
114      */
115     @Test public void getPrecursorsTest(){
116         assertEquals(true,ft.getPrecursors(0).isEmpty());
117         ft.addForwardRouteEntry(fe1);
118         assertEquals(false, ft.getPrecursors(0).isEmpty());
119         assertTrue(ft.getPrecursors(0).contains(4));
120     }
121
122     @Test public void getNextRouteToExpire(){
123         ft.addForwardRouteEntry(fe1);
124         try {
125             assertEquals(fe1,ft.getNextRouteToExpire());
126         } catch (NoSuchRouteException e) {
127             assertTrue(false);
128         }
129     }
130
131     @Test public void getNextRouteToExpire2(){
132         ft.addForwardRouteEntry(fe1);
133         fe1.setValid(false);
134         try {
135             assertEquals(fe1,ft.getNextRouteToExpire());
136         } catch (NoSuchRouteException e) {
137             assertTrue(false);
138         }
```

```
139     }
140
141     @Test public void getNextRouteToExpire3(){
142        ft.addForwardRouteEntry(fe1);
143        ft.addForwardRouteEntry(fe2);
144        try {
145           assertEquals(fe1,ft.getNextRouteToExpire());
146        } catch (NoSuchRouteException e) {
147           assertTrue(false);
148        }
149        ft.removeEntry(fe1.getDestinationAddress());
150        try {
151           assertEquals(fe2,ft.getNextRouteToExpire());
152        } catch (NoSuchRouteException e) {
153           assertTrue(false);
154        }
155     }
156  }
```

D.2.2.4 RouteRequestTableTest

```
1   package adhoc.test;
2
3   import static org.junit.Assert.assertEquals;
4   import static org.junit.Assert.assertFalse;
5   import static org.junit.Assert.assertTrue;
6
7   import org.junit.After;
8   import org.junit.Before;
9   import org.junit.Test;
10
11  import adhoc.aodv.exception.NoSuchRouteException;
12  import adhoc.aodv.routes.RouteRequestEntry;
13  import adhoc.aodv.routes.RouteRequestTable;
14
15  public class RouteRequestTableTest {
16     RouteRequestTable rt;
17     RouteRequestEntry re1, re2, re3, re4, re5, re6;
18
19     @Before public void setUp() throws Exception {
20        rt = new RouteRequestTable();
21
22        re1 = new RouteRequestEntry(0, 0, 1, 1, 0);
23        re2 = new RouteRequestEntry(1, 0, 1, 1, 0);
24        re3 = new RouteRequestEntry(0, 1, 1, 1, 0);
25        re4 = new RouteRequestEntry(0, 0, 1, 1, 1);
26
27        re5 = new RouteRequestEntry(0, 0, 2, 1, 1);
28        re6 = new RouteRequestEntry(0, 0, 1, 2, 1);
29     }
30
```

```
31      @After public void tearDown() throws Exception {
32          rt = null;
33          re1 = null; re2 = null; re3 = null; re4 = null; re5 = null;
                re6 = null;
34      }
35
36      /**
37       * Testing an empty request table
38       */
39      @Test public void isRouteRequestTableEmpty(){
40          assertTrue(rt.isEmpty());
41          rt.addRouteRequestEntry(re1, false);
42          assertTrue(rt.isEmpty());
43      }
44
45      /**
46       * Testing an empty request table
47       */
48      @Test public void isRouteRequestTableEmpty2(){
49          assertTrue(rt.isEmpty());
50          rt.addRouteRequestEntry(re1, true);
51          assertFalse(rt.isEmpty());
52      }
53
54      /**
55       * Testing that the same entry is not accepted twice
56       */
57      @Test public void addSameEntry(){
58          assertTrue(rt.addRouteRequestEntry(re1, true));
59          assertFalse(rt.addRouteRequestEntry(re1, true));
60      }
61
62      /**
63       * Testing combinations of the second boolean parameter "
               setTimer"
64       * to test if it has any impact adding the entry element twice
65       */
66      @Test public void addSameReqEntry2(){
67          assertTrue(rt.addRouteRequestEntry(re1, false));
68          assertFalse(rt.addRouteRequestEntry(re1, false));
69      }
70
71      /**
72       * Testing combinations of the second boolean parameter "
               setTimer"
73       * to test if it has any impact adding the entry element twice
74       */
75      @Test public void addSameReqEntry3(){
76          assertTrue(rt.addRouteRequestEntry(re1, true));
77          assertFalse(rt.addRouteRequestEntry(re1, false));
78      }
79
80      /**
81       * Testing combinations of the second boolean parameter "
               setTimer"
```

```
82      * to test if it has any impact adding the entry element twice
83      */
84     @Test public void addSameReqEntry4(){
85        assertTrue(rt.addRouteRequestEntry(re1, false));
86        assertFalse(rt.addRouteRequestEntry(re1, true));
87     }
88
89     /**
90      * Testing that a req entry is uniquely defined by the (
          broadcastID,sourceAddress) pair
91      */
92     @Test public void uniqueReqEntryTest(){
93        assertTrue(rt.addRouteRequestEntry(re1, true));
94        assertTrue(rt.addRouteRequestEntry(re2, true));
95        assertTrue(rt.addRouteRequestEntry(re3, true));
96
97        assertFalse(rt.addRouteRequestEntry(re4, true));
98        assertFalse(rt.addRouteRequestEntry(re5, true));
99        assertFalse(rt.addRouteRequestEntry(re6, true));
100    }
101
102    @Test public void routeRequestEntryExistsTest(){
103       assertFalse(rt.routeRequestEntryExists(re1.getSourceAddress()
              , re1.getBroadcastID()));
104       rt.addRouteRequestEntry(re1, true);
105       assertTrue(rt.routeRequestEntryExists(re1.getSourceAddress(),
               re1.getBroadcastID()));
106    }
107
108    @Test public void getRouteRequestEntryTest(){
109       try {
110          rt.getRouteRequestEntry(0, 0, false);
111          assertTrue(false);
112       } catch (NoSuchRouteException e) {
113          assertTrue(true);
114       }
115       try {
116          rt.getRouteRequestEntry(0, 0, true);
117          assertTrue(false);
118       } catch (NoSuchRouteException e) {
119          assertTrue(true);
120       }
121       try {
122          rt.getRouteRequestEntry(1, 0, true);
123          assertTrue(false);
124       } catch (NoSuchRouteException e) {
125          assertTrue(true);
126       }
127       try {
128          rt.getRouteRequestEntry(1, 0, false);
129          assertTrue(false);
130       } catch (NoSuchRouteException e) {
131          assertTrue(true);
132       }
133       rt.addRouteRequestEntry(re1, true);
```

```
134         try {
135             assertEquals(re1,rt.getRouteRequestEntry(re1.
                    getSourceAddress(), re1.getBroadcastID(), false));
136         } catch (NoSuchRouteException e) {
137             assertTrue(false);
138         }
139         try {
140             assertEquals(re1,rt.getRouteRequestEntry(re1.
                    getSourceAddress(), re1.getBroadcastID(), true));
141         } catch (NoSuchRouteException e) {
142             assertTrue(false);
143         }
144         try {
145             rt.getRouteRequestEntry(re1.getSourceAddress(), re1.
                    getBroadcastID(), false);
146             assertTrue(false);
147         } catch (NoSuchRouteException e) {
148             assertTrue(true);
149         }
150     }
151
152     @Test public void removeEntryTest(){
153         rt.addRouteRequestEntry(re1, true);
154         assertTrue(rt.removeEntry(re1.getSourceAddress(), re1.
                getBroadcastID()));
155     }
156
157     @Test public void removeEntryTest2(){
158         rt.addRouteRequestEntry(re1, false);
159         assertTrue(rt.removeEntry(re1.getSourceAddress(), re1.
                getBroadcastID()));
160     }
161
162     @Test public void getNextRouteToExpire(){
163         try {
164             rt.getNextRouteToExpire();
165             assertTrue(false);
166         } catch (NoSuchRouteException e) {
167             assertTrue(true);
168         }
169     }
170     @Test public void getNextRouteToExpire2(){
171         rt.addRouteRequestEntry(re1, false);
172         try {
173             rt.getNextRouteToExpire();
174             assertTrue(false);
175         } catch (NoSuchRouteException e) {
176             assertTrue(true);
177         }
178     }
179     @Test public void getNextRouteToExpire3(){
180         rt.addRouteRequestEntry(re1, true);
181         try {
182             assertEquals(re1,rt.getNextRouteToExpire());
183             assertTrue(true);
```

```
184         } catch (NoSuchRouteException e) {
185             assertTrue(false);
186         }
187     }
188     @Test public void getNextRouteToExpire4(){
189         rt.addRouteRequestEntry(re1, true);
190         rt.addRouteRequestEntry(re2, true);
191         try {
192             assertEquals(re1,rt.getNextRouteToExpire());
193         } catch (NoSuchRouteException e) {
194             assertTrue(false);
195         }
196         rt.removeEntry(re1.getSourceAddress(), re1.getBroadcastID());
197         rt.addRouteRequestEntry(re1, false);
198         try {
199             assertEquals(re2,rt.getNextRouteToExpire());
200         } catch (NoSuchRouteException e) {
201             assertTrue(false);
202         }
203     }
204 
205     @Test public void setRouteRequestTimerTest(){
206         assertTrue(rt.isEmpty());
207         try {
208             rt.setRouteRequestTimer(1, 1);
209             assertTrue(false);
210         } catch (NoSuchRouteException e) {
211             assertTrue(true);
212         }
213         rt.addRouteRequestEntry(re1, false);
214         assertTrue(rt.isEmpty());
215         try {
216             rt.setRouteRequestTimer(re1.getSourceAddress(), re1.
                    getDestinationAddress());
217             assertTrue(true);
218         } catch (NoSuchRouteException e) {
219             assertTrue(false);
220         }
221         assertFalse(rt.isEmpty());
222     }
223 }
```

D.3 Text Messenger Unit Tests

D.3.0.5 ChatTest

```
1 package android.TextMessenger.model.test;
2 
3 import java.util.HashMap;
```

```
4
5  import android.TextMessenger.model.Chat;
6  import android.TextMessenger.model.pdu.Msg;
7  import android.test.AndroidTestCase;
8
9  public class ChatTest extends AndroidTestCase{
10    Chat chat;
11
12    protected void setUp() throws Exception {
13       HashMap<Integer, String> contacts = new HashMap<Integer,
             String>();
14       contacts.put(33, "33");
15       int chatID = 7357;
16       int myContactID = 22;
17       String myDisplayName = "22";
18       chat = new Chat(contacts, chatID, myContactID, myDisplayName)
             ;
19    }
20
21    protected void tearDown() throws Exception {
22    }
23
24    public void testAddMsg(){
25
26       assertEquals(true,   chat.addMsg(new Msg(1, 33, 7357, "Hej
             der")));
27       assertEquals(false,  chat.addMsg(new Msg(3, 33, 7357, "Hej
             der")));
28       assertEquals(false,  chat.addMsg(new Msg(4, 33, 7357, "Hej
             der")));
29       assertEquals(false,  chat.addMsg(new Msg(5, 33, 7357, "Hej
             der")));
30       assertEquals(true,   chat.addMsg(new Msg(2, 33, 7357, "Hej
             der")));
31       assertEquals(false,  chat.addMsg(new Msg(2, 22, 7357, "Hej
             der")));
32       assertEquals(true,   chat.addMsg(new Msg(1, 22, 7357, "Hej
             der")));
33
34    }
35
36
37 }
```

D.3.0.6 ChatManagerTest

```
1  package android.TextMessenger.model.test;
2
3  import java.net.BindException;
4  import java.net.SocketException;
5  import java.net.UnknownHostException;
```

```
6  import java.util.HashMap;
7
8  import adhoc.aodv.Node;
9  import adhoc.aodv.exception.InvalidNodeAddressException;
10 import android.TextMessenger.model.ChatManager;
11 import android.TextMessenger.model.ContactManager;
12 import android.TextMessenger.model.Sender;
13 import android.TextMessenger.model.Timer;
14 import android.TextMessenger.view.Connect;
15 import android.test.AndroidTestCase;
16
17 public class ChatManagerTest extends AndroidTestCase{
18    ChatManager chatManager;
19    ContactManager contactManager;
20    Node node;
21    HashMap<Integer, String> contactIDs;
22    Connect c;
23
24    protected void setUp(){
25       try {
26          node = new Node(4);
27       } catch (BindException e) {
28          // TODO Auto-generated catch block
29          e.printStackTrace();
30       } catch (InvalidNodeAddressException e) {
31          // TODO Auto-generated catch block
32          e.printStackTrace();
33       } catch (SocketException e) {
34          // TODO Auto-generated catch block
35          e.printStackTrace();
36       } catch (UnknownHostException e) {
37          // TODO Auto-generated catch block
38          e.printStackTrace();
39       }
40       ChatManager chatManager = new ChatManager("lasse", 44, node);
41
42       contactManager.addContact(33, "33der", false);
43       contactManager.addContact(44, "44der", false);
44       contactManager.addContact(55, "55der", false);
45    }
46
47    protected void tearDown() throws Exception {
48    }
49
50    public void addAndRemoveChatTest(){
51
52       contactIDs = new HashMap<Integer, String>();
53       contactIDs.put(33, "33der");
54       contactIDs.put(44, "44der");
55       contactIDs.put(55, "55der");
56
57       assertEquals(true, chatManager.newChat(contactIDs));
58       assertEquals(false, chatManager.newChat(contactIDs));
59
60       contactIDs = new HashMap<Integer, String>();
```

```
61          contactIDs.put(33, "33der");
62          contactIDs.put(44, "44der");
63
64          assertEquals(true, chatManager.newChat(contactIDs));
65          assertEquals(false, chatManager.newChat(contactIDs));
66
67          contactIDs = new HashMap<Integer, String>();
68          assertEquals(true, chatManager.newChat(contactIDs));
69
70          assertEquals(false, chatManager.removeChatsWhereContactIsIn
                (233));
71          assertEquals(true, chatManager.removeChatsWhereContactIsIn
                (33));
72          assertEquals(false, chatManager.removeChatsWhereContactIsIn
                (55));
73
74      }
75
76  }
```

D.3.0.7 TimerTest

```
1   package android.TextMessenger.model.test;
2
3   import java.net.BindException;
4   import java.net.SocketException;
5   import java.net.UnknownHostException;
6
7   import adhoc.aodv.Node;
8   import adhoc.aodv.exception.InvalidNodeAddressException;
9   import android.TextMessenger.model.ChatManager;
10  import android.TextMessenger.model.ClassConstants;
11  import android.TextMessenger.model.Timer;
12  import android.TextMessenger.model.pdu.Ack;
13  import android.test.AndroidTestCase;
14
15  public class TimerTest  extends AndroidTestCase{
16      Node node;
17      Timer timer;
18
19      protected void setUp(){
20          try {
21              node = new Node(4);
22          } catch (BindException e) {
23              // TODO Auto-generated catch block
24              e.printStackTrace();
25          } catch (InvalidNodeAddressException e) {
26              // TODO Auto-generated catch block
27              e.printStackTrace();
28          } catch (SocketException e) {
29              // TODO Auto-generated catch block
```

```
30              e.printStackTrace();
31          } catch (UnknownHostException e) {
32              // TODO Auto-generated catch block
33              e.printStackTrace();
34          }
35          ChatManager chatManager = new ChatManager("lasse", 44, node);
36          timer = new Timer(node, "Lasse", 44, ClassConstants.
                getInstance().getContactManager(), chatManager);
37      }
38
39      protected void tearDown() throws Exception {
40      }
41
42      public void timerTEst(){
43
44          Ack ack = new Ack();
45          ack.setSequenceNumber(33);
46          assertEquals(true,timer.setTimer(ack, 22));
47          assertEquals(false,timer.setTimer(ack, 22));
48
49          ack.setSequenceNumber(44);
50          assertEquals(true,timer.setTimer(ack, 22));
51
52
53          assertEquals(true,timer.removePDU(44));
54          assertEquals(false,timer.removePDU(44));
55
56      }
57  }
```

D.3.0.8

```
1   package android.TextMessenger.model.test;
2
3   import java.net.BindException;
4   import java.net.SocketException;
5   import java.net.UnknownHostException;
6   import java.util.HashMap;
7
8   import adhoc.aodv.Node;
9   import adhoc.aodv.exception.InvalidNodeAddressException;
10  import android.TextMessenger.model.ChatManager;
11  import android.TextMessenger.model.ClassConstants;
12  import android.TextMessenger.model.ContactManager;
13  import android.TextMessenger.model.Timer;
14  import android.TextMessenger.view.Connect;
15  import android.test.AndroidTestCase;
16
17  public class ContactManagerTest extends AndroidTestCase{
18      ContactManager contactManager;
19      Node node;
```

```
20
21      protected void setUp(){
22          try {
23              node = new Node(4);
24          } catch (BindException e) {
25              // TODO Auto-generated catch block
26              e.printStackTrace();
27          } catch (InvalidNodeAddressException e) {
28              // TODO Auto-generated catch block
29              e.printStackTrace();
30          } catch (SocketException e) {
31              // TODO Auto-generated catch block
32              e.printStackTrace();
33          } catch (UnknownHostException e) {
34              // TODO Auto-generated catch block
35              e.printStackTrace();
36          }
37          ChatManager chatManager = new ChatManager("lasse", 44, node);
38          contactManager = ClassConstants.getInstance().
                getContactManager();
39      }
40
41      protected void tearDown() throws Exception {
42      }
43
44      public void addAndRemoveChatTest(){
45
46
47
48          assertEquals(true, contactManager.addContact(44, "Olga",
                false));
49          assertEquals(false, contactManager.addContact(44, "Olga",
                false));
50
51          assertEquals(true, contactManager.addContact(55, "Olga", true
                ));
52          assertEquals(false, contactManager.addContact(55, "Olga",
                false));
53
54          assertEquals(false, contactManager.removeContact(66));
55          assertEquals(true, contactManager.removeContact(44));
56          assertEquals(false, contactManager.removeContact(44));
57
58
59      }
60
61  }
```

Appendix E

Ad-Hoc Library Source Code

The following source code is structured such that each class can be seen depending of which package it is from.

E.1 Aodv

E.1.0.9 Node.java

```
1  package adhoc.aodv;
2
3  import java.net.BindException;
4  import java.net.SocketException;
5  import java.net.UnknownHostException;
6  import java.util.Observable;
7  import java.util.Queue;
8  import java.util.concurrent.ConcurrentLinkedQueue;
9
10 import adhoc.aodv.exception.InvalidNodeAddressException;
11 import adhoc.aodv.pdu.AodvPDU;
12 import adhoc.aodv.pdu.UserDataPacket;
13 import adhoc.etc.Debug;
14
15
```

```
16
17 /**
18  * <pre>Note - Any observers should implement their update methods
        in the following way:
19 public void update(Observable o, Object arg) {
20    MessageToObserver msg = (MessageToObserver)arg;
21    int userPacketID, destination, type = msg.getMessageType();
22
23    switch (type) {
24    case ObserverConst.ROUTE_ESTABLISHMENT_FAILURE:
25       //Note: any messages that had same destination has been
             removed from sending
26       int unreachableDestinationAddrerss  = (Integer)msg.
             getContainedData();
27       ...
28       break;
29    case ObserverConst.DATA_RECEIVED:
30       byte[] data = (byte[])msg.getContainedData();
31       int senderAddress = (Integer)((PacketToObserver)msg).
             getSenderNodeAddress();
32       ...
33       break;
34    case ObserverConst.DATA_SENT_SUCCESS:
35       userPacketID = (Integer)msg.getContainedData();
36       ...
37       break;
38    case ObserverConst.INVALID_DESTINATION_ADDRESS:
39       userPacketID = (Integer)msg.getContainedData();
40       ...
41       break;
42    case ObserverConst.DATA_SIZE_EXCEEDES_MAX:
43       userPacketID = (Integer)msg.getContainedData();
44       ...
45       break;
46    case ObserverConst.ROUTE_INVALID:
47       destination  = (Integer)msg.getContainedData();
48       ...
49       break;
50    case ObserverConst.ROUTE_CREATED:
51       destination = (Integer)msg.getContainedData();
52       ...
53       break;
54    default:
55       break;
56    }
57 }
58  * </pre>
59  * @author Rabie
60  *
61  */
62 public class Node extends Observable implements Runnable {
63    private int nodeAddress;
64    private int nodeSequenceNumber = Constants.FIRST_SEQUENCE_NUMBER
          ;
65    private int nodeBroadcastID = Constants.FIRST_BROADCAST_ID;
```

```
66      private Sender sender;
67      private Receiver receiver;
68      private RouteTableManager routeTableManager;
69      private Object sequenceNumberLock = 0;
70      private Thread notifierThread;
71      private Queue<MessageToObserver> messagesForObservers;
72      private volatile boolean keepRunning = true;
73
74     /**
75      * Creates an instance of the Node class
76      * @param nodeAddress
77      * @throws InvalidNodeAddressException Is thrown if the given
            node address is outside of the valid interval of node
            addresses
78      * @throws SocketException is cast if the node failed to
            instantiate port connections to the ad-hoc network
79      * @throws UnknownHostException
80      * @throws BindException this exception is thrown if network
            interface already is connected to a another network
81      */
82      public Node(int nodeAddress) throws InvalidNodeAddressException
            , SocketException, UnknownHostException, BindException {
83        if(nodeAddress > Constants.MAX_VALID_NODE_ADDRESS
84              || nodeAddress < Constants.MIN_VALID_NODE_ADDRESS){
85           //given address is out of the valid range
86           throw new InvalidNodeAddressException();
87        }
88        this.nodeAddress = nodeAddress;
89        routeTableManager = new RouteTableManager(nodeAddress, this);
90          sender = new Sender(this, nodeAddress, routeTableManager);
91          receiver = new Receiver(sender, nodeAddress, this,
                routeTableManager);
92        messagesForObservers = new ConcurrentLinkedQueue<
              MessageToObserver>();
93      }
94
95      /**
96       * Starts executing the AODV routing protocol
97       * @throws UnknownHostException
98       * @throws SocketException
99       * @throws BindException
100      */
101     public void startThread(){
102       keepRunning = true;
103       routeTableManager.startTimerThread();
104       sender.startThread();
105       receiver.startThread();
106       notifierThread = new Thread(this);
107       notifierThread.start();
108       Debug.print("Node: all library threads are running");
109     }
110
111     /**
112      * Stops the AODV protocol.
113      *
```

```
114         * Note: using this method tells the running threads to
                terminate.
115         * This means that it does not insure that any remaining
                userpackets is sent before termination.
116         * Such behavior can be achieved by monitoring the
                notifications by registering as an observer.
117         */
118        public void stopThread(){
119          keepRunning = false;
120          receiver.stopThread();
121          sender.stopThread();
122          routeTableManager.stopTimerThread();
123          notifierThread.interrupt();
124          Debug.print("Node: all library threads are stopped");
125        }
126
127        /**
128         * Method to be used by the application layer to send data to a
                single destination node or all neighboring nodes (
                broadcast).
129         * @param packetIdentifier is an ID that is associated for this
                packet. This is given from the application layer to
                identify which packet failed or succeed in sending
130         * @param destinationAddress the address of the destination
                node. Should be set to Constants.BROADCAST_ADDRESS if the
                data is to be broadcasted.
131         * @param data an array of bytes containing the desired data to
                send. Note that the size of the data may not exceed
                Constants.MAX_PACKAGE_SIZE
132         */
133        public void sendData(int packetIdentifier, int
               destinationAddress, byte[] data){
134          sender.queueUserMessageFromNode(new UserDataPacket(
                packetIdentifier,destinationAddress, data, nodeAddress));
135        }
136
137        /**
138         * Method for getting the current sequence number for this node
139         * @return an integer value of the current sequence number
140         */
141       protected int getCurrentSequenceNumber(){
142          return nodeSequenceNumber;
143       }
144
145       /**
146        * Increments the given number but does NOT set this number as
               the nodes sequence number
147        * @param number is the number which to increment
148        */
149       protected int getNextSequenceNumber(int number){
150          if((number >= Constants.MAX_SEQUENCE_NUMBER || number <
                Constants.FIRST_SEQUENCE_NUMBER)){
151             return Constants.FIRST_SEQUENCE_NUMBER;
152          } else {
153             return number++;
```

```
154         }
155     }
156
157
158     /**
159      * Increments and set the sequence number before returning the
             new value.
160      * @return returns the next sequence number
161      */
162     protected int getNextSequenceNumber(){
163         synchronized (sequenceNumberLock) {
164             if(nodeSequenceNumber == Constants.UNKNOWN_SEQUENCE_NUMBER
165                     || nodeSequenceNumber == Constants.
                         MAX_SEQUENCE_NUMBER    ){
166
167                 nodeSequenceNumber = Constants.FIRST_SEQUENCE_NUMBER;
168             }
169             else{
170                 nodeSequenceNumber++;
171             }
172             return nodeSequenceNumber;
173         }
174     }
175
176     /**
177      * Increments the broadcast ID
178      * @return returns the incremented broadcast ID
179      */
180     protected int getNextBroadcastID() {
181         synchronized (sequenceNumberLock) {
182             if(nodeBroadcastID == Constants.MAX_BROADCAST_ID){
183                 nodeBroadcastID = Constants.FIRST_BROADCAST_ID;
184             } else {
185                 nodeBroadcastID++;
186             }
187             return nodeBroadcastID;
188         }
189     }
190
191     /**
192      * Only used for debugging
193      * @return returns the current broadcast ID of this node
194      */
195     protected int getCurrentBroadcastID(){
196         return nodeBroadcastID;
197     }
198
199     /**
200      * Notifies the application layer about
201      * @param senderNodeAddess the source node which sent a message
202      * @param data the actual data which the application message
             contained
203      */
204     protected void notifyAboutDataReceived(int senderNodeAddess,
            byte[] data) {
```

```
205         messagesForObservers.add(new PacketToObserver(
                senderNodeAddess,data,ObserverConst.DATA_RECEIVED));
206         wakeNotifierThread();
207     }
208
209     /**
210      * Notifies the observer(s) about the route establishment
              failure for a destination
211      * @param nodeAddress is the unreachable destination
212      */
213     protected void notifyAboutRouteEstablishmentFailure(int
            faliedToReachAddress) {
214        messagesForObservers.add(new ValueToObserver(
               faliedToReachAddress, ObserverConst.
               ROUTE_ESTABLISHMENT_FAILURE));
215        wakeNotifierThread();
216     }
217
218     /**
219      * Notifies the observer(s) that a packet is sent successfully
              from this node.
220      * NOTE: This does not guarantee that the packet also is
              received at the destination node
221      * @param packetIdentifier the ID of a packet which the above
              layer can recognize
222      */
223     protected void notifyAboutDataSentSucces(int packetIdentifier){
224        messagesForObservers.add(new ValueToObserver(packetIdentifier
               , ObserverConst.DATA_SENT_SUCCESS));
225        wakeNotifierThread();
226     }
227
228     /**
229      * Notifies the observer(s) that an invalid destination address
              where detected for a user packet to be sent
230      * @param packetIdentifier an integer that identifies the user
              packet with bad destination address
231      */
232     protected void notifyAboutInvalidAddressGiven(int
            packetIdentifier){
233        messagesForObservers.add(new ValueToObserver(packetIdentifier
               , ObserverConst.INVALID_DESTINATION_ADDRESS));
234        wakeNotifierThread();
235     }
236
237     protected void notifyAboutSizeLimitExceeded(int packetIdentifier
            ){
238        messagesForObservers.add(new ValueToObserver(packetIdentifier
               , ObserverConst.DATA_SIZE_EXCEEDES_MAX));
239        wakeNotifierThread();
240     }
241
242     protected void notifyAboutRouteToDestIsInvalid(int
            destinationAddress){
```

```
243         messagesForObservers.add(new ValueToObserver(
                destinationAddress, ObserverConst.ROUTE_INVALID));
244         wakeNotifierThread();
245     }
246
247     protected void notifyAboutNewNodeReachable(int
             destinationAddress){
248         messagesForObservers.add(new ValueToObserver(
                destinationAddress, ObserverConst.ROUTE_CREATED));
249         wakeNotifierThread();
250     }
251
252     private void wakeNotifierThread(){
253         synchronized (messagesForObservers) {
254             messagesForObservers.notify();
255         }
256     }
257     /**
258      * This interface defines the a structure for an observer to
                retrieve a message from the observable
259      * @author rabie
260      *
261      */
262     public interface MessageToObserver{
263
264         /**
265          *
266          * @return returns the type of this message as a String
267          */
268         public int getMessageType();
269
270         /**
271          * This method is used to retrieve the data that the
                    observable wants to notify about
272          * @return returns the object that is contained
273          */
274         public Object getContainedData();
275
276     }
277
278     public class ValueToObserver implements MessageToObserver{
279         private Integer value;
280         private int type;
281
282         public ValueToObserver(int value, int msgType) {
283             this.value = new Integer(value);
284             type = msgType;
285         }
286         @Override
287         public Object getContainedData() {
288             return value;
289         }
290
291         @Override
292         public int getMessageType() {
```

```
293             return type;
294         }
295 
296     }
297 
298     /**
299      * This class presents a received package from another node, to
           the application layer
300      * @author Rabie
301      *
302      */
303     public class PacketToObserver implements MessageToObserver{
304         private byte[] data;
305         private int senderNodeAddress;
306         private int type;
307 
308         public PacketToObserver(int senderNodeAddress, byte[] data,
               int msgType) {
309             type = msgType;
310             this.data = data;
311             this.senderNodeAddress = senderNodeAddress;
312         }
313 
314         /**
315          * A method to retrieve the senders address of this data
316          * @return returns an integer value representing the unique
               address of the sending node
317          */
318         public int getSenderNodeAddress(){
319             return senderNodeAddress;
320         }
321 
322         /**
323          * A method to retrieve the data sent
324          * @return returns a byte array containing the data which
325          * where sent by another node with this node as destination
326          */
327         @Override
328         public Object getContainedData() {
329             return data;
330         }
331 
332         @Override
333         public int getMessageType() {
334             return type;
335         }
336     }
337 
338     protected void queuePDUmessage(AodvPDU pdu) {
339         sender.queuePDUmessage(pdu);
340     }
341 
342     @Override
343     public void run() {
344         while(keepRunning){
```

```
345             try{
346                 synchronized (messagesForObservers) {
347                     while(messagesForObservers.isEmpty()){
348                         messagesForObservers.wait();
349                     }
350                 }
351                 setChanged();
352                 notifyObservers(messagesForObservers.poll());
353             }catch (InterruptedException e) {
354                 // thread stopped
355             }
356         }
357     }
358 }
```

E.1.0.10 RouteTableManager.java

```
1  package adhoc.aodv;
2
3  import java.util.ArrayList;
4
5  import adhoc.aodv.exception.NoSuchRouteException;
6  import adhoc.aodv.exception.RouteNotValidException;
7  import adhoc.aodv.pdu.InternalMessage;
8  import adhoc.aodv.pdu.RERR;
9  import adhoc.aodv.pdu.RREQ;
10 import adhoc.aodv.routes.ForwardRouteEntry;
11 import adhoc.aodv.routes.ForwardRouteTable;
12 import adhoc.aodv.routes.RouteEntry;
13 import adhoc.aodv.routes.RouteRequestEntry;
14 import adhoc.aodv.routes.RouteRequestTable;
15
16 public class RouteTableManager {
17
18     private volatile boolean keepRunning = true;
19     private ForwardRouteTable forwardRouteTable;
20     private RouteRequestTable routeRequestTable;
21     private final Object tableLocks = new Integer(0);
22     private TimeoutNotifier timeoutNotifier;
23     private int nodeAddress;
24     private Node parent;
25
26     public RouteTableManager(int nodeAddress, Node parent) {
27         this.nodeAddress = nodeAddress;
28         this.parent = parent;
29         forwardRouteTable = new ForwardRouteTable();
30         routeRequestTable = new RouteRequestTable();
31         timeoutNotifier = new TimeoutNotifier();
32     }
33
34     public void startTimerThread(){
```

```
35          keepRunning = true;
36          timeoutNotifier = new TimeoutNotifier();
37          timeoutNotifier.start();
38      }
39
40      public void stopTimerThread() {
41          keepRunning = false;
42          timeoutNotifier.stopThread();
43      }
44
45
46      /**
47       * Creates an entry and adds it to the appropriate table
48       * @param rreq The RREQ entry to be added
49       * @param setTimer is set to false if the timer should not start
                 count down the entry's time
50       * @return returns true if the route were created and added
                successfully.
51       */
52      protected boolean createRouteRequestEntry(RREQ rreq, boolean
             setTimer) {
53          RouteRequestEntry entry;
54          try {
55              entry = new RouteRequestEntry(   rreq.getBroadcastId(),
56                                       rreq.getSourceAddress(),
57                                       rreq.getDestinationSequenceNumber
                                             (),
58                                       rreq.getHopCount(),
59                                       rreq.getDestinationAddress());
60          } catch (RouteNotValidException e) {
61              return false;
62          }
63
64          if (routeRequestTable.addRouteRequestEntry(entry, setTimer))
                 {
65              if(setTimer){
66                  //notify the timer since the RREQ table (the sorted
                         list) isn't empty at this point
67                  synchronized (tableLocks) {
68                      tableLocks.notify();
69                  }
70              }
71              return true;
72          }
73          return false;
74      }
75
76      /**
77       * Creates an entry and adds it to the appropriate table
78       * @param destinationNodeAddress the destination address which
                this node will have a route for
79       * @param nextHopAddress is the neighbor address which to
                forward to if the destination should be reached
80       * @param destinationSequenceNumber is the sequence number of
                the destination
```

```
81      * @param hopCount the number of intermediate node which will
            participate to forward a possible package for the
            destination
82      * @return returns true if the route were created and added
            successfully.
83      */
84     protected boolean createForwardRouteEntry(int
           destinationNodeAddress, int nextHopAddress, int
           destinationSequenceNumber,
85             int hopCount, boolean notifyObserver) {
86        return createForwardRouteEntry(destinationNodeAddress,
              nextHopAddress, destinationSequenceNumber, hopCount, new
              ArrayList<Integer>(), notifyObserver);
87     }
88
89     /**
90      * Creates an entry and adds it to the appropriate table
91      * @param destinationNodeAddress the destination address which
            this node will have a route for
92      * @param nextHopAddress is the neighbor address which to
            forward to if the destination should be reached
93      * @param destinationSequenceNumber is the sequence number of
            the destination
94      * @param hopCount the number of intermediate node which will
            participate to forward a possible package for the
            destination
95      * @param precursorNodes a list of node addresses which has used
            this route to forward packages
96      * @return returns true if the route were created and added
            successfully.
97      */
98     protected boolean createForwardRouteEntry(int
           destinationNodeAddress, int nextHopAddress,
99                     int destinationSequenceNumber, int hopCount,
                           ArrayList<Integer> precursorNodes, boolean
                           notifyObserver) {
100       ForwardRouteEntry forwardRouteEntry;
101       try {
102          forwardRouteEntry = new ForwardRouteEntry(
                 destinationNodeAddress,
103                                             nextHopAddress,
104                                             hopCount,
105                                             destinationSequenceNumber
                                                   ,
106                                             precursorNodes );
107       } catch (RouteNotValidException e) {
108          return false;
109       }
110       if (forwardRouteTable.addForwardRouteEntry(forwardRouteEntry)
              ) {
111          synchronized (tableLocks) {
112             tableLocks.notify();
113          }
114          if(notifyObserver){
```

```
115             parent.notifyAboutNewNodeReachable(
                    destinationNodeAddress);
116         }
117         parent.queuePDUmessage(new InternalMessage(  Constants.
                FORWARD_ROUTE_CREATED,
118                                                 destinationNodeAddress) )
                                                    ;
119         return true;
120       }
121       return false;
122     }
123
124     protected boolean routeRequestExists(int sourceAddress, int
            broadcastID) {
125       return routeRequestTable.routeRequestEntryExists(
              sourceAddress, broadcastID);
126     }
127
128     /**
129      * method used to check the forward route table if a valid entry
              exist with a freshness that is as least as required
130      * @param destinationAddress the destination address of the node
              which a route is will be looked at
131      * @param destinationSequenceNumber specify any freshness
            requirement
132      * @return returns true if such a valid forward route exist with
              the seq number or higher
133      */
134     protected boolean validForwardRouteExists(int destinationAddress
            , int destinationSequenceNumber) {
135       RouteEntry forwardRoute;
136       try {
137         forwardRoute = (ForwardRouteEntry) forwardRouteTable.
                getForwardRouteEntry(destinationAddress);
138       } catch (NoSuchRouteException e) {
139         return false;
140       } catch (RouteNotValidException e) {
141         return false;
142       }
143
144       if (forwardRoute.getDestinationSequenceNumber() >=
              destinationSequenceNumber) {
145         return true;
146       }
147       return false;
148     }
149
150     /**
151      *
152      * @param sourceAddress
153      * @param broadcastID
154      * @param removeEntry
155      * @return returns a RouteRequestEntry if any where found
156      * @throws NoSuchRouteException a NoSuchRouteException is cast
            in the event of an unsuccessful search
```

```
157      */
158     protected RouteRequestEntry getRouteRequestEntry(int
            sourceAddress, int broadcastID, boolean removeEntry)
159           throws NoSuchRouteException {
160
161        return (RouteRequestEntry) routeRequestTable.
               getRouteRequestEntry(sourceAddress, broadcastID,
               removeEntry);
162     }
163
164     protected ForwardRouteEntry getForwardRouteEntry(int
            destinationAddress) throws NoSuchRouteException,
            RouteNotValidException {
165        return forwardRouteTable.getForwardRouteEntry(
               destinationAddress);
166     }
167
168     protected void updateForwardRouteEntry(ForwardRouteEntry
            oldEntry, ForwardRouteEntry newEntry) throws
            NoSuchRouteException{
169        if (Receiver.isIncomingRouteInfoBetter(   newEntry.
               getDestinationSequenceNumber(), oldEntry.
               getDestinationSequenceNumber(),
170                                    newEntry.getHopCount(),
                                           oldEntry.getHopCount())) {
171           if(forwardRouteTable.updateForwardRouteEntry(newEntry)){
172              synchronized (tableLocks) {
173                 tableLocks.notify();
174              }
175           }
176        }
177     }
178
179     protected boolean RemoveForwardRouteEntry(int destinationAddress
            ) {
180        return forwardRouteTable.removeEntry(destinationAddress);
181     }
182
183     protected int getLastKnownDestSeqNum(int destinationAddress)
            throws NoSuchRouteException {
184        return forwardRouteTable.getLastKnownDestSeqNumber(
               destinationAddress);
185     }
186
187     protected ArrayList<Integer> getPrecursors(int
            destinaitonAdrress){
188        return forwardRouteTable.getPrecursors(destinaitonAdrress);
189     }
190
191     /**
192      * Makes a forward route valid, updates it sequence number if
                necessary and resets the AliveTimeLeft
193      * @param destinationAddress used to determine which forward
                route to set valid
```

```
194      * @param newDestinationSeqNumber this destSeqNum is only set in
             the entry if it is greater that the existing destSeqNum
195      * @throws NoSuchRouteException thrown if no table information
             is known about the destination
196      */
197     protected void setValid(int destinationAddress, int
             newDestinationSeqNumber) throws NoSuchRouteException {
198         forwardRouteTable.setValid(destinationAddress,
                newDestinationSeqNumber,true);
199     }
200
201     protected void setInvalid(int destinationAddress, int
             newDestinationSeqNumber) throws NoSuchRouteException {
202         forwardRouteTable.setValid(destinationAddress,
                newDestinationSeqNumber,false);
203     }
204
205     /**
206      *
207      * @param destinationAddress
208      * @return
209      * @throws NoSuchRouteException thrown if no table information
             is known about the destination
210      * @throws RouteNotValidException thrown if a route were found,
             but is marked as invalid
211      */
212     protected int getHopCount(int destinationAddress) throws
             NoSuchRouteException, RouteNotValidException {
213         return ((ForwardRouteEntry) forwardRouteTable.
                getForwardRouteEntry(destinationAddress)).getHopCount();
214     }
215
216     /**
217      * resets the time left to live of the RREQ entry
218      *
219      * @param sourceAddress
220      * @param brodcastID
221      * @throws NoSuchRouteException thrown if no table information
             is known about the destination
222      */
223     protected void setRouteRequestTimer(int sourceAddress, int
             broadcastID) throws NoSuchRouteException {
224         routeRequestTable.setRouteRequestTimer(sourceAddress,
                broadcastID);
225         //wake the timer thread since a RREQ should be monitored
226         synchronized (tableLocks) {
227             tableLocks.notify();
228         }
229     }
230
231     private class TimeoutNotifier extends Thread {
232         public TimeoutNotifier() {
233             super("TimeoutNotifier");
234         }
235
```

```
236         public void run() {
237             while (keepRunning) {
238                 try {
239                     synchronized (tableLocks) {
240                         while (routeRequestTable.isEmpty() &&
                                forwardRouteTable.isEmpty()) {
241                             tableLocks.wait();
242                         }
243                     }
244                     long time = getMinimumTime() - System.
                            currentTimeMillis();
245                     //Debug.print("Timer is sleeping for: "+time+"
                            millSec");
246                     if (time > 0) {
247                         sleep(time);
248                     }
249
250                     try {
251                         // Route Request clean up
252                         RouteRequestEntry route = (RouteRequestEntry)
                                routeRequestTable.getNextRouteToExpire();
253                         while (route.getAliveTimeLeft() <= System.
                                currentTimeMillis()) {
254                             routeRequestTable.removeEntry(route.
                                    getSourceAddress(), route.getBroadcastID()
                                    );
255                             //Debug.print(route.toString());
256                             if (route.getSourceAddress() == nodeAddress) {
257                                 if (!validForwardRouteExists(route.
                                        getDestinationAddress(), route.
                                        getDestinationSequenceNumber())) {
258                                     if (route.resend()) {
259                                         //create a new RREQ message to
                                                broadcast
260                                         RREQ newReq = new RREQ(nodeAddress,
261                                                 route.
                                                     getDestinationAddress
                                                     (),
262                                                 parent.
                                                     getCurrentSequenceNumber
                                                     (),
263                                                 route.
                                                     getDestinationSequenceNumber
                                                     (),
264                                                 parent.
                                                     getNextBroadcastID
                                                     ());
265                                         //update the RREQ entry
266                                         route.setBroadcastID(newReq.
                                                getBroadcastId());
267                                         //reinsert the entry with no timer
268                                         routeRequestTable.
                                                addRouteRequestEntry(route, false
                                                );
269                                         //let the sender broadcast the RREQ
```

```
270                             parent.queuePDUmessage(newReq);
271                         } else {
272                             // all RREQ retires is used. Notify
                                    the application layer
273                             parent.queuePDUmessage(new
                                    InternalMessage(Constants.
                                    RREQ_FAILURE_PDU, route.
                                    getDestinationAddress()));
274                             parent.
                                    notifyAboutRouteEstablishmentFailure
                                    (route.getDestinationAddress());
275                         }
276                     }
277                 }
278                 route = (RouteRequestEntry) routeRequestTable.
                        getNextRouteToExpire();
279             }
280         } catch (NoSuchRouteException e) {
281             // route request table is empty
282         }
283
284         // Forward Route Cleanup
285         ForwardRouteEntry froute;
286         try {
287             froute = (ForwardRouteEntry) forwardRouteTable.
                    getNextRouteToExpire();
288
289             while (froute.getAliveTimeLeft() <= System.
                    currentTimeMillis()) {
290
291                 //is froute a neighbour?
292                 if (froute.getHopCount() == 1 && froute.
                        isValid()) {
293                     forwardRouteTable.toString();
294                     setInvalid(froute.getDestinationAddress(),
                            froute.getDestinationSequenceNumber());
295                     parent.notifyAboutRouteToDestIsInvalid(
                            froute.getDestinationAddress());
296
297                     for(RERR rerr :forwardRouteTable.
                            findBrokenRoutes(froute.
                            getDestinationAddress())){
298                         parent.queuePDUmessage(rerr);
299                     }
300                 }
301                 else if (froute.isValid()) {
302                     forwardRouteTable.setValid(froute.
                            getDestinationAddress(), froute.
                            getDestinationSequenceNumber(), false);
303                     parent.notifyAboutRouteToDestIsInvalid(
                            froute.getDestinationAddress());
304                 }
305                 else {
306                     forwardRouteTable.removeEntry(froute.
                            getDestinationAddress());
```

```
307                         }
308                         froute = (ForwardRouteEntry) forwardRouteTable
                                .getNextRouteToExpire();
309                     }
310                 } catch (NoSuchRouteException e1) {
311                     // ForwardRoute table is empty
312                 }
313             } catch (InterruptedException e) {
314
315             }
316         }
317     }
318
319     private long getMinimumTime() {
320         long a = Long.MAX_VALUE, b = Long.MAX_VALUE;
321         try {
322             a = routeRequestTable.getNextRouteToExpire().
                    getAliveTimeLeft();
323         } catch (NoSuchRouteException e) {
324
325         }
326         try {
327             b = forwardRouteTable.getNextRouteToExpire().
                    getAliveTimeLeft();
328         } catch (NoSuchRouteException e2) {
329             if(a == Long.MAX_VALUE){
330                 return -1;
331             }
332             return a;
333         }
334         return (a < b ? a : b);
335     }
336
337     public void stopThread() {
338         this.interrupt();
339     }
340 }
341 }
```

E.1.0.11 Sender.java

```
1  package adhoc.aodv;
2
3  import java.io.IOException;
4  import java.net.BindException;
5  import java.net.SocketException;
6  import java.net.UnknownHostException;
7  import java.util.Queue;
8  import java.util.concurrent.ConcurrentLinkedQueue;
9
10 import javax.naming.SizeLimitExceededException;
```

```
11
12 import adhoc.aodv.exception.AodvException;
13 import adhoc.aodv.exception.DataExceedsMaxSizeException;
14 import adhoc.aodv.exception.InvalidNodeAddressException;
15 import adhoc.aodv.exception.NoSuchRouteException;
16 import adhoc.aodv.pdu.AodvPDU;
17 import adhoc.aodv.pdu.HelloPacket;
18 import adhoc.aodv.pdu.Packet;
19 import adhoc.aodv.pdu.RERR;
20 import adhoc.aodv.pdu.RREQ;
21 import adhoc.aodv.pdu.UserDataPacket;
22 import adhoc.etc.Debug;
23 import adhoc.udp.UdpSender;
24
25 public class Sender implements Runnable{
26    private Node parent;
27     private int nodeAddress;
28     private NeighbourBroadcaster neighborBroadcaster;
29     private Queue<Packet> pduMessages;
30     private Queue<UserDataPacket> userMessagesToForward;
31     private Queue<UserDataPacket> userMessagesFromNode;
32     private final Object queueLock = new Integer(0);
33     private RouteTableManager routeTableManager;
34     private UdpSender udpSender;
35     private boolean isRREQsent = false;
36     private volatile boolean keepRunning = true;
37     private Thread senderThread;
38
39     public Sender(Node parent,int nodeAddress, RouteTableManager
           routeTableManager) throws SocketException,
           UnknownHostException, BindException {
40       this.parent = parent;
41         this.nodeAddress = nodeAddress;
42         neighborBroadcaster = new NeighbourBroadcaster();
43       udpSender = new UdpSender();
44         pduMessages = new ConcurrentLinkedQueue<Packet>();
45         userMessagesToForward = new ConcurrentLinkedQueue<
               UserDataPacket>();
46         userMessagesFromNode = new ConcurrentLinkedQueue<
               UserDataPacket>();
47         this.routeTableManager = routeTableManager;
48
49     }
50
51     public void startThread(){
52       keepRunning = true;
53       neighborBroadcaster = new NeighbourBroadcaster();
54       neighborBroadcaster.start();
55       senderThread = new Thread(this);
56       senderThread.start();
57     }
58
59     public void stopThread(){
60       keepRunning = false;
61       neighborBroadcaster.stopBroadcastThread();
```

```
62          senderThread.interrupt();
63      }
64
65      public void run(){
66          while(keepRunning){
67              try {
68                  synchronized(queueLock){
69                      while(pduMessages.isEmpty() && userMessagesToForward
                            .isEmpty() && (isRREQsent ||
                            userMessagesFromNode.isEmpty())){
70                          queueLock.wait();
71                      }
72                  }
73
74                  //Handle user data messages that is to be sent from
                        this node
75                  if(!isRREQsent){
76                      if(!userMessagesFromNode.isEmpty()){
77                          UserDataPacket userData = userMessagesFromNode.
                                peek();
78                          while(userData != null){
79                              try{
80                                  if(!sendUserDataPacket(userData)){
81                                      isRREQsent = true;
82                                      //do not process any user messages
                                            before the head is sent
83                                      break;
84                                  } else {
85                                      parent.notifyAboutDataSentSucces(
                                            userData.getPacketID());
86                                  }
87                              } catch (DataExceedsMaxSizeException e) {
88                                  parent.notifyAboutSizeLimitExceeded(
                                        userData.getPacketID());
89                              } catch (InvalidNodeAddressException e) {
90                                  parent.notifyAboutInvalidAddressGiven(
                                        userData.getPacketID());
91                              }
92                              //it is expected that the queue still has the
                                    same userDataHeader object as head
93                              userMessagesFromNode.poll();
94                              userData = userMessagesFromNode.peek();
95                          }
96                      }
97                  }
98
99                  //Handles messages user data messages (received by
                        other nodes) that are to be forwarded
100                 UserDataPacket userData = userMessagesToForward.peek();
101                 while(userData != null){
102                     try{
103                         if(!sendUserDataPacket(userData)){
104                             //do not process any user messages before the
                                    head is sent
105                             break;
```

```
106                 }
107              } catch (InvalidNodeAddressException e) {
108                 Debug.print(e.getStackTrace().toString());
109              } catch (DataExceedsMaxSizeException e) {
110                 Debug.print(e.getStackTrace().toString());
111              }
112              //it is expected that the queue still has the same
                     userDataHeader object as head
113              userMessagesToForward.poll();
114              userData = userMessagesToForward.peek();
115           }
116
117           // Handle protocol messages
118           Packet packet = pduMessages.poll();
119           while(packet != null){
120              if(packet instanceof AodvPDU){
121                 AodvPDU pdu = (AodvPDU)packet;
122                 try {
123                    handleAodvPDU(pdu);
124                 } catch (InvalidNodeAddressException e) {
125                    Debug.print(e.getMessage());
126                 } catch (DataExceedsMaxSizeException e) {
127                    Debug.print("FATAL ERROR: Aodv packet could
                           not be sent because data size exceeded
                           limit");
128                 }
129              } else if(packet instanceof HelloPacket){
130                 try {
131                    broadcastPacket(packet);
132                    Debug.print("Sender: broadcasting hello
                           message");
133                 } catch (DataExceedsMaxSizeException e) {
134                    Debug.print(e.getStackTrace().toString());
135                 }
136              } else {
137                 Debug.print("Sender queue contained an unknown
                        message Packet PDU!");
138              }
139              packet = pduMessages.poll();
140           }
141        } catch (InterruptedException e) {
142
143        }
144     }
145  }
146
147
148  private void handleAodvPDU(AodvPDU pdu) throws
         InvalidNodeAddressException, DataExceedsMaxSizeException{
149     switch (pdu.getType()) {
150        case Constants.RREQ_PDU:
151           broadcastPacket(pdu);
152           if(pdu.getSourceAddress() == nodeAddress){
153              try {
```

```
154                     routeTableManager.setRouteRequestTimer(    ((RREQ)
                            pdu).getSourceAddress(),
155                                                    ((RREQ)pdu).
                                                           getBroadcastId
                                                           ()   );
156                 } catch (NoSuchRouteException e) {
157                     Debug.print(e.getStackTrace().toString());
158                 }
159             }
160             break;
161
162         case Constants.RREP_PDU:
163             if(!sendAodvPacket(pdu,pdu.getSourceAddress())){
164                 Debug.print("Sender: Did not have a forward route
                        for sending back the RREP message to: "+pdu.
                        getSourceAddress()+" the requested destination
                        is: "+pdu.getDestinationAddress());
165             }
166             break;
167
168         case Constants.RERR_PDU:
169             RERR rerr = (RERR)pdu;
170             for(int nodeAddress: rerr.getAllDestAddresses()){
171                 if(!sendAodvPacket(new RERR(  rerr.
                        getUnreachableNodeAddress(),
172                                 rerr.
                                     getUnreachableNodeSequenceNumber
                                     (),
173                                 nodeAddress ), nodeAddress)){
174                     Debug.print("Sender: Did not have a forward route
                            for sending the RERR message!!");
175                 }
176             }
177             break;
178
179         case Constants.RREQ_FAILURE_PDU:
180             cleanUserDataPacketsFromNode(pdu.getDestinationAddress
                    ());
181             isRREQsent = false;
182             synchronized (queueLock) {
183                 queueLock.notify();
184             }
185             break;
186
187         case Constants.FORWARD_ROUTE_CREATED:
188             UserDataPacket userPacket = userMessagesFromNode.peek()
                    ;
189             if(userPacket != null && pdu.getDestinationAddress() ==
                      userPacket.getDestinationAddress()){
190                 isRREQsent = false;
191                 synchronized (queueLock) {
192                     queueLock.notify();
193                 }
194             }
195             break;
```

```
196
197             default:
198                 Debug.print("Sender queue contained an unknown message
                        AODV PDU!");
199                 break;
200         }
201     }
202     /**
203      *
204      * @param packet is the message which are to be broadcasted to
                the neighboring nodes
205      * @throws SizeLimitExceededException
206      */
207     private boolean broadcastPacket(Packet packet) throws
            DataExceedsMaxSizeException {
208         try {
209             return udpSender.sendPacket(Constants.BROADCAST_ADDRESS
                    , packet.toBytes());
210         } catch (IOException e) {
211             Debug.print(e.getStackTrace().toString());
212             return false;
213         }
214     }
215
216
217     private boolean sendUserDataPacket(UserDataPacket packet) throws
            DataExceedsMaxSizeException, InvalidNodeAddressException{
218         if(      packet.getDestinationAddress() != Constants.
                BROADCAST_ADDRESS
219             && packet.getDestinationAddress() >= Constants.
                    MIN_VALID_NODE_ADDRESS
220             && packet.getDestinationAddress() <= Constants.
                    MAX_VALID_NODE_ADDRESS){
221             if(packet.getDestinationAddress() == nodeAddress){
222                 throw new InvalidNodeAddressException("Sender: It is
                        not allowed to send to our own address: "+
                        nodeAddress);
223             }
224             try {
225                 int nextHop = routeTableManager.getForwardRouteEntry(
                        packet.getDestinationAddress()).getNextHop();
226                 try {
227                     return udpSender.sendPacket(nextHop, packet.toBytes
                            ());
228                 } catch (IOException e) {
229                     Debug.print(e.getStackTrace().toString());
230                     return false;
231                 }
232             }catch (DataExceedsMaxSizeException e){
233                 throw new DataExceedsMaxSizeException();
234             } catch (AodvException e) {
235                 //Discover the route to the desired destination
236                 //if a route to the destination isn't request before
237                 try {
```

```
238                 int lastKnownDestSeqNum = routeTableManager.
                        getLastKnownDestSeqNum(packet.
                        getDestinationAddress());
239                 if(packet.getSourceNodeAddress() == nodeAddress){
240                    //Discover the route to the desired destination
241                    //if a route to the destination isn't request
                           before
242                    if(!createNewRREQ(packet.getDestinationAddress(),
                            lastKnownDestSeqNum, false)){
243                       Debug.print("Sender: Failed to add new RREQ
                              entry to the request table. Src: "+
                              nodeAddress+" broadID: "+parent.
                              getCurrentBroadcastID());
244                       return false;
245                    }
246                 } else {
247                    queuePDUmessage(new RERR(  packet.
                           getDestinationAddress(),
248                                               lastKnownDestSeqNum,
249                                               packet.getSourceNodeAddress
                                                   () )  );
250                    cleanUserDataPacketsToForward(packet.
                           getDestinationAddress());
251                 }
252              } catch (NoSuchRouteException e1) {
253                 if(packet.getSourceNodeAddress() == nodeAddress){
254                    //Discover the route to the desired destination
255                    //if a route to the destination isn't request
                           before
256                    createNewRREQ(packet.getDestinationAddress(),
                           Constants.UNKNOWN_SEQUENCE_NUMBER, false);
257                 } else {
258                    queuePDUmessage(new RERR(  packet.
                           getDestinationAddress(),
259                                               Constants.
                                                   UNKNOWN_SEQUENCE_NUMBER,
260                                               packet.getSourceNodeAddress
                                                   () )  );
261                    cleanUserDataPacketsToForward(packet.
                           getDestinationAddress());
262                 }
263              }
264              return false;
265           }
266        } else if( packet.getDestinationAddress() == Constants.
              BROADCAST_ADDRESS){
267           return broadcastPacket(packet);
268        } else {
269           throw new InvalidNodeAddressException("Sender: got
                 request to send a user packet which had  an invalid
                 node address: "+packet.getDestinationAddress());
270        }
271     }
272
273     /**
```

```
274     * Note: this method is able to send messages to itself if
            necessary. Note: DO NOT USE FOR BROADCASTING
275     * @param destinationNodeAddress should not be exchanged as the
            'nextHopAddress'. DestinationNodeAddress is the final place
            for this packet to reach
276     * @param packet is the message to be sent
277     * @return false if no route to the desired destination is
            currently known.
278     * @throws InvalidNodeAddressException
279     * @throws SizeLimitExceededException
280     */
281    private boolean sendAodvPacket(AodvPDU packet, int
           destinationNodeAddress) throws InvalidNodeAddressException{
282      if(destinationNodeAddress >= Constants.MIN_VALID_NODE_ADDRESS
283            && destinationNodeAddress <= Constants.
                 MAX_VALID_NODE_ADDRESS){
284        try {
285          int nextHop = routeTableManager.getForwardRouteEntry(
                 destinationNodeAddress).getNextHop();
286          return udpSender.sendPacket(nextHop, packet.toBytes
                 ());
287        } catch (IOException e) {
288          Debug.print("Sender: IOExeption when trying to send a
                 packet to: "+destinationNodeAddress);
289          return false;
290        } catch (AodvException e){
291          return false;
292        }
293      } else {
294        throw new InvalidNodeAddressException("Sender: Tried to
               send an AODV packet but the destination address is out
               valid range");
295      }
296    }
297
298    /**
299     *
300     * Creates and queues a new RREQ
301     * @param destinationNodeAddress is the destination that you
           want to discover a route to
302     * @param lastKnownDestSeqNum
303     * @param setTimer is set to false if the timer should not
           start count down the entry's time
304     * @return returns true if the route were created and added
           successfully.
305     */
306    private boolean createNewRREQ(int destinationNodeAddress, int
           lastKnownDestSeqNum, boolean setTimer){
307      RREQ rreq = new RREQ(   nodeAddress,
308            destinationNodeAddress,
309            parent.getNextSequenceNumber(),
310            lastKnownDestSeqNum,
311            parent.getNextBroadcastID()    );
312      if(routeTableManager.createRouteRequestEntry(rreq,setTimer)){
313        queuePDUmessage(rreq);
```

```
314             return true;
315           }
316           return false;
317         }
318
319         /**
320          * Method for queuing protocol messages for sending
321          * @param aodvPDU is the Protocol Data Unit to be queued.
322          */
323         protected void queuePDUmessage(AodvPDU aodvPDU){
324           pduMessages.add(aodvPDU);
325           synchronized (queueLock) {
326             queueLock.notify();
327           }
328         }
329
330         private void queueHelloPacket(Packet helloPacket){
331           pduMessages.add(helloPacket);
332           synchronized (queueLock) {
333             queueLock.notify();
334           }
335         }
336
337
338         protected void queueUserMessageToForward(UserDataPacket
              userData){
339           userMessagesToForward.add(userData);
340           synchronized (queueLock) {
341             queueLock.notify();
342           }
343         }
344
345         protected void queueUserMessageFromNode(UserDataPacket
              userPacket){
346           userMessagesFromNode.add(userPacket);
347           synchronized (queueLock) {
348             queueLock.notify();
349           }
350         }
351
352
353         private void cleanUserDataPacketsToForward(int
              destinationAddress){
354           synchronized (userMessagesToForward) {
355             for(UserDataPacket msg: userMessagesToForward){
356               if(msg.getDestinationAddress() == destinationAddress){
357                 userMessagesToForward.remove(msg);
358               }
359             }
360           }
361         }
362
363         /**
364          * Removes every message from the user packet queue that
              matches the given destination
```

```
365         * @param destinationAddress the destination which to look for
366         */
367        private void cleanUserDataPacketsFromNode(int
               destinationAddress){
368           synchronized (userMessagesFromNode) {
369              for(UserDataPacket msg: userMessagesFromNode){
370                 if(msg.getDestinationAddress() == destinationAddress){
371                    userMessagesFromNode.remove(msg);
372                 }
373              }
374           }
375        }
376
377
378        private class NeighbourBroadcaster extends Thread {
379           private volatile boolean keepBroadcasting = true;
380
381           public NeighbourBroadcaster() {
382              super("NeighbourBroadcaster");
383           }
384
385           public void stopBroadcastThread(){
386              keepBroadcasting = false;
387              this.interrupt();
388           }
389
390           public void run(){
391              while(keepBroadcasting){
392                 try {
393                    sleep(Constants.BROADCAST_INTERVAL);
394                    queueHelloPacket(new HelloPacket(nodeAddress,parent.
                           getCurrentSequenceNumber()));
395                 } catch (InterruptedException e) {
396
397                 }
398              }
399           }
400        }
401     }
```

E.1.0.12 Receiver.java

```
1  package adhoc.aodv;
2
3  import java.net.BindException;
4  import java.net.SocketException;
5  import java.net.UnknownHostException;
6  import java.util.ArrayList;
7  import java.util.Queue;
8  import java.util.concurrent.ConcurrentLinkedQueue;
9
```

```
10 import adhoc.aodv.exception.AodvException;
11 import adhoc.aodv.exception.BadPduFormatException;
12 import adhoc.aodv.exception.NoSuchRouteException;
13 import adhoc.aodv.exception.RouteNotValidException;
14 import adhoc.aodv.pdu.HelloPacket;
15 import adhoc.aodv.pdu.RERR;
16 import adhoc.aodv.pdu.RREP;
17 import adhoc.aodv.pdu.RREQ;
18 import adhoc.aodv.pdu.UserDataPacket;
19 import adhoc.aodv.routes.ForwardRouteEntry;
20 import adhoc.etc.Debug;
21 import adhoc.udp.UdpReceiver;
22
23 public class Receiver implements Runnable {
24    private Sender sender;
25    private Queue<Message> receivedMessages;
26    private RouteTableManager routeTableManager;
27    private UdpReceiver udpReceiver;
28    private int nodeAddress;
29    private Thread receiverThread;
30
31     /**
32      */
33     private Node parent;
34    private volatile boolean keepRunning = true;
35
36    public Receiver(Sender sender, int nodeAddress, Node parent,
           RouteTableManager routeTableManager) throws SocketException,
            UnknownHostException, BindException {
37       this.parent = parent;
38       this.nodeAddress = nodeAddress;
39       this.sender = sender;
40       receivedMessages = new ConcurrentLinkedQueue<Message>();
41       this.routeTableManager = routeTableManager;
42       udpReceiver = new UdpReceiver(this, nodeAddress);
43
44    }
45
46    public void startThread(){
47       keepRunning = true;
48       udpReceiver.startThread();
49       receiverThread = new Thread(this);
50       receiverThread.start();
51    }
52
53    /**
54     * Stops the receiver thread.
55     */
56    public void stopThread() {
57       keepRunning = false;
58       udpReceiver.stopThread();
59       receiverThread.interrupt();
60    }
61
62    public void run() {
```

```
63          while (keepRunning) {
64             try {
65                synchronized (receivedMessages) {
66                   while (receivedMessages.isEmpty()) {
67                      receivedMessages.wait();
68                   }
69                }
70
71                Message msg = receivedMessages.poll();
72                if(msg.senderNodeAddress != nodeAddress){
73                   try {
74                      switch (msg.getType()) {
75                      case Constants.HELLO_PDU:
76                         HelloPacket hello = new HelloPacket();
77                         hello.parseBytes(msg.data);
78                         helloMessageReceived(hello);
79                         break;
80                      case Constants.RREQ_PDU:
81                         RREQ rreq = new RREQ();
82                         rreq.parseBytes(msg.data);
83                         routeRequestReceived(rreq, msg.
                               senderNodeAddress);
84                         break;
85                      case Constants.RREP_PDU:
86                         RREP rrep = new RREP();
87                         rrep.parseBytes(msg.data);
88                         routeReplyReceived(rrep, msg.
                               senderNodeAddress);
89                         break;
90                      case Constants.RERR_PDU:
91                         RERR rerr = new RERR();
92                         rerr.parseBytes(msg.data);
93                         routeErrorRecived(rerr);
94                         break;
95                      case Constants.USER_DATA_PACKET_PDU:
96                         UserDataPacket userDataPacket = new
                               UserDataPacket();
97                         userDataPacket.parseBytes(msg.data);
98                         userDataPacketReceived(userDataPacket);
99                         break;
100
101                     default:
102                        // The received message is not in the
                              domain of protocol messages
103                        break;
104                     }
105                  } catch (BadPduFormatException e) {
106                     Debug.print(e.getMessage());
107                  }
108               } else {
109               }
110            } catch (InterruptedException e) {
111               // Thread Stopped
112            }
113         }
```

```
114     }
115
116     /**
117      * Method used by the lower network layer to queue messages for
           later processing
118      *
119      * @param senderNodeAddress Is the address of the node that sent
           a message
120      * @param msg is an array of bytes which contains the sent data
121      */
122     public void addMessage(int senderNodeAddress, byte[] msg) {
123        receivedMessages.add(new Message(senderNodeAddress, msg));
124        synchronized (receivedMessages) {
125           receivedMessages.notify();
126        }
127     }
128
129     /**
130      * Handles a HelloHeader, when such a message is received from a
           neighbor
131      *
132      * @param hello is the HelloHeader message received
133      */
134     private void helloMessageReceived(HelloPacket hello) {
135        try {
136           routeTableManager.setValid(hello.getSourceAddress(), hello
               .getSourceSeqNr());
137        } catch (NoSuchRouteException e) {
138           routeTableManager.createForwardRouteEntry(   hello.
               getSourceAddress(),
139                                                 hello.getSourceAddress(),
140                                                 hello.getSourceSeqNr(),
141                                                 1, true);
142        }
143        Debug.print("Receiver: received hello pdu from: "+hello.
               getSourceAddress());
144     }
145
146     /**
147      * Handles the incoming RREP messages
148      *
149      * @param rrep is the message received
150      * @param senderNodeAddress the address of the sender
151      */
152     private void routeReplyReceived(RREP rrep, int senderNodeAddress
           ) {
153        //rrepRoutePrecursorAddress is an local int used to hold the
               next-hop address from the forward route
154        int rrepRoutePrecursorAddress = -1;
155        //Create route to previous node with unknown seqNum (
               neighbour)
156        if(routeTableManager.createForwardRouteEntry(
               senderNodeAddress,
157                                             senderNodeAddress,
```

```
158                                         Constants.
                                                UNKNOWN_SEQUENCE_NUMBER,
                                                 1, true)){
159            Debug.print("Receiver: RREP where received and route to: "
                   +senderNodeAddress+" where created with destSeq: "+
                   Constants.UNKNOWN_SEQUENCE_NUMBER);
160        }
161        rrep.incrementHopCount();
162
163        if (rrep.getSourceAddress() != nodeAddress) {
164            //forward the RREP, since this node is not the one which
                   requested a route
165            sender.queuePDUmessage(rrep);
166
167            // handle the first part of the route (reverse route) -
                   from this node to the one which originated a RREQ
168            try {
169                //add the sender node to precursors list of the reverse
                       route
170                ForwardRouteEntry reverseRoute = routeTableManager.
                       getForwardRouteEntry(rrep.getSourceAddress());
171                reverseRoute.addPrecursorAddress(senderNodeAddress);
172                rrepRoutePrecursorAddress = reverseRoute.getNextHop();
173            } catch (AodvException e) {
174                //no reverse route is currently known so the RREP is
                       not sure to reach the originator of the RREQ
175            }
176        }
177        // handle the second part of the route - from this node to
               the destination address in the RREP
178        try {
179            ForwardRouteEntry oldRoute = routeTableManager.
                   getForwardRouteEntry(rrep.getDestinationAddress());
180            if(rrepRoutePrecursorAddress != -1){
181                oldRoute.addPrecursorAddress(rrepRoutePrecursorAddress)
                       ;
182            }
183            //see if the RREP contains updates (better seqNum or
                   hopCountNum) to the old route
184            routeTableManager.updateForwardRouteEntry(oldRoute,
185                  new ForwardRouteEntry(  rrep.
                           getDestinationAddress(),
186                                          senderNodeAddress,
187                                          rrep.getHopCount(),
188                                          rrep.
                                               getDestinationSequenceNumber
                                               (),
189                                          oldRoute.getPrecursors()));
190        } catch (NoSuchRouteException e) {
191            ArrayList<Integer> precursorNode = new ArrayList<Integer
                   >();
192            if(rrepRoutePrecursorAddress != -1){
193                precursorNode.add(rrepRoutePrecursorAddress);
194            }
```

```
195             routeTableManager.createForwardRouteEntry(   rrep.
                    getDestinationAddress(),
196                                                 senderNodeAddress,
197                                                 rrep.
                                                        getDestinationSequenceNumber
                                                        (),
198                                                 rrep.getHopCount(),
199                                                 precursorNode, true);
200         } catch (RouteNotValidException e) {
201             //FIXME den er gal paa den
202             Debug.print("Receiver: FATAL ERROR");
203             try {
204                 //update the previously known route with the better
                        route contained in the RREP
205                 routeTableManager.setValid(rrep.getDestinationAddress()
                        , rrep.getDestinationSequenceNumber());
206                 if(rrepRoutePrecursorAddress != -1){
207                     routeTableManager.getForwardRouteEntry(rrep.
                            getDestinationAddress()).addPrecursorAddress(
                            rrepRoutePrecursorAddress);
208                 }
209             }catch (AodvException e1) {
210
211             }
212         }
213     }
214
215     /**
216      * Handles a RREQ message when received
217      *
218      * @param rreq the RREQ message that were received
219      * @param senderNodeAddress the node (a neighbor) which sent
            this message
220      */
221     private void routeRequestReceived(RREQ rreq, int
            senderNodeAddress) {
222         if (routeTableManager.routeRequestExists(rreq.
                getSourceAddress(), rreq.getBroadcastId())) {
223             return;
224         }
225         //Create route to previous node with unknown seqNum (
                neighbour)
226         if(routeTableManager.createForwardRouteEntry(
                senderNodeAddress,
227                                                 senderNodeAddress,
228                                                 Constants.
                                                        UNKNOWN_SEQUENCE_NUMBER,
                                                         1, true)){
229             Debug.print("Receiver: RREQ where received from: "+
                    senderNodeAddress+" and route where created with
                    destSeq: "+Constants.UNKNOWN_SEQUENCE_NUMBER);
230         }
231
232         // Increments the hopCount and Adds the RREQ to the table
233         rreq.incrementHopCount();
```

```
234         routeTableManager.createRouteRequestEntry(rreq, true);
235
236         //a reverse route may already exists, so we need to compare
                route info value to know what to update
237         try {
238            ForwardRouteEntry oldRoute = routeTableManager.
                   getForwardRouteEntry(rreq.getSourceAddress());
239
240            if(isIncomingRouteInfoBetter( rreq.getSourceSequenceNumber
                   (),
241                                    oldRoute.
                                           getDestinationSequenceNumber()
                                           ,
242                                    rreq.getHopCount(),
243                                    oldRoute.getHopCount())){
244               //remove the old entry and then replace with new
                       information
245               routeTableManager.updateForwardRouteEntry(oldRoute,
246                     new ForwardRouteEntry(  rreq.getSourceAddress(),
247                                       senderNodeAddress,
248                                       rreq.getHopCount(),
249                                       rreq.getSourceSequenceNumber(),
250                                       oldRoute.getPrecursors()));
251            }
252         } catch (NoSuchRouteException e) {
253            // Creates a reverse route for the RREP that may be
                   received later on
254            routeTableManager.createForwardRouteEntry(   rreq.
                   getSourceAddress(),
255                                          senderNodeAddress,
256                                          rreq.
                                              getSourceSequenceNumber
                                              (),
257                                          rreq.getHopCount(), true)
                                              ;
258         } catch (RouteNotValidException e) {
259            try {
260               routeTableManager.setValid(rreq.getSourceAddress(),
                      rreq.getSourceSequenceNumber());
261            } catch (NoSuchRouteException e1) {
262               routeTableManager.createForwardRouteEntry(   rreq.
                      getSourceAddress(),
263                                             senderNodeAddress,
264                                             rreq.
                                                 getSourceSequenceNumber
                                                 (),
265                                             rreq.getHopCount(),
                                                 true);
266            }
267         }
268
269         //check if this node is the destination,
270         RREP rrep = null;
271         try {
272            if (rreq.getDestinationAddress() == nodeAddress) {
```

```
273             if(parent.getNextSequenceNumber(parent.
                    getCurrentSequenceNumber()) == rreq.
                    getDestinationSequenceNumber()){
274                parent.getNextSequenceNumber();
275             }
276             // the RREQ has reached it's destination, so this node
                    has to reply with a RREP
277             rrep = new RREP(  rreq.getSourceAddress(),
278                               nodeAddress,
279                               rreq.getSourceSequenceNumber(),
280                               parent.getCurrentSequenceNumber()    );
281          } else {
282             //this node is not the destination of the RREQ so we
                    need to check if we have the requested route
283             ForwardRouteEntry entry = routeTableManager.
                    getForwardRouteEntry(rreq.getDestinationAddress());
284
285             // If a valid route exists with a seqNum thats is
                    grater or equal to the RREQ, then send a RREP
286             if (isIncomingSeqNrBetter(entry.
                    getDestinationSequenceNumber(), rreq.
                    getDestinationSequenceNumber())) {
287                rrep = new RREP(  rreq.getSourceAddress(),
288                                  entry.getDestinationAddress(),
289                                  rreq.getSourceSequenceNumber(),
290                                  entry.getDestinationSequenceNumber(),
291                                  entry.getHopCount()  );
292                // Gratuitous RREP for the destination Node
293                RREP gRrep = new RREP(  entry.getDestinationAddress
                       (),
294                                        rreq.getSourceAddress(),
295                                        entry.getDestinationSequenceNumber
                                           (),
296                                        rreq.getSourceSequenceNumber(),
297                                        rreq.getHopCount()    );
298                sender.queuePDUmessage(gRrep);
299             }
300          }
301       } catch (NoSuchRouteException e) {
302          //this node is an intermediate node, but do not know a
                route to the desired destination
303       } catch (RouteNotValidException e) {
304          //this node know a route but it is not active any longer.
305          try {
306             int maxSeqNum = getMaximumSeqNum(routeTableManager.
                    getLastKnownDestSeqNum(rreq.getDestinationAddress()
                    ),
307                                              rreq.
                                                 getDestinationSequenceNumber
                                                 () );
308             rreq.setDestSeqNum(maxSeqNum);
309          } catch (NoSuchRouteException e1) {
310             //table route were deleted by the timer
311          }
312       } finally {
```

```
313             //if a RREP is created, then send it, otherwise broadcast
                    the RREQ
314             if (rrep == null) {
315                sender.queuePDUmessage(rreq);
316             } else {
317                sender.queuePDUmessage(rrep);
318             }
319          }
320       }
321
322       /**
323        * Handles a RERR message when received
324        *
325        * @param rerrMsg is the received error message
326        */
327       private void routeErrorRecived(RERR rerrMsg) {
328          Debug.print("Receiver: RRER received, unreachableNode: "+
               rerrMsg.getUnreachableNodeAddress());
329          try {
330             ForwardRouteEntry entry = routeTableManager.
                   getForwardRouteEntry(
331                                                 rerrMsg.
                                                     getUnreachableNodeAddress
                                                     ());
332
333             //only send a RERR if the message contain a seqNum that is
                    greater or equal to the entry known in the table
334             if (isIncomingSeqNrBetter(rerrMsg.
                   getUnreachableNodeSequenceNumber(),
335                                       entry.getDestinationSequenceNumber())
                                           )
336             {
337                RERR rerr = new RERR(   rerrMsg.
                      getUnreachableNodeAddress(),
338                                       rerrMsg.
                                           getUnreachableNodeSequenceNumber
                                           (),
339                                       entry.getPrecursors()    );
340                sender.queuePDUmessage(rerr);
341                routeTableManager.setInvalid(rerrMsg.
                      getUnreachableNodeAddress(), rerrMsg.
                      getUnreachableNodeSequenceNumber());
342             }
343          } catch (AodvException e) {
344             //no route is known so we do not have to react on the
                    error message
345          }
346       }
347
348       /**
349        * Handles a userDataPacket when received
350        * @param userData is the received packet
351        * @param senderNodeAddress the originator of the message
352        */
353       private void userDataPacketReceived(UserDataPacket userData) {
```

```
354         if (userData.getDestinationAddress() == nodeAddress
355                 || userData.getDestinationAddress() == Constants.
                        BROADCAST_ADDRESS    ) {
356             parent.notifyAboutDataReceived(userData.
                    getSourceNodeAddress(), userData.getData());
357         } else {
358             sender.queueUserMessageToForward(userData);
359         }
360     }
361
362     /**
363      * Computes the maximum of the two sequence numbers, such that
            the possibility of rollover is taken to account
364      * @param firstSeqNum the first of the given sequence numbers
            which to compare
365      * @param secondSeqNum the second of the given sequence numbers
            which to compare
366      * @return returns the maximum sequence number
367      */
368     public static int getMaximumSeqNum(int firstSeqNum, int
            secondSeqNum){
369         if(isIncomingSeqNrBetter(firstSeqNum, secondSeqNum)){
370             return firstSeqNum;
371         } else {
372             return secondSeqNum;
373         }
374     }
375
376     /**
377      * Used to compare sequence numbers
378      * @param incomingSeqNum the sequence number contained in a
            received AODV PDU message
379      * @param currentSeqNum the sequence number contained in a known
            forward route
380      * @return returns true if incomingSeqNr is greater or equal to
            currentSeqNr
381      */
382     private static boolean isIncomingSeqNrBetter(int incomingSeqNum,
            int currentSeqNum) {
383         return isIncomingRouteInfoBetter(incomingSeqNum,
                currentSeqNum, 0, 1);
384     }
385
386     /**
387      * Used to compare sequence numbers and hop count
388      * @param incommingSeqNum the sequence number contained in a
            received AODV PDU message
389      * @param currentSeqNum the sequence number contained in a known
            forward route
390      * @return returns true if incomingSeqNum > currentSeqNum OR
            incomingSeqNum == currentSeqNum AND incomingHopCount <
            currentHopCount
391      */
392     protected static boolean isIncomingRouteInfoBetter(int
            incomingSeqNum, int currentSeqNum, int incomingHopCount, int
```

```
          currentHopCount) {
393       if (Math.abs(incomingSeqNum - currentSeqNum) > Constants.
              SEQUENCE_NUMBER_INTERVAL) {
394
395          if ((incomingSeqNum % Constants.SEQUENCE_NUMBER_INTERVAL)
                 >= (currentSeqNum % Constants.SEQUENCE_NUMBER_INTERVAL
                 )) {
396             if ((incomingSeqNum % Constants.
                    SEQUENCE_NUMBER_INTERVAL) == (currentSeqNum %
                    Constants.SEQUENCE_NUMBER_INTERVAL)
397                   && incomingHopCount > currentHopCount) {
398                return false;
399             }
400             return true;
401          } else {
402             //the node have an older route so it should not be used
403             return false;
404          }
405       } else {
406          if (incomingSeqNum >= currentSeqNum) {
407             if (incomingSeqNum == currentSeqNum && incomingHopCount
                    > currentHopCount) {
408                return false;
409             }
410             return true;
411          } else {
412             return false;
413          }
414       }
415    }
416
417    /**
418     * @author Rabie A class to contain the received data from a
            lower network layer (UDP). Objects
419     *         of this type is stored in a receiving queue for later
            processing
420     *
421     */
422    private class Message {
423       private int senderNodeAddress;
424       private byte[] data;
425
426       public Message(int senderNodeAddress, byte[] data) {
427          this.senderNodeAddress = senderNodeAddress;
428          this.data = data;
429       }
430
431       public byte getType() throws NumberFormatException {
432          String[] s = new String(data).split(";", 2);
433          if(s.length == 2){
434             return Byte.parseByte(s[0]);
435          } else
436             throw new NumberFormatException();
437       }
438    }
```

```
439 }
```

E.2 Routes

E.2.0.13 ForwardRouteTable.java

```
1  package adhoc.aodv.routes;
2
3  import java.util.ArrayList;
4  import java.util.HashMap;
5  import java.util.LinkedList;
6
7  import adhoc.aodv.Receiver;
8  import adhoc.aodv.exception.NoSuchRouteException;
9  import adhoc.aodv.exception.RouteNotValidException;
10 import adhoc.aodv.pdu.RERR;
11 import adhoc.etc.Debug;
12
13 public class ForwardRouteTable {
14
15    private HashMap<Integer, ForwardRouteEntry> entries;
16    private LinkedList<ForwardRouteEntry> sortedEntries;
17    private final Object tableLock = new Integer(0);
18
19    public ForwardRouteTable() {
20       // contains known routes
21       entries = new HashMap<Integer, ForwardRouteEntry>();
22
23       // containing the known routes, sorted such that the route
             with the
24       // least 'aliveTimeLeft' is head
25       sortedEntries = new LinkedList<ForwardRouteEntry>();
26    }
27
28    /**
29     * Adds the given entry to the forwardRoute table
30     * @param forwardRouteEntry the entry to be stored
31     * @return returns true if the route were added successfully. A
           successful add requires that no matching entry exists in
           the table
32     */
33    public boolean addForwardRouteEntry(ForwardRouteEntry
          forwardRouteEntry) {
34       synchronized (tableLock) {
35          if(!entries.containsKey(forwardRouteEntry.
                getDestinationAddress())) {
36             entries.put(forwardRouteEntry.getDestinationAddress(),
                   forwardRouteEntry);
37             sortedEntries.addLast(forwardRouteEntry);
38             Debug.print("ForwardRouteTable: Adding new forward
                   route entry for dest: "+forwardRouteEntry.
                   getDestinationAddress());
39             Debug.print(this.toString());
40             return true;
41          }
```

```
42              return false;
43          }
44      }
45
46      /**
47       *
48       * @param destAddress the destination address which to search
             for in the table
49       * @return returns false if the route does not exist
50       */
51      public boolean removeEntry(int destAddress){
52          synchronized (tableLock) {
53              RouteEntry entry = entries.remove(destAddress);
54              if (entry != null) {
55                  sortedEntries.remove(entry);
56                  Debug.print("ForwardRouteTable: removing forward route
                        entry for dest: "+destAddress);
57                  Debug.print(this.toString());
58                  return true;
59              }
60              return false;
61          }
62      }
63
64      public boolean updateForwardRouteEntry(ForwardRouteEntry entry)
            throws NoSuchRouteException{
65          synchronized (tableLock) {
66              if(removeEntry(entry.getDestinationAddress())
67                    && addForwardRouteEntry(entry)){
68                  Debug.print("updateForwardRouteEntry: Updating route
                        for dest: "+entry.getDestinationAddress() );
69                  return true;
70              }
71          }
72          throw new NoSuchRouteException();
73      }
74
75      /**
76       * Method used to known the last known information about a
             routes 'freshness'
77       * @param destinationAddress the given destination which to
             search for in the table
78       * @return returns the destination sequence number of the
             forward entry
79       * @throws NoSuchRouteException is thrown if no such exists
80       */
81      public int getLastKnownDestSeqNumber(int destinationAddress)
            throws NoSuchRouteException{
82          RouteEntry entry = entries.get(destinationAddress);
83          if(entry != null){
84              return entry.getDestinationSequenceNumber();
85          }
86          throw new NoSuchRouteException();
87      }
88
```

```
89      public ArrayList<Integer> getPrecursors(int destinationAddress){
90          synchronized (tableLock) {
91              ForwardRouteEntry entry = entries.get(destinationAddress);
92              if(entry != null){
93                  return entry.getPrecursors();
94              }
95              return new ArrayList<Integer>();
96          }
97      }
98
99      /**
100     * Makes a forward route valid, updates it sequence number if
            necessary and resets the AliveTimeLeft
101     * @param destinationAddress used to determine which forward
            route to set valid
102     * @param newDestinationSeqNumber this destSeqNum is only set in
            the entry if it is greater that the existing destSeqNum
103     * @throws NoSuchRouteException thrown if no table information
            is known about the destination
104     */
105     public void setValid(int destinationAddress, int
            destinationSeqNumber, boolean validValue) throws
            NoSuchRouteException {
106         ForwardRouteEntry entry = entries.get(destinationAddress);
107         if(entry != null){
108             entry.setValid(validValue);
109             entry.resetAliveTimeLeft();
110             synchronized (tableLock) {
111                 sortedEntries.remove(entry);
112                 sortedEntries.addLast(entry);
113             }
114             entry.setSeqNum(  Receiver.getMaximumSeqNum(
                    destinationSeqNumber,
115                                                 entry.
                                                        getDestinationSequenceNumber
                                                        ()    )  );
116             return;
117         }
118         throw new NoSuchRouteException();
119     }
120
121     /**
122      *
123      * @param nodeAddress
124      * @return RouteEntry
125      * @throws NoSuchRouteException thrown if no table information
            is known about the destination
126      * @throws RouteNotValidException thrown if a route were found,
            but is marked as invalid
127      */
128     public ForwardRouteEntry getForwardRouteEntry(int
            destinationAddress) throws NoSuchRouteException,
            RouteNotValidException {
129         ForwardRouteEntry entry = entries.get(destinationAddress);
130         if (entry != null) {
```

```
131             entry.resetAliveTimeLeft();
132             synchronized (tableLock) {
133                 sortedEntries.remove(entry);
134                 sortedEntries.addLast(entry);
135             }
136             if (!(entry).isValid()) {
137                 throw new RouteNotValidException();
138             }
139             return entry;
140         }
141         throw new NoSuchRouteException();
142     }
143 
144     /**
145      * Method for knowing if the table (sorted list) contain any
             entries
146      * @return true if the sortedlist is empty
147      */
148     public boolean isEmpty(){
149         return sortedEntries.isEmpty();
150     }
151 
152     /**
153      *
154      * @return returns the route entry with the minimum time to live
             before expire
155      * @throws NoSuchRouteException is thrown if no such exists
156      */
157     public RouteEntry getNextRouteToExpire() throws
             NoSuchRouteException{
158         RouteEntry route = null;
159         route = sortedEntries.peek();
160         if(route != null){
161             return route;
162         }
163         throw new NoSuchRouteException();
164     }
165 
166     /**
167      * Searches the table for routes which match on the '
             nextHopAddress'.
168      * The destination node of the matching entries is then used in
             a RERR pdu for later processing.
169      * The state of matching route entries is set to invalid
170      * @param brokenNodeAddress is the destination node which can
             not be reached any more
171      * @return ArrayList<RERR> returns an ArrayList of RERR messages
172      */
173     public ArrayList<RERR> findBrokenRoutes(int brokenNodeAddress){
174         ArrayList<RERR> brokenRoutes = new ArrayList<RERR>();
175         LinkedList<ForwardRouteEntry> currentEntries = new LinkedList
                 <ForwardRouteEntry>();
176         synchronized (tableLock) {
177             for(ForwardRouteEntry entry: sortedEntries){
178                 currentEntries.add(entry);
```

```
179             }
180
181             for(ForwardRouteEntry entry : currentEntries){
182                 if(entry.getNextHop() == brokenNodeAddress){
183                     RERR rerr = new RERR(entry.getDestinationAddress(),
                            entry.getDestinationSequenceNumber(), entry.
                            getPrecursors());
184                     brokenRoutes.add(rerr);
185                     try {
186                         setValid(entry.getDestinationAddress(), entry.
                                getDestinationSequenceNumber(), false);
187                     } catch (NoSuchRouteException e) {
188                         Debug.print("RouteTableManager:
                                NoSuchRouteException where thrown in
                                findBrokenRoutes");
189                     }
190                 }
191             }
192         }
193         return brokenRoutes;
194     }
195
196     /**
197      * only used for debugging
198      */
199     public String toString(){
200         synchronized (tableLock) {
201             if(entries.size() != sortedEntries.size()){
202                 Debug.print("ForwardRouteTable: FATAL ERROR -
                        inconsistensy in this table");
203             }
204             if(entries.isEmpty()){
205                 return "Forward Table is empty\n";
206             }
207             String returnString = "----------------------\n"+
208                                   "|Forward Route Table:\n"+
209                                   "----------------------";
210             for(ForwardRouteEntry f :entries.values()){
211                 returnString += "\n"+"|Dest: "+f.getDestinationAddress
                        ()+" destSeqN: "+f.getDestinationSequenceNumber()+"
                         nextHop: "+f.getNextHop()+" hopCount: "+f.
                        getHopCount()+" isValid: "+f.isValid()+" TTL: "+(f.
                        getAliveTimeLeft()-System.currentTimeMillis())+"
                        precursors: ";
212                 for(int p  : f.getPrecursors()){
213                     returnString += p+" ";
214                 }
215             }
216             return returnString+"\n----------------------\n";
217         }
218     }
219 }
```

E.2.0.14 RouteRequestTable.java

```
1  package adhoc.aodv.routes;
2
3  import java.util.HashMap;
4  import java.util.LinkedList;
5
6  import adhoc.aodv.exception.NoSuchRouteException;
7  import adhoc.etc.Debug;
8
9  public class RouteRequestTable {
10
11    private HashMap<EntryKey, RouteRequestEntry> entries;
12    private LinkedList<RouteRequestEntry> sortedEntries;
13    private final Object tableLock = new Integer(0);
14
15    public RouteRequestTable(){
16       // contains known routes
17       entries = new HashMap<EntryKey, RouteRequestEntry>();
18
19       // containing the known routes, sorted such that the route
             with the
20       // least 'aliveTimeLeft' is head
21       sortedEntries = new LinkedList<RouteRequestEntry>();
22    }
23
24
25    public boolean routeRequestEntryExists(int sourceAddress, int
          broadcastID){
26       return entries.containsKey(new EntryKey(sourceAddress,
             broadcastID));
27    }
28
29    /**
30     * Adds the given entry to the RREQ table
31     * @param rreqEntry the entry to be stored
32     * @param setTimer is set to false if the timer should not start
           count down the entry's time
33     * @return returns true if the route were added successfully. A
         successful add requires that no matching entry exists in
         the table
34     */
35    public boolean addRouteRequestEntry(RouteRequestEntry rreqEntry,
         boolean setTimer){
36       synchronized (tableLock) {
37          EntryKey key = new EntryKey(rreqEntry.getSourceAddress(),
               rreqEntry.getBroadcastID());
38          if(!entries.containsKey(key)){
39             entries.put(key, rreqEntry);
40             Debug.print(toString());
41             if(setTimer){
42                sortedEntries.addLast(rreqEntry);
43             }
44             return true;
```

```
45          }
46          return false;
47       }
48    }
49
50    public void setRouteRequestTimer(int sourceAddres, int
          broadcastID) throws NoSuchRouteException{
51       RouteRequestEntry rreqEntry = entries.get(new EntryKey(
             sourceAddres, broadcastID));
52       if(rreqEntry != null){
53          rreqEntry.resetAliveTimeLeft();
54          synchronized (tableLock) {
55             sortedEntries.addLast(rreqEntry);
56          }
57          return;
58       }
59       throw new NoSuchRouteException();
60    }
61
62    /**
63     * This method returns the desired RREQ entry
64     * OtherWise the method returns and remove the RREQ entry from
            the table
65     * @param sourceAddress the originator of the RREQ broadcast
66     * @param broadcastID the ID of the RREQ broadcast
67     * @param removeEntry is set to true if the table also should
            remove the entry in the table
68     * @return returns a RREQ entry
69     * @throws NoSuchRouteException Thrown if no table information
            is known about the entry
70     */
71    public RouteEntry getRouteRequestEntry(int sourceAddress, int
          broadcastID, boolean removeEntry) throws
          NoSuchRouteException{
72       synchronized (tableLock) {
73          RouteRequestEntry entry = (RouteRequestEntry)entries.get(
                new EntryKey(sourceAddress, broadcastID));
74          if (entry != null) {
75             if(removeEntry){
76                removeEntry(entry.getSourceAddress(), entry.
                      getBroadcastID());
77             }
78             return entry;
79          }
80          throw new NoSuchRouteException();
81       }
82    }
83
84    /**
85     * Removes an rreq entry from the table
86     * @param sourceAddress the node address of the originator
87     * @param broadcastID the broadcastID of the originator
88     * @return returns true if the entry existed and where removed
            successfully
89     */
```

```
90      public boolean removeEntry(int sourceAddress, int broadcastID) {
91          synchronized (tableLock) {
92              RouteEntry rreqEntry = entries.remove(new EntryKey(
                    sourceAddress,broadcastID));
93              if (rreqEntry != null) {
94                  sortedEntries.remove(rreqEntry);
95                  Debug.print(toString());
96                  return true;
97              }
98              return false;
99          }
100     }
101
102     public RouteEntry getNextRouteToExpire() throws
            NoSuchRouteException{
103         RouteEntry route = sortedEntries.peek();
104         if(route != null){
105             return route;
106         }
107         throw new NoSuchRouteException();
108     }
109
110     public boolean isEmpty(){
111         return sortedEntries.isEmpty();
112     }
113
114
115     /**
116      * A route request is uniquely defined by the tuple (nodeAddress
               , broadcastID).
117      * This class is then used for generating a hashcode from this
               tuple so entries can be stored with an appropriate key
118      * @author Rabie
119      *
120      */
121     private class EntryKey{
122         private int nodeAddress;
123         private int broadcastID;
124
125         public EntryKey(int nodeAddress, int broadcastID){
126             this.nodeAddress = nodeAddress;
127             this.broadcastID = broadcastID;
128         }
129
130         @Override
131         public boolean equals(Object obj) {
132             EntryKey k = (EntryKey)obj;
133             if(k.getNodeAddress() == nodeAddress && k.getBroadcastID()
                        == broadcastID){
134                 return true;
135             }
136             return false;
137         }
138
139         @Override
```

```
140         public int hashCode(){
141             return (Integer.toString(nodeAddress)+";"+Integer.toString
                    (broadcastID)).hashCode();
142         }
143 
144         public int getNodeAddress(){
145             return nodeAddress;
146         }
147 
148         public int getBroadcastID(){
149             return broadcastID;
150         }
151     }
152 
153     public String toString(){
154         synchronized (tableLock) {
155             if(entries.isEmpty()){
156                 return "RouteRequestTable is empty\n";
157             }
158             String returnString = "----------------------\n"+
159                                   "|Route Request Table:\n"+
160                                   "----------------------";
161             for(RouteRequestEntry f :entries.values()){
162                 returnString += "\n"+"|Dest: "+f.getDestinationAddress
                        ()+" destSeqN: "+f.getDestinationSequenceNumber()+"
                         src: "+f.getSourceAddress()+" broadID: "+f.
                        getBroadcastID()+" retries left: "+f.getRetriesLeft
                        ()+" hopCount: "+f.getHopCount()+" TTL: "+(f.
                        getAliveTimeLeft()-System.currentTimeMillis());
163 
164             }
165             return returnString+"\n----------------------\n";
166         }
167     }
168 }
```

E.2.0.15 RouteEntry.java

```
1  package adhoc.aodv.routes;
2 
3  import adhoc.aodv.Constants;
4  import adhoc.aodv.exception.RouteNotValidException;
5 
6  public abstract class RouteEntry {
7      protected int destAddress;
8      protected volatile long alivetimeLeft;
9      protected volatile int destSeqNum;
10     protected int hopCount;
11     protected final Object aliveTimeLock = new Integer(0);
12 
```

```
13      public RouteEntry(int hopCount, int destSeqNum, int destAddress
            ) throws RouteNotValidException{
14        if(destAddress <= Constants.MAX_VALID_NODE_ADDRESS &&
              destAddress >= Constants.MIN_VALID_NODE_ADDRESS
15              && (destSeqNum <= Constants.MAX_SEQUENCE_NUMBER
16                    && destSeqNum >= Constants.FIRST_SEQUENCE_NUMBER
17                     || destSeqNum == Constants.
                           UNKNOWN_SEQUENCE_NUMBER)){
18            this.hopCount = hopCount;
19            this.destSeqNum = destSeqNum;
20            this.destAddress = destAddress;
21        } else {
22            throw new RouteNotValidException("RouteEntry: invalid
                  parameters given");
23        }
24      }
25
26      /**
27       *
28       * @return the system time of when the route becomes stale
29       */
30      public long getAliveTimeLeft(){
31        synchronized (aliveTimeLock) {
32            return alivetimeLeft;
33        }
34      }
35
36      public abstract void resetAliveTimeLeft();
37
38      public int getDestinationSequenceNumber(){
39          return destSeqNum;
40      }
41
42      public int getHopCount(){
43        return hopCount;
44      }
45
46      public int getDestinationAddress() {
47          return destAddress;
48      }
49
50  }
```

E.2.0.16 ForwardRouteEntry.java

```
1   package adhoc.aodv.routes;
2
3
4   import java.util.ArrayList;
5
6   import adhoc.aodv.Constants;
```

```
7  import adhoc.aodv.exception.RouteNotValidException;
8  import adhoc.etc.Debug;
9
10 public class ForwardRouteEntry extends RouteEntry {
11
12     private ArrayList<Integer> precursorNodes = new ArrayList<
           Integer>();
13     private volatile boolean isValid = true;
14     private int nextHop;
15
16     public ForwardRouteEntry(int destAddress, int nextHopAddress,
           int hopCount, int destSeqNum, ArrayList<Integer>
           precursorNodes) throws RouteNotValidException {
17       super(hopCount, destSeqNum, destAddress);
18       if(nextHopAddress <= Constants.MAX_VALID_NODE_ADDRESS
19             && nextHopAddress >= Constants.MIN_VALID_NODE_ADDRESS
20             && precursorNodes != null){
21          this.nextHop = nextHopAddress;
22            for(int node : precursorNodes){
23             addPrecursorAddress(node);
24            }
25            resetAliveTimeLeft();
26          } else {
27             throw new RouteNotValidException("RouteEntry: invalid
                   parameters given");
28          }
29     }
30
31     /**
32      * Adds node as a precursor, so a RRER can be sent to this node
              in case of route failure
33      * @param nodeAddress the address of the node which is using
              this forward route
34      * @return
35      */
36     public boolean addPrecursorAddress(int nodeAddress){
37         synchronized (precursorNodes) {
38          if(!precursorNodes.contains(nodeAddress)
39                && nodeAddress <= Constants.MAX_VALID_NODE_ADDRESS
40                && nodeAddress >= Constants.MIN_VALID_NODE_ADDRESS){
41             precursorNodes.add(nodeAddress);
42             return true;
43          }
44          return false;
45       }
46     }
47
48     public ArrayList<Integer> getPrecursors(){
49       ArrayList<Integer> copy = new ArrayList<Integer>();
50       synchronized (precursorNodes) {
51          for(int address: precursorNodes){
52             copy.add(address);
53          }
54       }
55       return copy;
```

```
56      }
57
58      public void resetAliveTimeLeft(){
59        synchronized (aliveTimeLock) {
60           alivetimeLeft = Constants.ROUTE_ALIVETIME + System.
                 currentTimeMillis();
61        }
62      }
63
64      /**
65       * @return returns true if this route is allowed to be used for
               packet forwarding.
66       */
67      public boolean isValid(){
68        return isValid;
69      }
70
71      public void setValid(boolean valid){
72        if(isValid != valid){
73           Debug.print("Forward Entry: isValid has changed to: "+
                 valid);
74        }
75        isValid = valid;
76      }
77
78      public boolean setSeqNum(int newSeqNr){
79        if(newSeqNr >= Constants.FIRST_SEQUENCE_NUMBER && newSeqNr <=
               Constants.MAX_SEQUENCE_NUMBER){
80           destSeqNum = newSeqNr;
81           return true;
82        }
83        return false;
84      }
85
86      public int getNextHop(){
87        return nextHop;
88      }
89  }
```

E.2.0.17 RouteRequestEntry.java

```
1   package adhoc.aodv.routes;
2
3   import adhoc.aodv.Constants;
4   import adhoc.aodv.exception.RouteNotValidException;
5
6   public class RouteRequestEntry extends RouteEntry{
7      //unique id for each route request
8      private volatile int broadcastID;
9      private int sourceAddress;
10     //the number of route request retries 'RREQ' has left
```

```
11      private int retries = Constants.MAX_NUMBER_OF_RREQ_RETRIES;
12
13
14  /**
15   * @param broadcastID unique id for this route request
16   * @param sourceAddress
17   * @param destinationSequenceNumber
18   * @param hopCount
19   * @throws RouteNotValidException
20   */
21      public RouteRequestEntry(int broadcastID, int sourceAddress, int
             destinationSequenceNumber, int hopCount, int
            destinationAddress) throws RouteNotValidException{
22          super(hopCount, destinationSequenceNumber,destinationAddress)
                ;
23          if(sourceAddress <= Constants.MAX_VALID_NODE_ADDRESS
24                  && sourceAddress >= Constants.MIN_VALID_NODE_ADDRESS
25                  && broadcastID <= Constants.MAX_BROADCAST_ID
26                  && broadcastID >= Constants.FIRST_BROADCAST_ID){
27              this.sourceAddress = sourceAddress;
28              this.broadcastID = broadcastID;
29              resetAliveTimeLeft();
30          } else {
31              throw new RouteNotValidException("RouteEntry: invalid
                    parameters given");
32          }
33      }
34
35      public int getSourceAddress(){
36          return sourceAddress;
37      }
38
39      public int getBroadcastID(){
40          return broadcastID;
41      }
42
43      public void resetAliveTimeLeft(){
44          synchronized (aliveTimeLock) {
45              alivetimeLeft = System.currentTimeMillis() + Constants.
                    PATH_DESCOVERY_TIME;
46          }
47      }
48
49      /**
50       * Method only used by the timer thread, to decrement the number
             of retries which this request is sent
51       * @return returns false if the request has been sent 3 times
52       */
53      public boolean resend(){
54          retries--;
55          if(retries <= 0){
56              return false;
57          }
58          return true;
59      }
```

```
60
61      /**
62       * Only used for debugging purposes!
63       * @return number of flood retries left
64       */
65      public int getRetriesLeft(){
66          return retries;
67      }
68
69      public boolean setBroadcastID(int broadcastID) {
70          if(broadcastID <= Constants.MAX_BROADCAST_ID && broadcastID
                >= Constants.FIRST_BROADCAST_ID){
71              this.broadcastID = broadcastID;
72              return true;
73          }
74          else return false;
75      }
76
77      /**
78       * used for debugging
79       */
80      public String toString(){
81          return "SrcAdr:"+sourceAddress+" BroadID:"+broadcastID+"
                Retries:"+retries+" DestAdr:"+destAddress+" DestSeqNumb:"
                +destSeqNum;
82      }
83  }
```

E.3 Udp

E.3.0.18 UdpSender.java

```
1  package adhoc.udp;
2
3  import java.io.IOException;
4  import java.net.BindException;
5  import java.net.DatagramPacket;
6  import java.net.DatagramSocket;
7  import java.net.InetAddress;
8  import java.net.SocketException;
9  import java.net.UnknownHostException;
10
11 import javax.naming.SizeLimitExceededException;
12
13 import adhoc.aodv.Constants;
14 import adhoc.aodv.exception.DataExceedsMaxSizeException;
15
16 public class UdpSender {
17    private DatagramSocket datagramSocket;
18    private int receiverPort = 8888;
19    private String subNet = "192.168.2.";
20
21    public UdpSender() throws SocketException, UnknownHostException,
          BindException{
22       datagramSocket = new DatagramSocket(8881);
23    }
24
25    /**
26     * Sends data using the UDP protocol to a specific receiver
27     * @param destinationNodeID indicates the ID of the receiving
          node. Should be a positive integer.
28     * @param data is the message which is to be sent.
29     * @throws IOException
30     * @throws SizeLimitExceededException is thrown if the length of
          the data to be sent exceeds the limit
31     */
32    public boolean sendPacket(int destinationNodeID, byte[] data)
          throws IOException, DataExceedsMaxSizeException{
33       if(data.length <= Constants.MAX_PACKAGE_SIZE){
34             InetAddress IPAddress = InetAddress.getByName(subNet+
                  destinationNodeID);
35             //do we have a packet to be broadcasted?
36             DatagramPacket sendPacket;
37             if(destinationNodeID == Constants.BROADCAST_ADDRESS){
38                datagramSocket.setBroadcast(true);
39                sendPacket = new DatagramPacket(data, data.length,
                      IPAddress, receiverPort+1);
40             }else {
41                datagramSocket.setBroadcast(false);
42                sendPacket = new DatagramPacket(data, data.length,
                      IPAddress, receiverPort);
```

```
43              }
44
45              datagramSocket.send(sendPacket);
46              return true;
47          } else {
48              throw new DataExceedsMaxSizeException();
49          }
50      }
51
52      public void closeSoket(){
53          datagramSocket.close();
54      }
55
56  }
```

E.3.0.19 UdpReceiver.java

```
1   package adhoc.udp;
2
3   import java.io.IOException;
4   import java.net.BindException;
5   import java.net.DatagramPacket;
6   import java.net.DatagramSocket;
7   import java.net.InetSocketAddress;
8   import java.net.SocketException;
9   import java.net.UnknownHostException;
10
11  import adhoc.aodv.Receiver;
12
13
14  /**
15   * Class running as a separate thread, and responsible for
         receiving data packets over the UDP protocol.
16   * @author Rabie
17   *
18   */
19  public class UdpReceiver implements Runnable{
20      private Receiver parent;
21      private DatagramSocket datagramSocket;
22      private UdpBroadcastReceiver udpBroadcastReceiver;
23      private volatile boolean keepRunning = true;
24      private Thread udpReceiverthread;
25
26      public UdpReceiver(Receiver parent, int nodeAddress) throws
            SocketException, UnknownHostException, BindException{
27          this.parent = parent;
28          datagramSocket = new DatagramSocket(new InetSocketAddress("
                192.168.2."+nodeAddress ,8888));
29          datagramSocket.setBroadcast(true);
30          udpBroadcastReceiver = new UdpBroadcastReceiver(8888);
31      }
```

```
32
33      public void startThread(){
34          keepRunning = true;
35          udpBroadcastReceiver.startBroadcastReceiverthread();
36          udpReceiverthread = new Thread(this);
37          udpReceiverthread.start();
38      }
39
40      public void stopThread(){
41          keepRunning = false;
42          udpBroadcastReceiver.stopBroadcastThread();
43          udpReceiverthread.interrupt();
44      }
45
46      public void run(){
47          while(keepRunning){
48              try {
49                  // 52kb buffer
50                  byte[] buffer = new byte[52000];
51                  DatagramPacket receivePacket = new DatagramPacket(
                        buffer,buffer.length);
52
53                  datagramSocket.receive(receivePacket);
54                   byte[] result = new byte[receivePacket.getLength()];
55                   System.arraycopy(receivePacket.getData(), 0, result,
                        0, receivePacket.getLength());
56
57                   String[] ip = receivePacket.getAddress().toString().
                        split("\\.");
58
59                   int address = -1;
60                   address = Integer.parseInt(ip[ip.length-1]);
61                   parent.addMessage(address,result);
62              } catch (IOException e) {
63
64              }
65          }
66      }
67
68      private class UdpBroadcastReceiver implements Runnable{
69          private DatagramSocket brodcastDatagramSocket;
70          private volatile boolean keepBroadcasting = true;
71          private Thread udpBroadcastReceiverThread;
72
73          public UdpBroadcastReceiver( int receiverPort) throws
                SocketException, BindException{
74              brodcastDatagramSocket = new DatagramSocket(receiverPort
                    +1);
75          }
76
77          public void startBroadcastReceiverthread(){
78              keepBroadcasting = true;
79              udpBroadcastReceiverThread = new Thread(this);
80              udpBroadcastReceiverThread.start();
81          }
```

```
82
83          private void stopBroadcastThread(){
84              keepBroadcasting = false;
85              udpBroadcastReceiverThread.interrupt();
86          }
87
88          public void run(){
89              while(keepBroadcasting){
90                  try {
91                      // 52kb buffer
92                      byte[] buffer = new byte[52000];
93                      DatagramPacket brodcastReceivePacket = new
                            DatagramPacket(buffer,buffer.length);
94
95                      brodcastDatagramSocket.receive(brodcastReceivePacket
                            );
96
97                       byte[] result = new byte[brodcastReceivePacket.
                            getLength()];
98                       System.arraycopy(brodcastReceivePacket.getData(),
                            0, result, 0, brodcastReceivePacket.getLength()
                            );
99
100                      String[] ip = brodcastReceivePacket.getAddress().
                            toString().split("\\.");
101
102                      int address = -1;
103                      address = Integer.parseInt(ip[ip.length-1]);
104                      parent.addMessage(address,result);
105                 } catch (IOException e) {
106
107                 }
108             }
109         }
110     }
111
112 }
```

E.4 Pdu

E.4.0.20 Packet.java

```
1  package adhoc.aodv.pdu;
2
3  import adhoc.aodv.exception.BadPduFormatException;
4
5  public interface Packet {
6
7     public byte[] toBytes();
8
9     public String toString();
10
11    public void parseBytes(byte[] rawPdu) throws
          BadPduFormatException;
12
13    public int getDestinationAddress();
14 }
```

E.4.0.21 HelloPacket.java

```
1  package adhoc.aodv.pdu;
2
3  import adhoc.aodv.Constants;
4  import adhoc.aodv.exception.BadPduFormatException;
5
6  public class HelloPacket implements Packet{
7     private byte pduType;
8     private int sourceAddress;
9     private int sourceSeqNr;
10
11    public HelloPacket(){
12
13    }
14
15    public HelloPacket(int sourceAddress, int sourceSeqNr){
16       pduType = Constants.HELLO_PDU;
17       this.sourceAddress = sourceAddress;
18       this.sourceSeqNr = sourceSeqNr;
19    }
20
21    public int getSourceAddress(){
22       return sourceAddress;
23    }
24
25    @Override
26    public int getDestinationAddress() {
27       //broadcast address
```

```
28          return Constants.BROADCAST_ADDRESS;
29      }
30
31      public int getSourceSeqNr(){
32          return sourceSeqNr;
33      }
34
35      @Override
36      public byte[] toBytes() {
37          return toString().getBytes();
38      }
39
40      @Override
41      public String toString(){
42          return pduType+";"+sourceAddress+";"+sourceSeqNr;
43      }
44
45      @Override
46      public void parseBytes(byte[] rawPdu) throws
            BadPduFormatException {
47          String[] s = new String(rawPdu).split(";",3);
48          if(s.length != 3){
49              throw new BadPduFormatException( "HelloPacket: could not
                    split " +
50                                      "the expected # of arguments
                                            from rawPdu. " +
51                                      "Expecteded 3 args but were
                                            given "+s.length );
52          }
53          try {
54              pduType = Byte.parseByte(s[0]);
55              if(pduType != Constants.HELLO_PDU){
56                  throw new BadPduFormatException( "HelloPacket: pdu type
                        did not match. " +
57                                          "Was expecting: "+Constants.
                                                HELLO_PDU+
58                                          " but parsed: "+pduType );
59              }
60              sourceAddress = Integer.parseInt(s[1]);
61              sourceSeqNr = Integer.parseInt(s[2]);
62          } catch (NumberFormatException e) {
63              throw new BadPduFormatException("HelloPacket: falied in
                    parsing arguments to the desired types");
64          }
65      }
66
67  }
```

E.4.0.22 UserDataPacket.java

```
1  package adhoc.aodv.pdu;
```

```
2
3  import adhoc.aodv.Constants;
4  import adhoc.aodv.exception.BadPduFormatException;
5
6  public class UserDataPacket implements Packet{
7     private byte[] data;
8     private int destAddress;
9     private byte pduType;
10    private int sourceAddress;
11    private int packetID;
12
13    public UserDataPacket(){
14
15    }
16
17    public UserDataPacket(int packetIdentifier,int
          destinationAddress, byte[] data, int sourceAddress){
18       pduType = Constants.USER_DATA_PACKET_PDU;
19       packetID = packetIdentifier;
20       destAddress = destinationAddress;
21       this.data = data;
22       this.sourceAddress = sourceAddress;
23    }
24
25    public byte[] getData(){
26       return data;
27    }
28
29    public int getSourceNodeAddress(){
30       return sourceAddress;
31    }
32
33    @Override
34     public int getDestinationAddress() {
35         return destAddress;
36     }
37
38    @Override
39    public byte[] toBytes() {
40       return toString().getBytes();
41    }
42
43    @Override
44    public String toString(){
45       return pduType+";"+sourceAddress+";"+destAddress+";"+new
             String(data);
46    }
47
48    @Override
49    public void parseBytes(byte[] rawPdu) throws
          BadPduFormatException {
50       String[] s = new String(rawPdu).split(";",4);
51       if(s.length != 4){
52          throw new BadPduFormatException( "UserDataPacket: could
               not split " +
```

```
53                                      "the expected # of arguments
                                            from rawPdu. " +
54                                      "Expecteded 4 args but were
                                            given "+s.length );
55          }
56          try {
57             pduType = Byte.parseByte(s[0]);
58             if(pduType != Constants.USER_DATA_PACKET_PDU){
59                throw new BadPduFormatException( "UserDataPacket: pdu
                       type did not match. " +
60                                         "Was expecting: "+Constants.
                                               USER_DATA_PACKET_PDU+
61                                         " but parsed: "+pduType );
62             }
63             sourceAddress = Integer.parseInt(s[1]);
64             destAddress = Integer.parseInt(s[2]);
65             data = s[3].getBytes();
66          } catch (NumberFormatException e) {
67             throw new BadPduFormatException( "UserDataPacket: falied
                       in parsing " +
68                                      "arguments to the desired types
                                            " );
69          }
70       }
71
72       public int getPacketID() {
73          return packetID;
74       }
75
76    }
```

E.4.0.23 AodvPDU.java

```
1  package adhoc.aodv.pdu;
2
3
4
5  public abstract class AodvPDU implements Packet{
6     protected byte pduType;
7      protected int srcAddress, destAddress;
8      protected int destSeqNum;
9
10
11     public AodvPDU(){
12
13     }
14
15     public AodvPDU(int sourceAddress, int destinationAddess, int
           destinationSequenceNumber){
16       srcAddress = sourceAddress;
17       destAddress = destinationAddess;
```

```
18        destSeqNum = destinationSequenceNumber;
19      }
20
21      public int getSourceAddress() {
22          return srcAddress;
23      }
24
25      @Override
26      public int getDestinationAddress() {
27          return destAddress;
28      }
29
30      public int getDestinationSequenceNumber() {
31          return destSeqNum;
32      }
33
34      public byte getType(){
35        return pduType;
36      }
37
38      @Override
39      public String toString(){
40        return Byte.toString(pduType)+";"+srcAddress+";"+destAddress+
              ";"+destSeqNum+";";
41      }
42  }
```

E.4.0.24 RREQ.java

```
1   package adhoc.aodv.pdu;
2
3   import adhoc.aodv.Constants;
4   import adhoc.aodv.exception.BadPduFormatException;
5
6
7   public class RREQ extends AodvPDU {
8       private int srcSeqNum;
9       private int hopCount = 0;
10      private int broadcastID;
11
12      public RREQ(){
13
14      }
15
16      /**
17       * Constructor for creating a route request PDU
18       * @param sourceNodeAddress the originators node address
19       * @param destinationNodeAddress the address of the desired
             node
20       * @param sourceSequenceNumber originators sequence number
```

```
21      * @param destinationSequenceNumber should be set to the last
            known sequence number of the destination
22      * @param broadcastId along with the source address this number
            uniquely identifies this route request PDU
23      */
24     public RREQ(int sourceNodeAddress, int destinationNodeAddress,
           int sourceSequenceNumber, int destinationSequenceNumber,
           int broadcastId) {
25       super(sourceNodeAddress, destinationNodeAddress,
             destinationSequenceNumber);
26       pduType = Constants.RREQ_PDU;
27         srcSeqNum = sourceSequenceNumber;
28         this.broadcastID = broadcastId;
29     }
30
31    public int getBroadcastId(){
32       return broadcastID;
33    }
34
35    public int getSourceSequenceNumber(){
36       return srcSeqNum;
37    }
38
39    public void setDestSeqNum(int destinationSequenceNumber){
40       destSeqNum = destinationSequenceNumber;
41    }
42
43    public int getHopCount(){
44       return hopCount;
45    }
46
47    public void incrementHopCount(){
48       hopCount++;
49    }
50
51    @Override
52    public byte[] toBytes() {
53       return this.toString().getBytes();
54    }
55
56    @Override
57    public String toString(){
58       return super.toString()+srcSeqNum+";"+hopCount+";"+
             broadcastID;
59    }
60
61    @Override
62    public void parseBytes(byte[] rawPdu) throws
         BadPduFormatException {
63       String[] s = new String(rawPdu).split(";",7);
64       if(s.length != 7){
65          throw new BadPduFormatException( "RREQ: could not split "
               +
66                                      "the expected # of arguments
                                           from rawPdu. " +
```

```
67                                          "Expecteded 7 args but were
                                                given "+s.length );
68          }
69          try {
70             pduType = Byte.parseByte(s[0]);
71             if(pduType != Constants.RREQ_PDU){
72                throw new BadPduFormatException( "RREQ: pdu type did
                       not match. " +
73                                               "Was expecting: "+Constants.
                                                   RREQ_PDU+
74                                               " but parsed: "+pduType );
75             }
76             srcAddress = Integer.parseInt(s[1]);
77             destAddress = Integer.parseInt(s[2]);
78             destSeqNum =Integer.parseInt(s[3]);
79             srcSeqNum = Integer.parseInt(s[4]);
80             hopCount = Integer.parseInt(s[5]);
81             broadcastID = Integer.parseInt(s[6]);
82          } catch (NumberFormatException e) {
83             throw new BadPduFormatException("RREQ: falied in parsing
                    arguments to the desired types");
84          }
85       }
86    }
```

E.4.0.25 RREP.java

```
1  package adhoc.aodv.pdu;
2
3  import adhoc.aodv.Constants;
4  import adhoc.aodv.exception.BadPduFormatException;
5
6  public class RREP extends AodvPDU {
7      private int hopCount = 0;
8      private int srcSeqNum;
9
10
11     public RREP(){
12     }
13
14     public RREP(  int sourceAddress,
15                   int destinationAddress,
16                   int sourceSequenceNumber,
17                   int destinationSequenceNumber,
18                   int hopCount){
19
20       super(sourceAddress,destinationAddress,
             destinationSequenceNumber);
21       pduType = Constants.RREP_PDU;
22       srcSeqNum = sourceSequenceNumber;
23       this.hopCount = hopCount;
```

```
24      }
25
26      public RREP(  int sourceAddress,
27                    int destinationAddress,
28                    int sourceSequenceNumber,
29                    int destinationSequenceNumber) {
30
31        super(sourceAddress,destinationAddress,
              destinationSequenceNumber);
32        pduType = Constants.RREP_PDU;
33        srcSeqNum = sourceSequenceNumber;
34      }
35
36     public int getHopCount(){
37        return hopCount;
38     }
39
40     public void incrementHopCount(){
41        hopCount++;
42     }
43
44     public int getDestinationSequenceNumber(){
45        return destSeqNum;
46     }
47
48     @Override
49     public byte[] toBytes() {
50        return this.toString().getBytes();
51     }
52
53     @Override
54     public String toString() {
55        return super.toString()+srcSeqNum+";"+hopCount;
56     }
57
58     @Override
59     public void parseBytes(byte[] rawPdu) throws
           BadPduFormatException {
60        String[] s = new String(rawPdu).split(";",6);
61        if(s.length != 6){
62           throw new BadPduFormatException( "RREP: could not split "
                 +
63                                       "the expected # of arguments
                                           from rawPdu. " +
64                                       "Expecteded 6 args but were
                                           given "+s.length );
65        }
66        try {
67           pduType = Byte.parseByte(s[0]);
68           if(pduType != Constants.RREP_PDU){
69              throw new BadPduFormatException( "RREP: pdu type did
                    not match. " +
70                                          "Was expecting: "+Constants.
                                              RREP_PDU+
71                                          " but parsed: "+pduType );
```

```
72              }
73              srcAddress = Integer.parseInt(s[1]);
74              destAddress = Integer.parseInt(s[2]);
75              destSeqNum =Integer.parseInt(s[3]);
76              srcSeqNum = Integer.parseInt(s[4]);
77              hopCount = Integer.parseInt(s[5]);
78          } catch (NumberFormatException e) {
79              throw new BadPduFormatException("RREP: falied in parsing
                    arguments to the desired types");
80          }
81
82      }
83  }
```

E.4.0.26 RERR.java

```
1   package adhoc.aodv.pdu;
2
3   import java.util.ArrayList;
4
5   import adhoc.aodv.Constants;
6   import adhoc.aodv.exception.BadPduFormatException;
7
8   public class RERR extends AodvPDU {
9       private int unreachableNodeAddress;
10      private int unreachableNodeSequenceNumber;
11      private ArrayList<Integer> destAddresses = new ArrayList<Integer
            >();
12
13
14
15      public RERR(){
16
17      }
18
19      /**
20       *
21       * @param unreachableNodeAddress
22       * @param unreachableNodeSequenceNumber
23       * @param destinationAddresses
24       */
25      public RERR(int unreachableNodeAddress ,int
            unreachableNodeSequenceNumber, ArrayList<Integer>
            destinationAddresses) {
26          this.unreachableNodeAddress = unreachableNodeAddress;
27          this.unreachableNodeSequenceNumber =
                unreachableNodeSequenceNumber;
28          pduType = Constants.RERR_PDU;
29          destAddresses = destinationAddresses;
30          destAddress = -1;
31      }
```

```
32
33      /**
34       * Constructor of a route error message
35       * @param
36       * @param
37       * @param destinationAddress the node which hopefully will
             receive this PDU packet
38       */
39       public RERR(int unreachableNodeAddress ,int
             unreachableNodeSequenceNumber, int destinationAddress){
40         this.unreachableNodeAddress = unreachableNodeAddress;
41         this.unreachableNodeSequenceNumber =
               unreachableNodeSequenceNumber;
42         pduType = Constants.RERR_PDU;
43           destAddress = destinationAddress;
44       }
45
46      public int getUnreachableNodeAddress(){
47         return unreachableNodeAddress;
48      }
49
50      public int getUnreachableNodeSequenceNumber(){
51         return unreachableNodeSequenceNumber;
52      }
53
54      public ArrayList<Integer> getAllDestAddresses(){
55         return destAddresses;
56      }
57
58      @Override
59      public byte[] toBytes() {
60         return this.toString().getBytes();
61      }
62
63      @Override
64      public String toString() {
65         return Byte.toString(pduType)+";"+unreachableNodeAddress+";"+
               unreachableNodeSequenceNumber;
66      }
67
68      @Override
69      public void parseBytes(byte[] rawPdu) throws
            BadPduFormatException {
70         String[] s = new String(rawPdu).split(";",3);
71         if(s.length != 3){
72            throw new BadPduFormatException( "RERR: could not split "
                  +
73                                        "the expected # of arguments
                                             from rawPdu. " +
74                                        "Expecteded 3 args but were
                                             given "+s.length );
75         }
76         try {
77            pduType = Byte.parseByte(s[0]);
78            if(pduType != Constants.RERR_PDU){
```

```
79              throw new BadPduFormatException( "RERR: pdu type did
                    not match. " +
80                                      "Was expecting: "+Constants.
                                            RERR_PDU+
81                                      " but parsed: "+pduType );
82          }
83          unreachableNodeAddress = Integer.parseInt(s[1]);
84          unreachableNodeSequenceNumber = Integer.parseInt(s[2]);
85       } catch (NumberFormatException e) {
86          throw new BadPduFormatException("RERR: falied in parsing
                arguments to the desired types");
87       }
88    }
89 }
```

E.4.0.27 InternalMessage.java

```
1  package adhoc.aodv.pdu;
2
3  import adhoc.aodv.exception.BadPduFormatException;
4  import adhoc.etc.Debug;
5
6  public class InternalMessage extends AodvPDU{
7
8
9     public InternalMessage(byte pduType, int destinationAddress){
10       this.pduType = pduType;
11       this.destAddress = destinationAddress;
12    }
13
14
15    @Override
16    public void parseBytes(byte[] rawPdu) throws
          BadPduFormatException {
17       Debug.print("DO NOT USE");
18
19    }
20
21    @Override
22    public byte[] toBytes() {
23       Debug.print("DO NOT USE");
24       return null;
25    }
26
27 }
```

E.5 Setup

E.5.0.28 AdhocManager.java

```
1  package adhoc.setup;
2
3  import android.app.Activity;
4  import android.net.wifi.WifiManager;
5
6  public class AdhocManager {
7     Activity parent;
8     private WifiManager wifiManager;
9     private String ip = "192.168.2.";
10
11    public AdhocManager(Activity parent, WifiManager wifiManager){
12       this.parent = parent;
13       this.wifiManager = wifiManager;
14    }
15
16 // public boolean startAdhocNetwork(int phoneType, int nodeAddress)
       {
17 //    disableWifi();
18 //    ip = ip+Integer.toString(nodeAddress);
19 //    int result = NativeCommand.runCommand("su -c \""+"
       startstopadhoc start "+phoneType+" "+ip+"\"");
20 //
21 //    return (result == 0 ? true:false);
22 // }
23 //
24 // public boolean stopAdhocNetwork(int phoneType){
25 //    int result = NativeCommand.runCommand("su -c \""+"
       startstopadhoc start "+phoneType+" "+ip+"\"");
26 //    return (result == 0 ? true:false);
27 // }
28
29
30
31    private void disableWifi() {
32          if (wifiManager.isWifiEnabled()){
33             wifiManager.setWifiEnabled(false);
34          }
35       }
36 }
```

E.5.0.29 startstopadhoc.c included in adhocsetup.so

```
1  #include <string.h>
2  #include <stdio.h>
3  #include <stdlib.h>
```

```
4  #include <sys/syscall.h>
5  #include <unistd.h>
6  #include <fcntl.h>
7
8  int file_exists(const char* fileName) {
9     FILE *file = NULL;
10    if (!(file = fopen(fileName, "r"))) {
11       return -1;
12    }
13    return 0;
14 }
15
16 int rmmod(const char *modname) {
17    return syscall(__NR_delete_module, modname, O_NONBLOCK | O_EXCL)
          ;
18 }
19
20 int startwifi(const int* phoneType, const char* ip[]) {
21
22    switch (*phoneType) {
23    case 0: //NEXSUS
24       system("insmod /system/lib/modules/bcm4329.ko");
25       system("ifconfig eth0 %s netmask 255.255.255.0", *ip);
26       system("ifconfig eth0 up");
27       system("iwconfig eth0 mode ad-hoc");
28       system("iwconfig eth0 essid nexusbac");
29       system("iwconfig eth0 channel 6");
30       system("iwconfig eth0 commit");
31       break;
32
33    case 1: //HERO
34       system("insmod /system/lib/modules/wlan.ko");
35       system("wlan_loader -f /system/etc/wifi/Fw1251r1c.bin -e /
             proc/calibration -i /data/local/bin/tiwlan.ini");
36       system("ifconfig rmnet0 %s netmask 255.255.255.0", *ip);
37       system("ifconfig rmnet0 up");
38       break;
39
40    default:
41       return -1;
42    }
43    return 0;
44 }
45
46 int stopwifi(const int* phoneType) {
47
48    switch (*phoneType) {
49    case 0: //NEXUS
50       system("ifconfig eth0 down");
51       system("killall iwconfig");
52       system("rmmod bcm4329");
53       break;
54    case 1: //HERO
55       system("ifconfig rmnet0 down");
56
```

```
57
58          rmmod("wlan");
59          //system("rmmod wlan");
60          break;
61      default:
62          return -1;
63      }
64      return 0;
65  }
66
67  int main(int argc, char *argv[]) {
68
69      if (argc != 3 && strcmp(argv[1], "start")) {
70          return -1;
71      } else if (argc !=2 && strcmp(argv[1], "stop"))
72      {
73          return -1;
74      }
75
76      int phoneType = atoi(argv[2]);
77      char *ip[] = argv[3];
78      //printf("phoneType as char: %s \nphoneType as int: %d \
            nphoneType as address %d",argv[2],phoneType,&phoneType);
79      //printf("ip: %s",ip);
80
81      if (strcmp(argv[1], "start") == 0) {
82          return startwifi(&phoneType, &ip[]);
83      } else if (strcmp(argv[1], "stop") == 0) {
84          return stopwifi(&phoneType);
85      }
86      return -1;
87  }
```

E.5.0.30 native_task.c included in adhocsetup.so

```
1  #include <jni.h>
2  #include <string.h>
3
4
5  JNIEXPORT jint JNICALL  Java_bachelor_TextMsg_TextMsg_runCommand
6  (JNIEnv *env, jclass class, jstring command)
7  {
8      const char *commandString;
9      commandString = (*env)->GetStringUTFChars(env, command, 0);
10     int exitcode = system(commandString);
11     (*env)->ReleaseStringUTFChars(env, command, commandString);
12     return (jint)exitcode;
13 }
```

E.5.0.31 NativeCommand.java

```
1  package adhoc.setup;
2
3  public class NativeCommand {
4
5      public static native int runCommand(String command);
6
7      static {
8        System.loadLibrary("adhocsetup");
9     }
10
11
12 }
```

E.5.0.32 PhoneType.java

```
1  package adhoc.setup;
2
3  public interface PhoneType {
4
5     // Class defining all supported Android phones
6     public static final int NEXUS_ONE = 0;
7     public static final int HTC_HERO = 1;
8     public static final int HTC_DREAM = 1;
9
10 }
```

E.6 Exception

E.6.0.33 AodvException.java

```
1  package adhoc.aodv.exception;
2
3  public abstract class AodvException extends Exception{
4
5     /**
6      *
7      */
8     private static final long serialVersionUID = 1L;
9
10    public AodvException(){}
11
12    public AodvException(String message) {
13       super(message);
14    }
15
16 }
```

E.6.0.34 BadPduFormatException.java

```
1  package adhoc.aodv.exception;
2
3  public class BadPduFormatException extends AodvException {
4
5     /**
6      *
7      */
8     private static final long serialVersionUID = 1L;
9
10    public BadPduFormatException(){
11
12    }
13
14    public BadPduFormatException(String message){
15       super(message);
16    }
17
18 }
```

E.6.0.35 DataExceedsMaxSizeException.java

```
1  package adhoc.aodv.exception;
```

```
2
3  public class DataExceedsMaxSizeException extends AodvException {
4
5      /**
6       *
7       */
8      private static final long serialVersionUID = 1L;
9
10     public DataExceedsMaxSizeException(){
11
12     }
13
14     public DataExceedsMaxSizeException(String message){
15        super(message);
16     }
17
18 }
```

E.6.0.36 InvalidNodeAddressException.java

```
1  package adhoc.aodv.exception;
2
3  public class InvalidNodeAddressException extends AodvException{
4
5      /**
6       *
7       */
8      private static final long serialVersionUID = 1L;
9
10     public InvalidNodeAddressException() {
11
12     }
13
14     public InvalidNodeAddressException(String message){
15        super(message);
16     }
17 }
```

E.6.0.37 NoSuchRouteException.java

```
1  package adhoc.aodv.exception;
2
3  public class NoSuchRouteException extends AodvException{
4
5      /**
6       *
7       */
```

```
8     private static final long serialVersionUID = 1L;
9
10    public NoSuchRouteException(){
11
12    }
13
14    public NoSuchRouteException(String message) {
15       super(message);
16    }
17
18
19 }
```

E.6.0.38 RouteNotValidException.java

```
1  package adhoc.aodv.exception;
2
3  public class RouteNotValidException extends AodvException{
4
5     /**
6      *
7      */
8     private static final long serialVersionUID = 1L;
9
10    public RouteNotValidException(){
11
12    }
13
14    public RouteNotValidException(String message) {
15       super(message);
16    }
17 }
```

E.7 Etc

E.7.0.39 Debug.java

```
1  package adhoc.etc;
2
3  import java.io.PrintStream;
4
5  public class Debug {
6     static private PrintStream debugStream = null;
7
8     static public void setDebugStream( PrintStream printstream){
9        debugStream = printstream;
10    }
11
12    static public void print(String s){
13       if(debugStream != null){
14          debugStream.println(s);
15       }
16    }
17 }
```

Appendix F

Text Messenger Source Code

The following source code is structured such that each class can be seen depending of which package it is from.

F.1 Model

F.1.0.40 AODVObserver.java

```
1  package android.TextMessenger.model;
2
3  import java.util.Observable;
4  import java.util.Observer;
5
6  import adhoc.aodv.Node;
7  import adhoc.aodv.ObserverConst;
8  import adhoc.aodv.Node.MessageToObserver;
9  import adhoc.aodv.Node.PacketToObserver;
10 import adhoc.aodv.exception.BadPduFormatException;
11 import android.TextMessenger.model.pdu.ChatRequest;
12 import android.TextMessenger.model.pdu.Hello;
13 import android.TextMessenger.model.pdu.Msg;
14 import android.TextMessenger.model.pdu.NoSuchChat;
15
```

```
16 public class AODVObserver implements Observer {
17    private Timer timer;
18    private ContactManager contactManager;
19    private ChatManager chatManager;
20
21    public AODVObserver(Node node, String myDisplayName, int
          myContactID, Timer time, ContactManager contactManager ,
          ChatManager chatManager) {
22       this.chatManager = chatManager;
23       this.timer = timer;
24       this.contactManager = contactManager;
25       node.addObserver(this);
26    }
27
28    @Override
29    public void update(Observable o, Object arg) {
30       MessageToObserver msg = (MessageToObserver)arg;
31       int userPacketID, destination, type = msg.getMessageType();
32       switch (type) {
33       case ObserverConst.ROUTE_ESTABLISHMENT_FAILURE:
34          int unreachableDestinationAddrerss  = (Integer)msg.
               getContainedData();
35          contactManager.routeEstablishmentFailurRecived(
               unreachableDestinationAddrerss);
36          break;
37       case ObserverConst.DATA_RECEIVED:
38          parseMessage(  (Integer)((PacketToObserver)msg).
               getSenderNodeAddress(),
39                      (byte[])msg.getContainedData()   );
40          break;
41       case ObserverConst.INVALID_DESTINATION_ADDRESS:
42          userPacketID = (Integer)msg.getContainedData();
43          //FIXME slet fra timer og Contacts
44          break;
45       case ObserverConst.DATA_SIZE_EXCEEDES_MAX:
46          userPacketID = (Integer)msg.getContainedData();
47          //FIXME slet fra timer
48          break;
49       case ObserverConst.ROUTE_INVALID:
50          destination  = (Integer)msg.getContainedData();
51          contactManager.routeInvalidRecived(destination);
52          break;
53       case ObserverConst.ROUTE_CREATED:
54          destination = (Integer)msg.getContainedData();
55          contactManager.routeEstablishedRecived(destination);
56          break;
57       default:
58          break;
59       }
60    }
61
62    private void parseMessage(int senderID, byte[] data){
63       String[] split = new String(data).split(";",2);
64       try {
65          int type = Integer.parseInt(split[0]);
```

```
66            switch (type) {
67
68            case Constants.PDU_MSG:
69               Msg msg = new Msg(senderID);
70               msg.parseBytes(data);
71               chatManager.textReceived(msg, senderID);
72               break;
73            case Constants.PDU_ACK:
74               timer.removePDU(Integer.parseInt(split[1]));
75               break;
76            case Constants.PDU_CHAT_REQUEST:
77               ChatRequest chatReq = new ChatRequest();
78               chatReq.parseBytes(data);
79               chatManager.chatRequestReceived(chatReq,senderID);
80               break;
81            case Constants.PDU_HELLO:
82               Hello hello = new Hello();
83               hello.parseBytes(data);
84               contactManager.helloRecived(hello, senderID);
85               break;
86            case Constants.PDU_NO_SUCH_CHAT:
87               NoSuchChat noSuchChat = new NoSuchChat();
88               noSuchChat.parseBytes(data);
89               chatManager.noSuchChatRecived(noSuchChat,senderID);
90               break;
91            default:
92               break;
93            }
94         } catch (NumberFormatException e) {
95            //discard the message.
96         } catch (BadPduFormatException e) {
97            //discard the message
98            // Message is in the domain of invalid messages
99         }
100      }
101
102   }
```

F.1.0.41 Chat.java

```
1  package android.TextMessenger.model;
2
3  import java.util.ArrayList;
4  import java.util.HashMap;
5  import java.util.Observable;
6
7  import android.TextMessenger.model.pdu.Msg;
8  import android.TextMessenger.view.ObserverConst;
9
10 public class Chat extends Observable {
11    private HashMap<Integer, String> contacts;
```

```
12      private ArrayList<Msg> messages;
13      private boolean newMsg, active;
14      private int chatID, myContactID;
15      private Integer messageNum;
16      private String myDisplayName;
17      private ArrayList<Msg> earlyMessages = new ArrayList<Msg>();
18
19      public Chat(HashMap<Integer, String> contacts, int chatID, int
            myContactID, String myDisplayName) {
20          this.chatID = chatID;
21          this.contacts = contacts;
22          this.myContactID = myContactID;
23          this.myDisplayName = myDisplayName;
24          messages = new ArrayList<Msg>();
25          newMsg = false;
26          messageNum = 0;
27          active = true;
28      }
29
30      public HashMap<Integer, String> getContacts() {
31          return new HashMap<Integer, String>(contacts);
32      }
33
34      private boolean addMsgToBuffer(Msg msg) {
35          // adds the message to a sorted buffer list
36          for (int x = 0; x < earlyMessages.size(); x++) {
37              if (earlyMessages.get(x).getContactID() == msg.
                    getContactID()) {
38
39                  if (earlyMessages.get(x).getMssageNumber() == msg.
                        getMssageNumber()) {
40                      return false;
41                  }
42
43                  else if (earlyMessages.get(x).getMssageNumber() > msg.
                        getMssageNumber()) {
44                      earlyMessages.add(x, msg);
45                      return false;
46                  }
47              }
48          }
49          earlyMessages.add(msg);
50          return false;
51      }
52
53      public void addBufferdMsg(Msg msg) {
54          int lastMessageNumber = msg.getMssageNumber();
55          ArrayList<Msg> removedMSG = new ArrayList<Msg>();
56          for (Msg emsg : earlyMessages) {
57              if (msg.getContactID() == emsg.getContactID() &&
                    lastMessageNumber == (emsg.getMssageNumber() - 1)) {
58                  messages.add(emsg);
59                  notTextToObserver(emsg);
60                  removedMSG.add(emsg);
61                  lastMessageNumber = emsg.getMssageNumber();
```

```
62              } else {
63                  break;
64              }
65          }
66          for (Msg emsg : removedMSG) {
67              earlyMessages.remove(emsg);
68          }
69      }
70
71      public boolean addMsg(Msg msg) {
72          synchronized (messages) {
73              Msg iMsg;
74              for (int i = messages.size(); i > 0; i--) {
75                  iMsg = messages.get(i - 1);
76                  if (iMsg.getContactID() == msg.getContactID()) {
77                      //
78                      if (iMsg.getMssageNumber() == (msg.getMssageNumber()
                            - 1)) {
79                          messages.add(msg);
80                          newMsg = true;
81                          addBufferdMsg(msg);
82                          notTextToObserver(msg);
83                          return true;
84                      }
85
86                      else if (iMsg.getMssageNumber() < (msg.
                            getMssageNumber() - 1)) {
87                          return addMsgToBuffer(msg);
88                      } else if (iMsg.getMssageNumber() == msg.
                            getMssageNumber()) {
89                          return false;
90                      }
91                  }
92              }
93              if (msg.getMssageNumber() != 1) {
94                  return addMsgToBuffer(msg);
95              }
96              messages.add(msg);
97              newMsg = true;
98              // Adds all the msg from earlyMessages that can be added
99              addBufferdMsg(msg);
100
101         }
102         // Notifying the observer
103         notTextToObserver(msg);
104         return true;
105     }
106
107     private void notTextToObserver(Msg msg) {
108         String message;
109         if (msg.getContactID() == myContactID) {
110             message = (myDisplayName + " writes at " + msg.getTime() +
                        ":\n");
111         } else {
```

```
112             message = (contacts.get(msg.getContactID()) + " writes at
                    " + msg.getTime() + ":\n");
113         }
114         message += (msg.getText() + "\n\n");
115         setChanged();
116         notifyObservers(new ObjToObsever(message, ObserverConst.
                TEXT_RECIVED));
117     }
118 
119     /**
120      * Sets the hasBeanViewed when the Chat is displayed or closed.
121      *
122      * @param newMsg
123      */
124     public void setHaveBeenViewde() {
125         newMsg = false;
126     }
127 
128     public boolean isTherNewMsg() {
129         return newMsg;
130     }
131 
132     public void getTextHistory() {
133         synchronized (messages) {
134             for (Msg msg : messages) {
135                 notTextToObserver(msg);
136             }
137         }
138 
139     }
140 
141     public String getDisplayname(int contactID) {
142         if (contactID == myContactID) {
143             return myDisplayName;
144         } else {
145             return contacts.get(contactID);
146         }
147     }
148 
149     public void disableChat() {
150         active = false;
151         setChanged();
152         notifyObservers(new ObjToObsever(null, ObserverConst.
                REMOVE_CHAT));
153     }
154 
155     public int getNextMessageNum() {
156         synchronized (messageNum) {
157             messageNum++;
158             return messageNum;
159         }
160     }
161 
162     public int getID() {
163         return chatID;
```

```
164     }
165
166     public void notifyTextNotSent(Msg msg) {
167         setChanged();
168         notifyObservers(new ObjToObsever(msg, ObserverConst.
                TEXT_NOT_SENT));
169     }
170 }
```

F.1.0.42 ChatManager.java

```
1  package android.TextMessenger.model;
2
3  import java.util.ArrayList;
4  import java.util.HashMap;
5  import java.util.Observable;
6
7  import adhoc.aodv.Node;
8  import android.TextMessenger.exceptions.ContactOfflineException;
9  import android.TextMessenger.model.pdu.Ack;
10 import android.TextMessenger.model.pdu.ChatRequest;
11 import android.TextMessenger.model.pdu.Msg;
12 import android.TextMessenger.model.pdu.NoSuchChat;
13 import android.TextMessenger.view.ObserverConst;
14
15 public class ChatManager extends Observable {
16     private HashMap<Integer, Chat> chats;
17     private Sender sender;
18     private ContactManager contactManager;
19     private int myContactID;
20     private String myDisplayName;
21
22     public ChatManager(String myDisplayName, int myContactID, Node
           node) {
23         this.myContactID = myContactID;
24         this.myDisplayName = myDisplayName;
25         chats = new HashMap<Integer, Chat>();
26         contactManager = new ContactManager(myDisplayName,
               myContactID, this, node);
27         sender = contactManager.getSender();
28
29         ClassConstants classConstant = ClassConstants.getInstance();
30         classConstant.setContactManager(contactManager);
31         classConstant.setChatManager(this);
32     }
33
34     public void sendText(String text, int chatID) throws
           ContactOfflineException {
35         Chat chat = chats.get(chatID);
36         if (chat != null) {
37             //adds the messages to its own chat
```

```
38          Msg msg = new Msg(chat.getNextMessageNum(), myContactID,
                chatID, text);
39          chat.addMsg(msg);
40          //sends the message to all other contacts in the chat
41          for (int contact : chat.getContacts().keySet()) {
42             if (contactManager.isContactOnline(contact)) {
43                sender.sendPDU(msg, contact);
44             } else {
45                //removes the chat
46                removeChat(chatID);
47                throw new ContactOfflineException();
48             }
49          }
50       }
51    }
52
53    public void textReceived(Msg msg, int sourceContact){
54       Chat chat = chats.get(msg.getChatID());
55       Ack ack = new Ack(msg.getSequenceNumber());
56       sender.resendPDU(ack, sourceContact);
57
58       if(chat != null){
59          if(chat.addMsg(msg)){
60             setChanged();
61             notifyObservers(new ObjToObsever(msg.getChatID(),
                   ObserverConst.TEXT_RECIVED));
62          }
63       }
64       else{
65          sender.sendPDU(new NoSuchChat(msg.getChatID()),
                sourceContact);
66       }
67
68    }
69
70    private boolean removeChat(int chatID){
71       Chat c = null;
72       synchronized (chats) {
73          c = chats.remove(chatID);
74       }
75       if( c != null){
76          c.disableChat();
77          setChanged();
78          //Notify
79          notifyObservers(new ObjToObsever(c, ObserverConst.
                REMOVE_CHAT));
80          return true;
81       }
82       return false;
83    }
84
85    public boolean removeChatsWhereContactIsIn(int contactID) {
86       synchronized (chats) {
87          ArrayList<Integer> removeChatID = new ArrayList<Integer>()
                ;
```

```
88          for (Chat c : chats.values()) {
89             if (c.getContacts().containsKey(contactID)) {
90                removeChatID.add(c.getID());
91             }
92          }
93          if (removeChatID.isEmpty()) {
94             return false;
95          } else {
96             for (int i : removeChatID) {
97                removeChat(i);
98             }
99             return true;
100         }
101      }
102   }
103
104   /**FIXME the chats hashmap is in danger for cuncurentsy
105    * when creating a chat from GUI no need to notify because it is
          the one creating the chat
106    * @param contactIDs
107    * @return returns true if a chat were created, and false if the
          chat already exists
108    * @throws Exception If one of the Contacts is offline
109    */
110   public boolean newChat(HashMap<Integer, String> contactIDs) {
111      // Creates a chat ID
112      contactIDs.put(myContactID, myDisplayName);
113      int chatID = createChatID(contactIDs.keySet().toArray());
114
115      Chat chat = new Chat(contactIDs, chatID, myContactID,
             myDisplayName);
116      Object returnResult = null;
117      synchronized (chats) {
118         returnResult = chats.put(chatID, chat);
119      }
120      if (returnResult == null) {
121         contactIDs.put(myContactID, myDisplayName);
122         for (Integer contactID : contactIDs.keySet()) {
123            if(contactID != myContactID){
124            ChatRequest chatRequest = new ChatRequest(chatID,
                   contactIDs);
125            sender.sendPDU(chatRequest, contactID);
126            }
127         }
128         setChanged();
129         notifyObservers(new ObjToObsever(chat, ObserverConst.
               NEW_CHAT));
130         contactIDs.remove(myContactID);
131         return true;
132      }
133      contactIDs.remove(myContactID);
134      return false;
135   }
136
137   public Chat getChat(int ID){
```

```
138         return chats.get(ID);
139     }
140 
141     /**
142      *
143      * @param pdu
144      * @param sourceContact
145      */
146     public void chatRequestReceived(ChatRequest pdu, int
            sourceContact) {
147         Ack ack = new Ack(pdu.getSequenceNumber());
148         sender.resendPDU(ack,sourceContact);
149         HashMap<Integer, String> contacts = pdu.getFriends();
150         contacts.remove(myContactID);
151         Chat chat = new Chat(contacts, pdu.getChatID(), myContactID,
                myDisplayName);
152         Object returnResult = null;
153         synchronized (chats) {
154             returnResult = chats.put(pdu.getChatID(), chat);
155         }
156         if (returnResult == null) {
157             for(int c : contacts.keySet()){
158                 contactManager.addContact(c , pdu.getFriends().get(c),
                        false);
159             }
160             //Notify
161             setChanged();
162             notifyObservers(new ObjToObsever(chat, ObserverConst.
                    NEW_CHAT));
163         }
164 
165     }
166 
167     public void notifyTextNotSent(Msg msg) {
168         chats.get(msg.getChatID()).notifyTextNotSent(msg);
169         setChanged();
170         notifyObservers(new ObjToObsever(msg, ObserverConst.
                TEXT_NOT_SENT));
171     }
172 /**
173  * Creats a unique hash sum from a unique String, with user id's
        sortet with insert sort
174  * @param contactIDs
175  * @return Unique ID for chat with contacts
176  */
177     private int createChatID(Object contactIDs[]) {
178         int n = contactIDs.length;
179         for (int i = 1; i < n; i++) {
180             int value = (Integer) contactIDs[i];
181             int j = i;
182             while ((j > 0) && (((Integer) contactIDs[j - 1]) > value))
                    {
183                 contactIDs[j] = contactIDs[j - 1];
184                 j--;
185             }
```

```
186             contactIDs[j] = value;
187         }
188         String chatIDString = contactIDs[0] + "";
189         for (int i = 1; i < n; i++) {
190             chatIDString += ";" + contactIDs[i];
191         }
192         return chatIDString.hashCode();
193     }
194 
195     public boolean chatExists(int cahtID){
196         return chats.containsKey(cahtID);
197     }
198 
199     public void noSuchChatRecived(NoSuchChat noChat, int
            sourceContact){
200         Ack ack = new Ack(noChat.getSequenceNumber());
201         sender.resendPDU(ack, sourceContact);
202         removeChat(noChat.getChatID());
203     }
204 }
```

F.1.0.43 ClassConstants.java

```
1  package android.TextMessenger.model;
2 
3  public class ClassConstants {
4      private ContactManager contactManager;
5      private ChatManager chatManager;
6      private static final ClassConstants INSTANCE = new
           ClassConstants();
7 
8          // Private constructor prevents instantiation from other
               classes
9          private ClassConstants() {}
10 
11         public static ClassConstants getInstance() {
12             return INSTANCE;
13         }
14 
15         public void setContactManager(ContactManager contactManager){
16             this.contactManager = contactManager;
17         }
18 
19         public void setChatManager(ChatManager chatManager){
20             this.chatManager = chatManager;
21         }
22 
23         public ContactManager getContactManager(){
24             return contactManager;
25         }
26 
```

```
27        public ChatManager getChatmanager(){
28            return chatManager;
29        }
30
31
32    }
```

F.1.0.44 Constants.java

```
1   package android.TextMessenger.model;
2
3   public interface Constants {
4
5       //valid sequence number interval
6       public static final int MIN_VALID_SEQ_NUM = 0;
7       public static final int MAX_VALID_SEQ_NUM = Integer.MAX_VALUE;
8
9       //the time to wait before a sent pdu is timed out
10      public static final int MESSAGE_ALIVE_TIME = 3000;
11
12      //pdu types
13      public static final byte PDU_ACK = 1;
14      public static final byte PDU_CHAT_REQUEST = 2;
15      public static final byte PDU_HELLO = 3;
16      public static final byte PDU_MSG = 4;
17      public static final byte PDU_NO_SUCH_CHAT = 5;
18
19      //Time between hello to Offline
20      public static final int CHECK_TIME = 10000;
21
22  }
```

F.1.0.45 Contact.java

```
1   package android.TextMessenger.model;
2
3   import java.util.Observable;
4
5   public class Contact extends Observable{
6       private int ID;
7       private volatile boolean isOnline = false;
8       private String displayName;
9
10      public Contact(int ID, String displayName){
11      this.ID = ID;
12      //FIXME DET ER SUPER VIGTIGT AT EN displayName IKKE INDEHOLDER
            ";" eller "::"
```

```
13      this.displayName = displayName;
14      }
15
16      public int getID(){
17          return ID;
18      }
19
20      public String getDisplayName(){
21          synchronized (displayName) {
22              return displayName;
23          }
24      }
25
26      public void setIsOnline(boolean isOnline){
27          //TODO notify
28          this.isOnline = isOnline;
29      }
30
31      public boolean isOnline(){
32          return isOnline;
33      }
34
35      public void setDisplayName(String displayName) {
36          //TODO notify
37          synchronized (this.displayName) {
38              this.displayName = displayName;
39          }
40      }
41
42  }
```

F.1.0.46 ContactManager.java

```
1   package android.TextMessenger.model;
2
3   import java.util.HashMap;
4   import java.util.Observable;
5
6   import adhoc.aodv.Node;
7   import android.TextMessenger.model.pdu.Ack;
8   import android.TextMessenger.model.pdu.Hello;
9   import android.TextMessenger.view.ObserverConst;
10
11
12  /**
13   * Responsible for handling all received error messages about
         contacts: new user discovered, link breakage with an active
         user, user goes offline, user goes online..
14   * @author rabie
15   *
16   */
```

```
17 public class ContactManager extends Observable{
18    private HashMap<Integer, Contact> contacts;
19    private Sender sender;
20    private Timer timer;
21    private String myDisplayName;
22    private ChatManager chatManager;
23    private CheckOfflineStatus checkOfflineStatus;
24    private boolean offlineExists;
25    private int mData;
26
27    public ContactManager(String myDisplayName, int myContactID,
          ChatManager chatManager, Node node){
28       this.sender = sender;
29       this.myDisplayName = myDisplayName;
30       this.chatManager = chatManager;
31
32       contacts = new HashMap<Integer, Contact>();
33
34       offlineExists = false;
35       checkOfflineStatus = new CheckOfflineStatus();
36
37       timer = new Timer(node, myDisplayName, myContactID, this,
             chatManager);
38       sender = timer.getSender();
39       checkOfflineStatus.start();
40    }
41
42    public void stopThread(){
43       checkOfflineStatus.stopCheckOfflineStatusThread();
44    }
45
46    public Sender getSender(){
47       return sender;
48    }
49
50    public String[] getContacts(){
51       String[] items = {};
52       for(Contact s : contacts.values()){
53          items[items.length] = s.getDisplayName();
54       }
55       return items;
56    }
57    /**
58     * Creates a new contact or sets an existing contact to online
59     * @param contactID
60     * @param displayName
61     * @return true if new contact was created and false if contact
            already exists
62     */
63    public boolean addContact(int contactID, String displayName,
          boolean sendHello){
64       Contact contact = contacts.get(contactID);
65       if(contact == null){
66          contact = new Contact(contactID, displayName);
67          synchronized (contacts) {
```

```
68              contacts.put(contactID, contact);
69          }
70          if(sendHello){
71              sender.sendPDU(new Hello(myDisplayName, true),
                    contactID);
72              //setContactOnlineStatus(contactID, false);
73          }
74          setChanged();
75          notifyObservers(new ObjToObsever(contact, ObserverConst.
                NEW_CONTACT));
76
77          return true;
78      }
79      if(!sendHello){
80          setContactOnlineStatus(contactID, true);
81      }
82      return false;
83  }
84
85  public void setContactOnlineStatus(int contactID, boolean
        isOnline) {
86      Contact c = contacts.get(contactID);
87      if (c != null) {
88          c.setIsOnline(isOnline);
89          if (!isOnline) {
90              offlineExists = true;
91              synchronized (contacts) {
92                  contacts.notify();
93              }
94          }
95          setChanged();
96          notifyObservers(new ObjToObsever(contactID, ObserverConst.
                CONTACT_ONLINE_STATUS_CHANGED));
97      }
98  }
99
100 //FIXME denne metode bliver ikke brugt af nogen, og hvis den
        skal bruges af GUI saa skal contacten ikke bare saettes til
        offline, men fjernes HELT fra hashmappet
101 public boolean removeContact(int contactID){
102     //setContactOnlineStatus(contactID, false);
103     chatManager.removeChatsWhereContactIsIn(contactID);
104     timer.removeAllPDUForContact(contactID);
105     //TODO Behover maaske ikke notifye observer hvis det kun er
            view der bruger denne funktion
106     setChanged();
107     notifyObservers(new ObjToObsever(contacts.get(contactID),
            ObserverConst.REMOVE_CONTACT));
108     contacts.remove(contactID);
109     return true;
110 }
111
112 public boolean isContactOnline(int contactID) {
113     Contact c = contacts.get(contactID);
114     if(c != null){
```

```
115             return c.isOnline();
116         }
117         return false;
118     }
119 
120     public void routeEstablishmentFailurRecived(int contactID){
121         setContactOnlineStatus(contactID, false);
122         chatManager.removeChatsWhereContactIsIn(contactID);
123         timer.removeAllPDUForContact(contactID);
124     }
125 
126     public void routeInvalidRecived(int contactID){
127         Hello hello = new Hello(myDisplayName, false);
128         sender.sendPDU(hello, contactID);
129     }
130 
131     public void routeEstablishedRecived(int contactID){
132         if(!contacts.containsKey(contactID) || !contacts.get(
                contactID).isOnline()){
133             Hello hello = new Hello(myDisplayName, true);
134             sender.sendPDU(hello, contactID);
135         }
136     }
137 
138     public void helloRecived(Hello hello, int sourceContactID){
139         Ack ack = new Ack(hello.getSequenceNumber());
140         sender.resendPDU(ack, sourceContactID);
141         if(hello.replyThisMessage()){
142             Hello reHello = new Hello(myDisplayName, false);
143             sender.sendPDU(reHello, sourceContactID);
144         }
145         Contact c = contacts.get(sourceContactID);
146         if(c != null){
147             setContactOnlineStatus(sourceContactID, true);
148             c.setDisplayName(hello.getSourceDisplayName());
149         }
150         else{
151             addContact(sourceContactID, hello.getSourceDisplayName(),
                    false);
152         }
153 
154     }
155 
156     public String getContactDisplayName(int contactID){
157         synchronized(contacts){
158             Contact c = contacts.get(contactID);
159             if(c != null)
160             return c.getDisplayName();
161             else return "Contact removed";
162         }
163     }
164 
165     private void helloToOffline(){
166         offlineExists = false;
167         synchronized (contacts) {
```

```
168             for(Contact c : contacts.values()){
169                 if(!c.isOnline()){
170                     Hello hello = new Hello(myDisplayName, true);
171                     sender.sendPDU(hello, c.getID());
172                 }
173             }
174         }
175     }
176 
177     private class CheckOfflineStatus extends Thread{
178         private volatile boolean keepChecking = true;
179         @Override
180         public void run() {
181             while (keepChecking) {
182                 try {
183                     sleep(Constants.CHECK_TIME);
184                     synchronized (contacts) {
185                         while (!offlineExists)
186                             contacts.wait();
187                     }
188                     helloToOffline();
189                 } catch (InterruptedException e) {
190                 }
191             }
192         }
193 
194         public void stopCheckOfflineStatusThread(){
195             keepChecking = false;
196             this.interrupt();
197         }
198 
199     }
200 
201 }
```

F.1.0.47 ObjToObsever.java

```
1  package android.TextMessenger.model;
2 
3 
4 
5     public class ObjToObsever {
6         private Object obj;
7         private int type;
8 
9         public ObjToObsever(Object obj, int msgType) {
10            this.obj = obj;
11            type = msgType;
12        }
13 
14        public Object getContainedData() {
```

```
15          return obj;
16      }
17
18
19      public int getMessageType() {
20          return type;
21      }
22
23  }
```

F.1.0.48 Sender.java

```
1  package android.TextMessenger.model;
2
3  import adhoc.aodv.Node;
4  import android.TextMessenger.model.pdu.PduInterface;
5
6  public class Sender {
7      private Node node;
8      private Timer timer;
9      private int sequenceNumber = Constants.MIN_VALID_SEQ_NUM;
10     public Sender(Node node, Timer timer){
11         this.node = node;
12         this.timer = timer;
13     }
14
15     public void sendPDU(PduInterface pdu, int destinationContactID){
16         pdu.setSequenceNumber(getNextSequenceNumber());
17         if(timer.setTimer(pdu,destinationContactID)){
18             node.sendData(pdu.getSequenceNumber(),
                   destinationContactID, pdu.toBytes());
19         } else {
20             //TWO MESSAGES HAVE THE SAME SEQ NUMBER!
21         }
22
23     }
24
25     /**
26      * Method used if a pdu by some reason didn't receive an ACK
            message. Resends a message with the same sequence number.
27      */
28     public void resendPDU(PduInterface pdu, int destinationContactID
           ){
29         node.sendData(pdu.getSequenceNumber(),
               destinationContactID, pdu.toBytes());
30     }
31
32     private int getNextSequenceNumber(){
33         if(sequenceNumber < Constants.MAX_VALID_SEQ_NUM){
34             return (sequenceNumber++);
35         }
```

```
36          return (sequenceNumber = Constants.MIN_VALID_SEQ_NUM);
37      }
38  }
```

F.1.0.49 Timer.java

```
1   package android.TextMessenger.model;
2
3   import java.util.HashMap;
4   import java.util.LinkedList;
5
6   import adhoc.aodv.Node;
7   import android.TextMessenger.model.pdu.ChatRequest;
8   import android.TextMessenger.model.pdu.Msg;
9   import android.TextMessenger.model.pdu.PduInterface;
10
11  public class Timer extends Thread {
12      private ChatManager chatManager;
13      private ContactManager contactManager;
14      private Sender sender;
15      private volatile boolean keepRunning = true;
16      private HashMap<Integer, Integer> pduIdentifiers = new HashMap<
            Integer, Integer>();
17      private LinkedList<PduInterface> aliveQueue = new LinkedList<
            PduInterface>();
18      //TODO sync??
19      //private final Object timerLock = new Integer(0);
20
21      public Timer(Node node, String myDisplayName, int myContactID,
            ContactManager contactManager, ChatManager chatManager) {
22          this.sender = new Sender(node, this);
23          this.contactManager = contactManager;
24          this.chatManager = chatManager;
25          AODVObserver aodvobs = new AODVObserver(node, myDisplayName,
                myContactID, this, contactManager, chatManager);
26          this.start();
27      }
28
29      public Sender getSender(){
30          return sender;
31      }
32
33      public boolean setTimer(PduInterface pdu, int destContactID) {
34          Integer pduExists;
35          pdu.setTimer();
36
37          synchronized (pduIdentifiers) {
38              pduExists = pduIdentifiers.put(pdu.getSequenceNumber(),
                    destContactID);
39          }
40          if (pduExists != null){
```

```
41          return false;
42       }
43
44       aliveQueue.addLast(pdu);
45       synchronized (aliveQueue) {
46          aliveQueue.notify();
47       }
48
49       return true;
50    }
51
52    public void run(){
53       while(keepRunning){
54          try {
55             synchronized (aliveQueue) {
56                while(aliveQueue.isEmpty()){
57                   aliveQueue.wait();
58                }
59             }
60
61             PduInterface pdu = aliveQueue.peek();
62             long timeToDie = pdu.getAliveTime();
63             long sleepTime = timeToDie - System.currentTimeMillis()
                  ;
64             if(sleepTime > 0){
65                sleep(sleepTime);
66             }
67
68                while(timeToDie <= System.currentTimeMillis()){
69                   synchronized (pduIdentifiers) {
70                      if(pduIdentifiers.containsKey(pdu.
                           getSequenceNumber())){
71                         if(resetTimer(pdu)){
72                            aliveQueue.remove();
73                            sender.resendPDU(pdu,pduIdentifiers.get(
                                 pdu.getSequenceNumber()));
74                            pdu.setTimer();
75                            aliveQueue.addLast(pdu);
76                         }
77
78                      } else {
79                         aliveQueue.remove();
80                      }
81                   }
82                   pdu = aliveQueue.peek();
83                   timeToDie = pdu.getAliveTime();
84                }
85          } catch (InterruptedException e) {
86             //thread stopped
87          }
88       }
89
90    }
91
92    public void stopThread(){
```

```
93          keepRunning = false;
94          this.interrupt();
95      }
96
97      /**
98       * Method used whenever a message in the timer has received an
             corresponding ACK message
99       * @param sequenceNumber is the message to be removed from the
             timer
100      * @return returns true if the pdu successfully where removed as
             defined by the remove method of a HashSet
101      */
102     public boolean removePDU(int sequenceNumber){
103         synchronized (pduIdentifiers) {
104             if ((pduIdentifiers.remove(sequenceNumber)) != null){
105                 return true;
106             }
107         }
108          return false;
109     }
110
111     public void removeAllPDUForContact(int contactID){
112         synchronized (pduIdentifiers) {
113             for(int id: pduIdentifiers.keySet()){
114                 if(id == contactID){
115                     pduIdentifiers.remove(id);
116                 }
117             }
118         }
119     }
120
121     private boolean resendChatReq(ChatRequest chatReq){
122         if(chatManager.chatExists(chatReq.getChatID())){
123             for(int id: chatReq.getFriends().keySet()){
124                  if(!contactManager.isContactOnline(id)){
125                     return false;
126                 }
127             }
128             return true;
129         }
130         return false;
131     }
132
133     private boolean resendMsg(Msg msg){
134         if(contactManager.isContactOnline(pduIdentifiers.get(msg.
                 getSequenceNumber()))){
135             return true;
136         }
137         return false;
138     }
139
140     private boolean resetTimer(PduInterface pdu){
141         switch (pdu.getPduType()) {
142
143         case Constants.PDU_CHAT_REQUEST:
```

```
144             if(!resendChatReq((ChatRequest) pdu)){
145                 return false;
146             }
147             break;
148 
149         case Constants.PDU_HELLO:
150             return true;
151 
152         case Constants.PDU_MSG:
153             if(!resendMsg((Msg) pdu)){
154                 chatManager.notifyTextNotSent((Msg)pdu);
155                 return false;
156             }
157             break;
158 
159         default:
160             return false;
161 
162         }
163         return true;
164     }
165 }
```

F.2 View

F.2.0.50 AddFriend.java

```
1  package android.TextMessenger.view;
2 
3  import android.TextMessenger.control.ButtonListner;
4  import android.app.Activity;
5  import android.os.Bundle;
6  import android.widget.Button;
7  import android.widget.EditText;
8 
9 
10 public class AddFriend extends Activity {
11     private Button add;
12     private ButtonListner listener;
13 
14     public void onCreate(Bundle savedInstanceState) {
15         super.onCreate(savedInstanceState);
16         setContentView(R.layout.add_friend);
17 
18         add = (Button) findViewById(R.id.find);
19         listener = new ButtonListner(this);
20         add.setOnClickListener(listener);
21 
22     }
```

```
23
24      public String getContactId(){
25          EditText name = (EditText) findViewById(R.id.friendID);
26          return name.getText().toString();
27      }
28
29  }
```

F.2.0.51 AddChat.java

```
1   package android.TextMessenger.view;
2
3   import java.util.HashMap;
4
5   import android.R.integer;
6   import android.TextMessenger.control.ButtonListner;
7   import android.TextMessenger.model.ClassConstants;
8   import android.TextMessenger.model.ContactManager;
9   import android.app.Activity;
10  import android.os.Bundle;
11  import android.widget.Button;
12  import android.widget.EditText;
13
14
15  public class AddChat extends Activity {
16      private Button add;
17      private ButtonListner listener;
18
19      public void onCreate(Bundle savedInstanceState) {
20          super.onCreate(savedInstanceState);
21          setContentView(R.layout.add_chat);
22
23          add = (Button) findViewById(R.id.startChat);
24          listener = new ButtonListner(this);
25          add.setOnClickListener(listener);
26
27      }
28
29      public HashMap<Integer, String> getContactIds(){
30          HashMap<Integer, String> contacts = new HashMap<Integer,
                String>();
31          ContactManager contactManager = ClassConstants.getInstance().
                getContactManager();
32
33          String contactID = ((EditText) findViewById(R.id.frindToChat1
                )).getText().toString();
34          if(contactID != ""){
35              contacts.put(Integer.parseInt(contactID), contactManager.
                    getContactDisplayName(Integer.parseInt(contactID)));
36          }
37
```

```
38          contactID = ((EditText) findViewById(R.id.frindToChat2)).
                getText().toString();
39          if(contactID != ""){
40             contacts.put(Integer.parseInt(contactID), contactManager.
                   getContactDisplayName(Integer.parseInt(contactID)));
41          }
42
43          contactID = ((EditText) findViewById(R.id.frindToChat3)).
                getText().toString();
44          if(contactID != ""){
45             contacts.put(Integer.parseInt(contactID), contactManager.
                   getContactDisplayName(Integer.parseInt(contactID)));
46          }
47
48          contactID = ((EditText) findViewById(R.id.frindToChat4)).
                getText().toString();
49          if(contactID != ""){
50             contacts.put(Integer.parseInt(contactID), contactManager.
                   getContactDisplayName(Integer.parseInt(contactID)));
51          }
52
53          return contacts;
54       }
55
56    }
```

F.2.0.52 ChatScreen.java

```
1   package android.TextMessenger.view;
2
3
4   import java.util.Observable;
5   import java.util.Observer;
6
7   import android.TextMessenger.control.ButtonListner;
8   import android.TextMessenger.exceptions.ContactOfflineException;
9   import android.TextMessenger.model.Chat;
10  import android.TextMessenger.model.ChatManager;
11  import android.TextMessenger.model.ClassConstants;
12  import android.TextMessenger.model.ObjToObsever;
13  import android.app.Activity;
14  import android.os.Bundle;
15  import android.widget.Button;
16  import android.widget.EditText;
17
18
19  public class ChatScreen extends Activity implements Observer{
20     private EditText messageText;
21     private EditText messageHistoryText;
22     private Button sendMessageButton;
23     private Chat chat;
```

```
24      private int chatID;
25      private ButtonListner listener;
26      private ChatManager chatManager;
27
28      public void onCreate(Bundle savedInstanceState) {
29          // TODO Auto-generated method stub
30          super.onCreate(savedInstanceState);
31          setContentView(R.layout.messaging_screen);
32
33          chatID = getIntent().getIntExtra("chatID", 0);
34          chatManager = ClassConstants.getInstance().getChatmanager();
35
36          messageHistoryText = (EditText) findViewById(R.id.
                messageHistory);
37          messageText = (EditText) findViewById(R.id.message);
38          messageText.requestFocus();
39          sendMessageButton = (Button) findViewById(R.id.
                sendMessageButton);
40          listener = new ButtonListner(this);
41          sendMessageButton.setOnClickListener(listener);
42
43          chat = ClassConstants.getInstance().getChatmanager().getChat(
                chatID);
44
45          if(chat == null){
46              //close
47          }
48          chat.addObserver(this);
49          chat.getTextHistory();
50      }
51
52      @Override
53      protected void onResume() {
54          super.onResume();
55          chat.addObserver(this);
56          chat.getTextHistory();
57      }
58
59       @Override
60          protected void onStop() {
61              // TODO Auto-generated method stub
62              super.onStop();
63              chat.deleteObserver(this);
64              chat.setHaveBeenViewde();
65          }
66
67      public void addMessageToHistory(String message) {
68          if (message != null) {
69              messageHistoryText.append(message);
70          }
71      }
72
73      public void sendMessage(){
74          try {
```

```
75            chatManager.sendText(messageText.getText().toString(),
                  chatID);
76            messageText.setText("");
77            messageText.requestFocus();
78        } catch (ContactOfflineException e) {
79            //Close chat
80        }
81    }
82
83     @Override
84       public void update(Observable observable, Object arg) {
85        ObjToObsever msg = (ObjToObsever)arg;
86          int type = msg.getMessageType();
87          switch (type) {
88
89          case ObserverConst.TEXT_RECIVED:
90             addMessageToHistory((String)msg.getContainedData());
91             break;
92          case ObserverConst.REMOVE_CHAT:
93             this.finish();
94             break;
95          default:
96             break;
97
98
99          }
100
101       }
102
103
104 }
```

F.2.0.53 ChatsView.java

```
1  package android.TextMessenger.view;
2
3  import java.util.ArrayList;
4  import java.util.HashMap;
5  import java.util.Observable;
6  import java.util.Observer;
7
8  import android.TextMessenger.control.ButtonListner;
9  import android.TextMessenger.control.ItemClickListener;
10 import android.TextMessenger.model.Chat;
11 import android.TextMessenger.model.ChatManager;
12 import android.TextMessenger.model.ClassConstants;
13 import android.TextMessenger.model.ObjToObsever;
14 import android.app.ListActivity;
15 import android.content.Intent;
16 import android.os.Bundle;
17 import android.os.Handler;
```

```
18 import android.os.Message;
19 import android.view.LayoutInflater;
20 import android.view.View;
21 import android.view.ViewGroup;
22 import android.widget.ArrayAdapter;
23 import android.widget.Button;
24 import android.widget.ImageView;
25 import android.widget.TextView;
26
27 public class ChatsView extends ListActivity implements Observer {
28    private TextView selection;
29    private ArrayList<String> chats;
30    private ChatManager chatManager;
31    private IconicAdapter ica;
32    private ItemClickListener itemlisterner;
33    private Button addChat;
34    private Handler handler;
35
36    public void onCreate(Bundle savedInstanceState) {
37       super.onCreate(savedInstanceState);
38
39       chats = new ArrayList<String>();
40       setContentView(R.layout.chats);
41       ica = new IconicAdapter();
42       setListAdapter(ica);
43       selection = (TextView) findViewById(R.id.selection1);
44       chatManager = ClassConstants.getInstance().getChatmanager();
45       chatManager.addObserver(this);
46       addChat = (Button) findViewById(R.id.addchat);
47       ButtonListner l = new ButtonListner(this);
48       addChat.setOnClickListener(l);
49
50       itemlisterner = new ItemClickListener(this,2);
51       getListView().setOnItemClickListener(itemlisterner);
52
53
54       handler = new Handler() {
55          @SuppressWarnings("unchecked")
56          @Override
57          public void handleMessage(Message msg) {
58             if(msg.getData().getInt("add")== 1){
59                ica.add(msg
60                      .getData().getString("msg"));
61             }
62             else{
63                ica.remove(msg
64                      .getData().getString("msg"));
65             }
66             ica.notifyDataSetChanged();
67          }
68       };
69
70    }
71
72    @SuppressWarnings("unchecked")
```

```
73      @Override
74      public void update(Observable observable, Object arg) {
75         Chat c;
76
77         ObjToObsever msg = (ObjToObsever) arg;
78         int type = msg.getMessageType();
79         switch (type) {
80
81         case ObserverConst.TEXT_RECIVED:
82            ica.notifyDataSetChanged();
83            break;
84         case ObserverConst.NEW_CHAT:
85            Message m = new Message();
86            Bundle b = new Bundle();
87            c = (Chat) msg.getContainedData();
88            b.putString("msg", c.getID()+"");
89            b.putInt("add", 1);
90            m.setData(b);
91            handler.sendMessage(m);
92            break;
93         case ObserverConst.REMOVE_CHAT:
94            Message m1 = new Message();
95            Bundle b1 = new Bundle();
96            c = (Chat) msg.getContainedData();
97            b1.putString("msg", c.getID()+"");
98            b1.putInt("add", 0);
99            m1.setData(b1);
100           handler.sendMessage(m1);
101           break;
102
103        default:
104           break;
105        }
106     }
107
108        public void openChat(int possion){
109           Intent i = new Intent(this, ChatScreen.class);
110           int chatID = Integer.parseInt(chats.get(possion));
111           i.putExtra("chatID", chatID);
112           startActivityForResult(i, 0);
113        }
114
115        public void newChat(){
116           Intent i = new Intent(this, AddChat.class);
117           startActivityForResult(i, 0);
118        }
119
120        @SuppressWarnings("unchecked")
121        class IconicAdapter extends ArrayAdapter {
122           IconicAdapter() {
123              super(ChatsView.this, R.layout.row, chats);
124           }
125
126           public View getView(int position, View convertView,
               ViewGroup parent) {
```

```
127             LayoutInflater inflater = getLayoutInflater();
128             View row = inflater.inflate(R.layout.row, parent, false
                    );
129             TextView label = (TextView) row.findViewById(R.id.label
                    );
130
131             label.setText(chats.get(position));
132             ImageView icon = (ImageView) row.findViewById(R.id.icon
                    );
133             if((chatManager.getChat(Integer.parseInt(chats.get(
                    position))).isTherNewMsg())){
134                icon.setImageResource(R.drawable.svambe_bob);
135             }
136             else{
137                icon.setImageResource(R.drawable.icon);
138             }
139             return (row);
140          }
141       }
142    }
```

F.2.0.54 Connect.java

```
1  package android.TextMessenger.view;
2
3  import java.net.BindException;
4  import java.net.SocketException;
5  import java.net.UnknownHostException;
6
7  import adhoc.aodv.Node;
8  import adhoc.aodv.exception.InvalidNodeAddressException;
9  import adhoc.setup.AdhocManager;
10 import adhoc.setup.PhoneType;
11 import android.TextMessenger.control.ButtonListner;
12 import android.TextMessenger.model.ChatManager;
13 import android.app.Activity;
14 import android.content.Context;
15 import android.content.Intent;
16 import android.net.wifi.WifiManager;
17 import android.os.Build;
18 import android.os.Bundle;
19 import android.util.Log;
20 import android.widget.Button;
21 import android.widget.EditText;
22
23 public class Connect extends Activity {
24    private Button connect;
25    private ButtonListner listener;
26    AdhocManager adHoc;
27    String ip;
28    int phoneType;
```

```
29
30      public void onCreate(Bundle savedInstanceState) {
31          super.onCreate(savedInstanceState);
32          setContentView(R.layout.connect);
33          listener = new ButtonListner(this);
34          connect = (Button) findViewById(R.id.connectButton);
35          connect.setOnClickListener(listener);
36
37
38          // TODO Start ADHOC NETWORK
39      }
40
41      private int getPhoneType(){
42          String model = Build.MODEL;
43
44          if(model.contains("Nexus")){
45              return PhoneType.NEXUS_ONE;
46          }
47          if(model.contains("Hero")){
48              return PhoneType.HTC_HERO;
49          }
50          return -1;
51      }
52
53      public static native int runCommand(String command);
54
55      static {
56          System.loadLibrary("adhocsetup");
57      }
58
59      /**
60       * When connect is clicked, a ad-hoc network is startet
61       */
62      public void clickConnect() {
63          EditText name = (EditText) findViewById(R.id.displayName);
64          String myDisplayName = name.getText().toString();
65          if (myDisplayName == "") {
66              return;
67          }
68          try {
69              int myContactID = nameToID(myDisplayName);
70              ip = "192.168.2." + myContactID;
71              phoneType = getPhoneType();
72              if(phoneType == -1){
73                  Log.d("PHONE", "No such phoneType");
74                  return;
75              }
76
77
78              WifiManager wifi = (WifiManager) getSystemService(Context.
                    WIFI_SERVICE);
79              adHoc = new AdhocManager(this, wifi);
80              //Starting an ad-hoc network
81              int result = Connect.runCommand("su -c \""+"
                    startstopadhoc start "+phoneType+" "+ip+"\"");
```

```
82            Log.d("RESULTAT", ""+result);
83            //Starting the routing protocol
84            Node node = new Node(myContactID);
85
86            ChatManager chatManager = new ChatManager(myDisplayName,
                  myContactID, node);
87
88            node.startThread();
89
90
91
92            Intent i = new Intent(this, TabView.class);
93            startActivity(i);
94
95        } catch (BindException e) {
96            e.printStackTrace();
97        } catch (InvalidNodeAddressException e) {
98            e.printStackTrace();
99        } catch (SocketException e) {
100           e.printStackTrace();
101       } catch (UnknownHostException e) {
102           e.printStackTrace();
103       }
104       Log.d("DEBUG", "Node startet ");
105   }
106
107   // FIXME HVORDAN SKAL MAN Faa TILDELT ID
108   private int nameToID(String displayName) {
109       return 4; // (int)(Math.random()*100);
110   }
111
112   @Override
113   protected void onResume() {
114       super.onResume();
115       // FIXME go to tabView
116   }
117
118   @Override
119   protected void onDestroy() {
120       super.onDestroy();
121       if (adHoc != null) {
122           runCommand("su -c \"" + " startstopadhoc stop " +
                      phoneType + " " + ip + "\"");
123       }
124   }
125 }
```

F.2.0.55 ContactsView.java

```
1  package android.TextMessenger.view;
2
```

```
3  import java.util.ArrayList;
4  import java.util.Observable;
5  import java.util.Observer;
6
7  import android.TextMessenger.control.ButtonListner;
8  import android.TextMessenger.control.ItemClickListener;
9  import android.TextMessenger.model.ClassConstants;
10 import android.TextMessenger.model.Contact;
11 import android.TextMessenger.model.ContactManager;
12 import android.TextMessenger.model.ObjToObsever;
13 import android.app.ListActivity;
14 import android.content.Intent;
15 import android.os.Bundle;
16 import android.view.LayoutInflater;
17 import android.view.View;
18 import android.view.ViewGroup;
19 import android.widget.ArrayAdapter;
20 import android.widget.Button;
21 import android.widget.ImageView;
22
23 import android.widget.TextView;
24
25 public class ContactsView extends ListActivity implements Observer
       {
26    private TextView selection;
27    private ContactManager contactManager;
28    //String[] items = {};
29    private ItemClickListener itemlisterner;
30    private IconicAdapter ica ;
31    private ArrayList<String> olga;
32    private Button addContact;
33    private ButtonListner listner;
34
35
36    public void onCreate(Bundle savedInstanceState) {
37       super.onCreate(savedInstanceState);
38
39       olga = new ArrayList<String>();
40       setContentView(R.layout.contacts);
41       ica = new IconicAdapter();
42       setListAdapter(ica);
43       itemlisterner = new ItemClickListener(this,1);
44       getListView().setOnItemClickListener(itemlisterner);
45       selection = (TextView) findViewById(R.id.selection);
46       addContact = (Button) findViewById(R.id.addcontact);
47       listner = new ButtonListner(this);
48       addContact.setOnClickListener(listner);
49
50
51       contactManager = ClassConstants.getInstance().
             getContactManager();
52       contactManager.addObserver(this);
53
54 //    contactManager.addContact(22, "22olga", false);
55 //    contactManager.addContact(33, "33olga", false);
```

```
56 //     contactManager.addContact(44, "44olga", false);
57 //     contactManager.addContact(55, "55olga", false);
58 //
59 //     contactManager.setContactOnlineStatus(44, true);
60 //     contactManager.setContactOnlineStatus(33, true);
61
62     }
63
64
65
66     @SuppressWarnings("unchecked")
67     @Override
68     public void update(Observable observable, Object arg) {
69         Contact c;
70         ObjToObsever msg = (ObjToObsever) arg;
71         int type = msg.getMessageType();
72         switch (type) {
73
74         case ObserverConst.CONTACT_ONLINE_STATUS_CHANGED:
75             ica.notifyDataSetChanged();
76             break;
77         case ObserverConst.NEW_CONTACT:
78             c = (Contact)msg.getContainedData();
79             ica.add(c.getID()+"");
80             ica.notifyDataSetChanged();
81             break;
82         case ObserverConst.REMOVE_CONTACT:
83             c = (Contact)msg.getContainedData();
84             ica.remove(c.getID()+"");
85             ica.notifyDataSetChanged();
86             break;
87         default:
88             break;
89         }
90     }
91
92     public void addContact(){
93         Intent i = new Intent(this, AddFriend.class);
94         startActivityForResult(i, 0);
95     }
96
97     @SuppressWarnings("unchecked")
98     class IconicAdapter extends ArrayAdapter {
99
100        IconicAdapter() {
101        // super(ContactsView.this, R.layout.row, /*(En eller anden
               liste)*/ );
102            super(ContactsView.this, R.layout.row, olga);
103
104        }
105
106        public View getView(int position, View convertView, ViewGroup
               parent) {
107            LayoutInflater inflater = getLayoutInflater();
108            View row = inflater.inflate(R.layout.row, parent, false);
```

```
109             //row.setOnClickListener(listener);
110             TextView label = (TextView) row.findViewById(R.id.label);
111
112             label.setText(contactManager.getContactDisplayName(Integer
                    .parseInt(olga.get(position))));
113
114             ImageView icon = (ImageView) row.findViewById(R.id.icon);
115
116             if (contactManager.isContactOnline(Integer.parseInt(olga.
                    get(position)))) {
117                 icon.setImageResource(R.drawable.svambe_bob);
118             } else {
119                 icon.setImageResource(R.drawable.icon);
120             }
121
122             return (row);
123         }
124     }
125 }
```

F.2.0.56 ObserverConst.java

```
1  package android.TextMessenger.view;
2
3  public interface ObserverConst {
4
5      public static final int NEW_CONTACT = 1;
6      public static final int CONTACT_ONLINE_STATUS_CHANGED = 2;
7      public static final int REMOVE_CONTACT = 4;
8      public static final int REMOVE_CHAT = 5;
9      public static final int NEW_CHAT = 6;
10     public static final int TEXT_RECIVED = 7;
11     public static final int TEXT_NOT_SENT = 8;
12
13
14 }
```

F.2.0.57 TabView.java

```
1  package android.TextMessenger.view;
2
3
4  import android.app.TabActivity;
5  import android.content.Intent;
6  import android.content.res.Resources;
7  import android.os.Bundle;
8  import android.widget.TabHost;
```

```
9
10 public class TabView extends TabActivity {
11
12    public void onCreate(Bundle savedInstanceState) {
13        super.onCreate(savedInstanceState);
14        setContentView(R.layout.tab);
15
16        Resources res = getResources(); // Resource object to get
              Drawables
17        TabHost tabHost = getTabHost();  // The activity TabHost
18        TabHost.TabSpec spec;  // Resusable TabSpec for each tab
19        Intent intent;  // Reusable Intent for each tab
20
21        // Create an Intent to launch an Activity for the tab (to be
              reused)
22        intent = new Intent().setClass(this, ContactsView.class);
23        // Initialize a TabSpec for each tab and add it to the
              TabHost
24        spec = tabHost.newTabSpec("contacts").setIndicator("Contacts
              ",res.getDrawable(R.drawable.ic_tab_contacts)).
              setContent(intent);
25        tabHost.addTab(spec);
26
27        // Do the same for the other tabs
28        intent = new Intent().setClass(this, ChatsView.class);
29        spec = tabHost.newTabSpec("chats").setIndicator("Chats", res
              .getDrawable(R.drawable.ic_tab_artists)).setContent(
              intent);
30        tabHost.addTab(spec);
31
32        tabHost.setCurrentTab(2);
33    }
34
35 }
```

F.3 Control

F.3.0.58 .java

```
1  package android.TextMessenger.control;
2
3  import android.TextMessenger.model.ChatManager;
4  import android.TextMessenger.model.ClassConstants;
5  import android.TextMessenger.view.AddChat;
6  import android.TextMessenger.view.AddFriend;
7  import android.TextMessenger.view.ChatScreen;
8  import android.TextMessenger.view.ChatsView;
9  import android.TextMessenger.view.Connect;
10 import android.TextMessenger.view.ContactsView;
```

```
11 import android.TextMessenger.view.R;
12 import android.app.Activity;
13 import android.util.Log;
14 import android.view.View;
15 import android.view.View.OnClickListener;
16
17 public class ButtonListner implements OnClickListener{
18    Activity parent;
19
20    public ButtonListner(Activity parent) {
21       this.parent = parent;
22    }
23
24    @Override
25    public void onClick(View v) {
26       if(v.equals(parent.findViewById(R.id.connectButton))){
27          Log.d("KLIK", "DER BLEV KLIKKET");
28          Connect c = (Connect)parent;
29          c.clickConnect();
30
31       }
32       else if(v.equals(parent.findViewById(R.id.sendMessageButton))
             ){
33          ChatScreen chatS = (ChatScreen)parent;
34          chatS.sendMessage();
35       }
36       else if (v.equals(parent.findViewById(R.id.addcontact))){
37          ((ContactsView) parent).addContact();
38       }
39       else if (v.equals(parent.findViewById(R.id.addchat))){
40          ((ChatsView) parent).newChat();
41       }
42       else if (v.equals(parent.findViewById(R.id.find))){
43          AddFriend add = (AddFriend) parent;
44          ClassConstants.getInstance().getContactManager().
                addContact(Integer.parseInt(add.getContactId()), "
                Searching", true);
45          add.finish();
46       }
47       else if (v.equals(parent.findViewById(R.id.startChat))){
48          AddChat addChat = (AddChat) parent;
49          ClassConstants.getInstance().getChatmanager().newChat(
                addChat.getContactIds());
50          addChat.finish();
51       }
52    }
53 }
```

F.3.0.59 .java

```
1 package android.TextMessenger.control;
```

```
2
3  import android.TextMessenger.model.ClassConstants;
4  import android.TextMessenger.view.ChatsView;
5  import android.TextMessenger.view.ContactsView;
6  import android.app.Activity;
7  import android.view.View;
8  import android.widget.AdapterView;
9  import android.widget.AdapterView.OnItemClickListener;
10
11 public class ItemClickListener implements OnItemClickListener{
12    int parrentType;
13    Activity parrent;
14    public ItemClickListener(Activity parrent, int parrentType){
15       this.parrentType = parrentType;
16       this.parrent = parrent;
17    }
18
19    @Override
20    public void onItemClick(AdapterView<?> arg0, View arg1, int
          position, long arg3) {
21
22
23       if(parrentType == 1){
24          ClassConstants.getInstance().getContactManager().
                addContact(position, "olgabolga"+position, false);
25       }
26       else if(parrentType == 2){
27          ((ChatsView)parrent).openChat(position);
28       }
29    }
30
31 }
```

F.4 Exceptions

F.4.0.60 .java

```
1  package android.TextMessenger.exceptions;
2
3  public class ContactOfflineException extends Exception{
4
5     private static final long serialVersionUID = 1L;
6
7     public ContactOfflineException() {
8     }
9
10    public ContactOfflineException(String message) {
11       super(message);
12    }
```

```
13 }
```

Bibliography

[1] Open source project - Android-WiFi-Tether
http://code.google.com/p/android-wifi-tether/.

[2] Open Handset Alliance
http://www.openhandsetalliance.com/.

[3] Android Software Stack
http://developer.android.com/guide/basics/what-is-android.html.

[4] Android Bluetooth API
http://developer.android.com/reference/android/bluetooth/package-descr.html.

[5] Java unit-test framework
http://www.junit.org/.

[6] Wireshark
http://www.wireshark.org/.

[7] Monis Akhlaq, M. Noman Jafri, Muzammil A. Khan, and Baber Aslam. Addressing security concerns of data exchange in aodv protocol. Technical report, World Academy of Science, Engineering and Technology, 2006.

[8] T. Clausen and P. Jacquet, editors. *Optimized Link State Routing Protocol (OLSR)*. RFC Editor, United States, October 2003.

[9] George F. Coulouris, Jean Dollimore, and Tim Kindberg. *Distributed systems: concepts and design*. Addison-Wesley Longman Publishing Co., Inc., 4. edition edition, 2005.

[10] D. Johnson, Y. Hu, and D. Maltz. *The Dynamic Source Routing Protocol (DSR) for Mobile Ad Hoc Networks.* February 2007. RFC 4728.

[11] Young-Bae Ko and Nitin H. Vaidya. Location-aided routing (lar) in mobile ad hoc networks. *Wirel. Netw.*, 6(4):307–321, 2000.

[12] Prasant Mohapathra and Srikanth V. Krishnamurthy. *Ad Hoc Networks: Technologies and Protocols.* Springer, 2005.

[13] Shree Murthy and J. J. Garcia-Luna-Aceves. An efficient routing protocol for wireless networks. *Mob. Netw. Appl.*, 1(2):183–197, 1996.

[14] R. Ogier, F. Templin, and M. Lewis. *Topology Dissemination Based on Reverse-Path Forwarding (TBRPF).* RFC Editor, February 2004.

[15] Charles E. Perkins, E. Belding-Royer, and S. Das. *Ad hoc On-Demand Distance Vector (AODV) Routing.* July 2003. RFC 3561.

[16] Charles E. Perkins and Pravin Bhagwat. *Highly Dynamic Destination-Sequenced Distance-Vector Routing (DSDV) for Mobile Computers.* Association of Computing, 1994.

[17] Charles E. Perkins and Elizabeth M. Royer. *Ad-hoc On-Demand Distance Vector Routing.* IEEE Computer Society, February 1999. Proceedings of the 2nd IEEE Workshop on Mobile Computing Systems and Applications.

[18] Khoi Anh Phan, Zahir Tari, and Peter Bertok. A benchmark on soap's transport protocols performance for mobile applications. In *SAC '06: Proceedings of the 2006 ACM symposium on Applied computing*, pages 1139–1144, New York, NY, USA, 2006. ACM.

[19] Robin Sharp. *The Poor Man's Guide to Computer Networks and their Applications.* October 2007.